MASTERS THESIS

AN LWE-BASED ATTRIBUTE BASED ENCRYPTION FRAMEWORK: POST-QUANTUM FINE-GRAINED ACCESS CONTROL FOR INSTITUTIONAL SYSTEMS

By

Avni Marwah
SAU/AM/MSC/2024/14

Under the supervision of

Prof. Jagdish Chand Bansal

Submitted in partial fulfillment of the requirements for the award of the degree of

Master of Science in Applied Mathematics

Department of Mathematics

Faculty of Mathematical Sciences

South Asian University

New Delhi-110068, India

May, 2026

DECLARATION

This is to certify that the master's thesis entitled **An LWE-Based Attribute Based Encryption Framework: Post-Quantum Fine-Grained Access Control for Institutional Systems**, which is being submitted for the award of the degree of **Master of Science in Applied Mathematics** to the **South Asian University, New Delhi**, is a record of the theoretical and computational survey with the insight to work out some new problems, under the supervision of **Prof. Jagdish Chand Bansal**. The results embodied in the project have not been submitted to any other university or institution for the award of any degree or diploma.

Place: South Asian University
Date: May, 2026

Avni Marwah

CERTIFICATE

This is to certify that the master's thesis entitled **An LWE-Based Attribute Based Encryption Framework: Post-Quantum Fine-Grained Access Control for Institutional Systems** submitted by **Avni Marwah** to the Department of Mathematics, Faculty of Mathematical Sciences, South Asian University, New Delhi, in partial fulfillment of the requirements for the award of the degree of **Master of Science in Applied Mathematics**, is an authentic record of the survey of research work carried out by her under my supervision and guidance.

Prof. Jagdish Chand Bansal
(Supervisor)
Professor
Department of Mathematics
South Asian University
New Delhi-110068, India

Prof. Pankaj Jain
Dean, FMS

ACKNOWLEDGMENT

This thesis did not come together in isolation and I would not want anyone reading it to think that it did.

My deepest gratitude goes to **Mr. Harshdeep Singh**, Scientist D, SAG Lab, DRDO, Ministry of Defence, who introduced me to these topics. Honestly, without that introduction, I would not have found a subject that excites me so much or one that I want to continue researching long after this thesis is submitted. He was generous with his knowledge and his time in ways I did not expect, and I hope this work reflects how seriously I took what he shared with me.

I owe an equal debt to my supervisor, **Prof. Jagdish Chand Bansal**, South Asian University, New Delhi, for his patience and the academic rigor he brought to every conversation we had. He gave me the freedom to explore difficult material at my own pace, while always making sure I was on the right track. I am grateful for his time and trust.

I would also like to thank **Dr. Pankaj Jain**, Dean, Faculty of Mathematical Sciences, South Asian University, for making this thesis writing opportunity available and for the environment he has helped build within the faculty.

To my mom and dad: thank you for supporting me through every difficult stretch of this process, academic or otherwise. You never asked me once to explain what I was working on to make sure it was worth the effort. You just trusted me, and that meant more than I can properly say here.

And to my best friend Kartikeya, who was always one call or text away, no matter the hour or how scattered my thoughts were: thank you for being there. Some of the clearest thinking in this thesis happened right after a conversation with you, even when the conversation had nothing to do with cryptography.

Avni Marwah

Abstract

Widely deployed public-key cryptosystems, including RSA and Attribute-Based Encryption built on pairing-based assumptions, derive their security from the computational hardness of integer factorization and the discrete logarithm problem. Shor's algorithm solves both problems efficiently on a sufficiently large quantum computer, and recent resource estimates place the factorization of 2048-bit RSA keys within reach of fewer than one million physical qubits over roughly one week, a threshold reduced by more than an order of magnitude from earlier projections through advances in approximate residue arithmetic and quantum error correction. Systems built on these classical assumptions are therefore already at risk through harvest-now, decrypt-later attacks, in which adversaries collect encrypted data today and defer decryption until quantum hardware matures.

This thesis traces the transition from classical public-key cryptography to quantum-resistant alternatives, with rigorous mathematics developed at every step. RSA is derived from its number-theoretic foundations and accompanied by a treatment of primality testing, covering the Fermat, Miller–Rabin, and AKS tests, and of factorization algorithms, including Fermat's method, Pollard's rho algorithm, and the General Number Field Sieve, together with a formal proof that the RSA encryption primitive is a bijection. Shor's algorithm is analyzed in detail, covering its reduction of factorization to period-finding and the role of the Quantum Fourier Transform in that reduction. Two families of post-quantum primitives are then constructed from first principles: code-based cryptography via the McEliece cryptosystem over binary Goppa codes, whose security rests on the NP-hardness of syndrome decoding; and lattice-based cryptography via the Learning With Errors problem and its ring-structured and module-structured variants, culminating in the Module Learning With Errors-based Key Encapsulation Mechanism known as Kyber, standardized by the National Institute of Standards and Technology in 2019.

The central contribution is a complete treatment of lattice-based Key-Policy Attribute-Based Encryption through the construction of Boneh, Gentry, Gorbunov, Halevi, Nikolaenko, Segev, Vaikuntanathan, and Vinayagamurthy, published at EUROCRYPT 2014, which was the first scheme to support arbitrary Boolean circuit access policies under standard Learning With Errors assumptions. The gadget matrix, its binary decomposition operator, tensor product attribute encodings, and the homomorphic evaluation algorithms for circuits and their transposes are developed in full, together with a selective security proof under the Learning With Errors assumption.

The scheme is applied to a university document access control system comprising twenty users, ten institutional attributes, and a depth-four Boolean access policy. Correctness is verified through a hand-computed numerical example over the integers modulo seventeen

and a Python simulation across all 680 message bits, confirming that authorised users recover the plaintext exactly while unauthorised users, including near-miss cases that satisfy all but one policy condition, receive cryptographically uncorrelated output.

Contents

List of Tables

List of Figures

CHAPTER 1

INTRODUCTION

The security of modern digital infrastructure rests on a small collection of computational assumptions. Two of these, the hardness of factoring large integers and the hardness of computing discrete logarithms in cyclic groups, underpin virtually every public-key cryptosystem in widespread deployment today, including RSA [2], ElGamal [3], and the classical pairing-based constructions used in Attribute-Based Encryption (ABE). These assumptions have withstood nearly five decades of classical cryptanalysis, but they share a structural vulnerability: both problems admit efficient quantum algorithms. Shor's 1994 result [12] showed that a sufficiently large quantum computer can factor an n-bit integer and compute discrete logarithms in $O(n^2 \log n \log \log n)$ operations, polynomial in the input size. Recent resource estimates [14] place RSA-2048 within reach of fewer than one million physical qubits running for approximately one week, a threshold that is no longer beyond the horizon of near-term engineering.

This thesis addresses the following question: what rigorous mathematical foundation supports the transition from the classical cryptographic infrastructure to quantum-resistant alternatives, and how far can that transition be carried within a complete, verifiable construction. The answer is developed in two stages. The first stage, covered in Chapters 2 through 4, traces the full lifecycle of RSA: its number-theoretic foundations, the primality and factorization algorithms that make it practical, and the quantum algorithm that breaks it. The second stage, covered in Chapters 5 and 6, presents lattice-based Key-Policy Attribute-Based Encryption via the BGG+14 construction [50] as a concrete post-quantum replacement, culminating in a hand-computed and Python-verified implementation over a university access control scenario.

1.1 The Problem Being Solved

A public-key cryptosystem allows two parties to communicate securely over an open channel without prior shared secrets. Its security rests on a computational asymmetry:

one direction of a mathematical operation (encryption, or key derivation) is efficient, while the reverse (decryption without the private key) is intractable. Classical systems achieve this asymmetry through number-theoretic problems—integer factorization in RSA, discrete logarithms in ElGamal and elliptic-curve schemes—that are hard for classical computers but not for quantum ones.

Attribute-Based Encryption [44, 45] extends public-key encryption to support fine-grained, policy-based access control: a ciphertext can be decrypted only by users whose attributes satisfy a specified access structure, without the encryptor knowing the recipient's identity in advance. Classical ABE constructions rely on bilinear pairings over elliptic curves, whose security is again grounded in the discrete logarithm problem and is equally vulnerable to Shor's algorithm. A quantum-resistant ABE scheme must therefore replace both the underlying hardness assumption and the algebraic mechanism that enforces the access policy.

1.2 Contributions of This Thesis

The main contributions of this work are as follows.

1. **A self-contained treatment of RSA and its quantum vulnerability.** Chapter 2 develops the number-theoretic foundations of RSA from first principles, including proofs of correctness via Euler's theorem, a comparative analysis of primality-testing algorithms (Fermat, Miller-Rabin, AKS), and a survey of classical factorization methods culminating in the General Number Field Sieve. Chapter 3 provides a detailed analysis of Shor's algorithm, its reduction of factorization to period-finding, and current physical qubit requirements for attacking RSA-2048.
2. **A first-principles development of two post-quantum families.** Chapter 4 builds code-based and lattice-based cryptography from the ground up. Code-based security is grounded in the NP-hardness of syndrome decoding, realized through the McEliece cryptosystem with binary Goppa codes. Lattice-based security is developed through SVP, CVP, LWE, Ring-LWE, and Module-LWE, leading to the ML-KEM (Kyber) key encapsulation mechanism standardized by NIST in 2019.
3. **A complete treatment of BGG+14 KP-ABE.** Chapter 5 develops the gadget matrix $\mathbf{G}_n$, the tensor product attribute encoding $\mathbf{x}^\top \otimes \mathbf{G}_n$, the homomorphic evaluation algorithms EvalF and EvalFX, and the key identity

$$\left(\mathbf{B} - \mathbf{x}^\top \otimes \mathbf{G}_n\right) \cdot \mathbf{H}_{\mathbf{B},C,\mathbf{x}} = \mathbf{B}_C - C(\mathbf{x}) \cdot \mathbf{G}_n,$$

which is the algebraic mechanism by which the scheme simultaneously enables decryption for authorised users and cryptographically blocks all others. A selective

security proof under the LWE assumption is provided.

4. **A verified concrete implementation.** Chapter 6 deploys the BGG+14 scheme in a university document management system with twenty users, ten institutional attributes, and a depth-4 Boolean access policy. A complete hand-computed example over $\mathbb{Z}_{17}$ is verified for four selected users, and a Python simulation confirms the expected access outcomes for all twenty users across 680 message bits. The full simulation code is provided in (Section 6.15).

1.3 Thesis Organisation

Chapter 2 presents the RSA cryptosystem and its mathematical foundations. Chapter 3 analyzes Shor's algorithm and the quantum threat to classical cryptography. Chapter 4 develops post-quantum alternatives. Chapter 5 presents the lattice-based KP-ABE construction. Chapter 6 contains the application and verification. Chapter 7 concludes with open directions. Throughout, definitions and theorems are stated formally, and all major results are accompanied either by proof or by explicit numerical verification.

CHAPTER 2

RSA

Why are prime numbers used in cryptography? The reason prime numbers are so useful in cryptography is due to the difficulty of factoring large numbers into their original prime factors. RSA is based on the process of factoring large integers, where the plaintext and ciphertext are integers between 0 and $n-1$ for some n (the range is $0 \leq m < n$ because RSA uses modular arithmetic modulo n. Allowing $m = n$ would reduce to 0 and lose the original data). A typical size for n is 2048 bits, or approximately 617 decimal digits. That is,

$$n < 2^{2048} \quad \text{since} \quad 2048 \cdot \log_{10}(2) = 2048 \cdot 0.301 \approx 617$$

Cryptographic systems use random numbers from a range of 2^n values to generate keys, and their strength is measured in bits. For example, *128-bit security* means an attacker would need to try up to 2^{128} possible keys in a brute-force attack.

2.1 Mathematical Concepts Required for RSA

2.1.1 Modular Arithmetic and Exponentiation

Congruence ($\equiv$) is used in cryptography instead of equality ($=$) because:

- It helps in working with large numbers securely and efficiently.
- It is the basis of modular arithmetic, which is central to modern encryption algorithms.
- It makes certain problems easy to compute in one direction but very hard to reverse, which is exactly what cryptography requires.

Congruence:

$$a \equiv b \pmod{n}$$

This means that when a and b are divided by n, they leave the same remainder. Alterna-

tively,

$$a = kn + b, \quad \text{for some integer } k$$

Modular exponentiation refers to computing:

$$a^b \mod n$$

That is, raise a to the power b, then take the remainder when divided by n. It is used in RSA to perform encryption and decryption.

2.1.2 Euler's Totient Function $\phi(n)$

Euler's totient function $\phi(n)$ is the count of positive integers less than or equal to n that are coprime to n. Two numbers are coprime if their greatest common divisor (gcd) is 1.

- If n is prime, then: $\phi(n) = n - 1$
- If $n = p \cdot q$ where p and q are distinct primes, then: $\phi(n) = (p-1)(q-1)$

Theorem 2.1.1 (Euler's Theorem)**.** *Euler's Theorem states that for any integers a and n that are relatively prime:*

$$a^{\phi(n)} \equiv 1 \pmod{n}$$

Proof. Let us list all the positive integers less than n that are also coprime to n. Let this set be:

$$\{r_1, r_2, \ldots, r_{\phi(n)}\}$$

Now multiply each element in this set by a, and reduce modulo n:

$$\{a \cdot r_1 \bmod n,\ a \cdot r_2 \bmod n,\ \ldots,\ a \cdot r_{\phi(n)} \bmod n\}$$

Because a is coprime to n, each result is still coprime to n, and the new set contains no repeated elements. It is therefore a rearranged version of the original set.

Multiplying all elements in the original set:

$$r_1 \cdot r_2 \cdots r_{\phi(n)} \pmod{n}$$

Multiplying all elements in the new set:

$$(a \cdot r_1)(a \cdot r_2) \cdots (a \cdot r_{\phi(n)}) = a^{\phi(n)} \cdot r_1 r_2 \cdots r_{\phi(n)} \pmod{n}$$

So we have:

$$r_1 r_2 \cdots r_{\phi(n)} \equiv a^{\phi(n)} \cdot r_1 r_2 \cdots r_{\phi(n)} \pmod{n}$$

Cancelling the common product (which is permitted since it is coprime to n) gives:

$$a^{\phi(n)} \equiv 1 \pmod{n}$$

□

Theorem 2.1.2 (Fermat's Little Theorem)**.** *If p is a prime number and a is an integer such that $p \nmid a$, then:*

$$a^{p-1} \equiv 1 \pmod{p}$$

This means that if any integer a that is not divisible by a prime p is raised to the power $p-1$ and the result is divided by p, the remainder is always 1

2.2 Primality Testing

To use large prime numbers in cryptography, efficient methods for testing whether a number is prime are required. There are two main categories of algorithms.

2.2.1 Deterministic Algorithms

The following algorithms always produce the correct result but are generally too slow for large numbers.

- **Divisibility Test:** Tries all divisors up to $\sqrt{n}$. Accurate but very slow for large inputs.
- **AKS Algorithm (Agrawal-Kayal-Saxena):** A deterministic polynomial-time algorithm discovered in 2002 [4], based on congruences of the form $(x-a)^p \equiv x^p - a \pmod{p}$. It is not yet practical due to the size of its complexity constant.

Example 2.2.1 (AKS Primality Test for $n = 5$)**.** To verify whether $n = 5$ is prime using the Agrawal-Kayal-Saxena (AKS) algorithm [4], the following steps are performed:

1. Perfect Power Check We first verify whether n is a perfect power a^b for $a, b > 1$. Since $2^2 = 4$ and $2^3 = 8$, it is clear that $n = 5$ is not a perfect power.
2. Selection of r We select a small integer r such that the order of n modulo r is sufficiently large. For this demonstration, let $r = 2$.
3. Greatest Common Divisor Check We compute $\gcd(a, n)$ for all $a \leq r$. Here, $\gcd(2, 5) = 1$. Since no factors are found, we proceed.

4. Polynomial Congruence Test The core of the algorithm is to verify the congruence:

$$(x + a)^n \equiv x^n + a \pmod{x^r - 1,\ n} \tag{2.1}$$

Setting $a = 1$, $n = 5$, and $r = 2$, we expand the left-hand side using the binomial theorem:

$$(x + 1)^5 = x^5 + 5x^4 + 10x^3 + 10x^2 + 5x + 1$$

Reducing the coefficients modulo $n = 5$ yields:

$$(x + 1)^5 \equiv x^5 + 1 \pmod{5}$$

Next, we reduce modulo $x^r - 1 = x^2 - 1$. Since $x^2 \equiv 1 \pmod{x^2 - 1}$, it follows that $x^4 \equiv 1$ and $x^5 \equiv x$. Substituting:

$$(x + 1)^5 \equiv x + 1 \pmod{x^2 - 1,\ 5}$$

Comparing with the right-hand side:

$$x^5 + 1 \equiv x + 1 \pmod{x^2 - 1,\ 5}$$

Conclusion: Both sides evaluate to $x + 1$, so the congruence holds. Therefore, $n = 5$ is prime.

2.2.2 Probabilistic Algorithms

Since it is difficult to determine if a number is prime with 100% surity, we require probabilistic algorithms. They are fast, and running them multiple times reduces the probability of failure in determination of primality to nearly zero.

Fermat Primality Test:

Based on Fermat's Little Theorem, first proposed by Pierre de Fermat in 1640, this test uses modular exponentiation to identify composite numbers [5]. It serves as the historical precursor to modern probabilistic primality tests used in asymmetric cryptography.

$$a^{n-1} \equiv 1 \pmod{n} \quad \text{for all } 1 \leq a < n$$

If this condition fails, the number is definitely composite. If it passes, the number may still be composite. Carmichael numbers, for example 561, can pass this test for all valid bases.

Example 2.2.2. To verify the primality of $n = 5$ using Fermat's Primality Test, we test

the congruence $a^{n-1} \equiv 1 \pmod{n}$ for all integers a in the range $1 < a < 5$:

1. For $a = 2$: $2^4 = 16$. Since $16 = (3 \times 5) + 1$, we have $16 \equiv 1 \pmod{5}$.
2. For $a = 3$: $3^4 = 81$. Since $81 = (16 \times 5) + 1$, we have $81 \equiv 1 \pmod{5}$.
3. For $a = 4$: $4^4 = 256$. Since $256 = (51 \times 5) + 1$, we have $256 \equiv 1 \pmod{5}$.

Conclusion: Because $a^4 \equiv 1 \pmod{5}$ holds for all bases $a \in \{2, 3, 4\}$, the value $n = 5$ passes the Fermat Primality Test for these bases, supporting the conclusion that 5 is prime.

Fermat's Pseudoprimes:

Carmichael Number

A Carmichael number is a composite integer n that satisfies the congruence:

$$b^{n-1} \equiv 1 \pmod{n}$$

for every integer b coprime with n. This behaviour mimics Fermat's Little Theorem, making such numbers *Fermat pseudoprimes* in all bases.

Characteristics

1. **Square-free:** Carmichael numbers have no repeated prime factors.
2. They have at least **three distinct odd prime factors**.
3. **Korselt's Criterion (1899):** A number n is a Carmichael number if and only if:
   - n is square-free, and
   - For each prime divisor p of n, $(p - 1)$ divides $(n - 1)$.

Examples

1. The smallest Carmichael number is $561 = 3 \cdot 11 \cdot 17$.
2. Other early examples include: 1105, 1729, 2465, 2821, 6601, 8911.
3. The number 1729 is also the **Hardy-Ramanujan number**, the smallest integer expressible as the sum of two cubes in two different ways: $(1^3 + 12^3)$ and $(9^3 + 10^3)$.

Miller-Rabin Primality Test:

Originally proposed as a deterministic test by Gary Miller in 1976 [6] and later modified by Michael Rabin in 1980 [7] into a probabilistic algorithm, this test is a core primitive in modern asymmetric cryptography. It evaluates the primality of an integer n by writing $n - 1 = 2^k \cdot m$ where m is odd, and then analysing the sequence of squares $a^{2^i m} \pmod{n}$.

Example 2.2.3 (Miller-Rabin Test for $n = 561$)**.** To determine whether $n = 561$ is prime,

we first write $n-1$ in the form $2^k \cdot m$:

$$561 - 1 = 560 = 2^4 \cdot 35 \implies k = 4,\ m = 35.$$

Choosing base $a = 2$, we compute the initial value $b_0 = a^m \pmod{n}$:

$$b_0 = 2^{35} \equiv 263 \pmod{561}.$$

Since $b_0 \not\equiv 1$ and $b_0 \not\equiv -1 \pmod{561}$, we compute successive squares $b_i = b_{i-1}^2 \pmod{561}$ for $i < k$:

$$\begin{aligned} b_1 &= 263^2 \equiv 166 \pmod{561} \\ b_2 &= 166^2 \equiv 67 \pmod{561} \\ b_3 &= 67^2 \equiv 1 \pmod{561} \end{aligned}$$

Condition Analysis:

- $b_0 \not\equiv 1 \pmod{561}$.
- No $b_i \equiv -1 \pmod{561}$ was found for $i \in \{0, 1, 2\}$.
- The sequence reached 1 without a preceding -1.

Conclusion: Because the conditions for primality are not satisfied, 561 is confirmed composite. This demonstrates that while 561 is a Carmichael number that passes Fermat's test for many bases, it is correctly identified as composite by the Miller-Rabin test.

2.2.3 Complexity Analysis of Primality Testing Algorithms

The development of primality testing has progressed from simple probabilistic checks to deterministic polynomial-time algorithms. Table 2.1 summarises the key algorithms discussed in this chapter.

Table 2.1: Chronological Summary and Complexity of Primality Algorithms

Year	**Algorithm**	**Type**	**Complexity**
1640	Fermat [5]	Probabilistic	$O(k \log^3 n)$
1976	Miller [6]	Deterministic*	$O(\log^4 n)$
1980	Miller-Rabin [7]	Probabilistic	$O(k \log^3 n)$
2002	AKS [4]	Deterministic	$O(\log^6 n)$

**Note: Miller's 1976 deterministic version relies on the Generalized Riemann Hypothesis.*

2.3 Factorization Algorithms

2.3.1 Fermat's Factorization Algorithm

Developed by Pierre de Fermat in 1643, this method uses the difference of squares to identify factors of an odd composite integer [5]. It is most efficient when the prime factors of n are close in value.

Mathematical Foundation The algorithm is based on the identity:

$$n = x^2 - y^2 = (x + y)(x - y) \tag{2.2}$$

Any odd composite n can be represented as a difference of two squares. If $n = pq$, we set $x = \frac{p+q}{2}$ and $y = \frac{q-p}{2}$ to recover the factors.

Algorithmic Steps To factor an odd composite n that is not a perfect square:

1. Initialise $x = \lceil \sqrt{n} \rceil$, the smallest integer $\geq \sqrt{n}$.
2. Compute $y^2 = x^2 - n$.
3. Evaluate whether y^2 is a perfect square:
   - **If yes:** The factors are $(x - y)$ and $(x + y)$.
   - **If no:** Increment x by 1 and repeat from Step 2.

2.3.2 Pollard's $p - 1$ Factorization Algorithm

Introduced by John Pollard in 1974, this algorithm is a special-purpose factorization method that is particularly effective when $p - 1$ is B-smooth for some prime factor p of n [8].

Operating Principle The method applies Fermat's Little Theorem: if $p \mid n$, then $a^{p-1} \equiv 1 \pmod{p}$. If M is a multiple of $p - 1$, then $a^M \equiv 1 \pmod{p}$, which means p divides $\gcd(a^M - 1, n)$.

Algorithmic Steps

1. Select a base a (typically $a = 2$) and a smoothness bound B.
2. Define M as the least common multiple of all integers up to B: $M = \text{lcm}(1, 2, \ldots, B)$.
3. Compute $a^M \pmod{n}$ using modular exponentiation.
4. Calculate $g = \gcd(a^M - 1, n)$.
5. **Result Analysis:**
   - If $1 < g < n$, then g is a non-trivial factor.

- If $g = 1$, increase the bound B.
- If $g = n$, select a different base a.

2.4 Survey of Algorithms for Integer Factorization: Timeline

The comparative efficiency and historical context of integer factorization algorithms are summarised in Table 2.2.

Table 2.2: Chronological and Performance Survey of Integer Factorization Algorithms

Method	**Description and Historical Context**	**Effective Digits**
Trial Division	Systematic division by primes $p \leq \sqrt{N}$. Described in *Liber Abaci* (1202) [5].	≤ 10 (small factors)
Pollard's Rho	Uses pseudo-random sequences and GCD; factored the 78-digit Fermat number $2^{256} + 1$ [8].	15–20 digits
Pollard's $p - 1$	Applies Fermat's Little Theorem for B-smooth factors [8].	25–30 digits
ECM	Generalisation of $p - 1$ using elliptic curves. Record 83-digit factor found in 2013 [9].	20–60 (factor size)
Fermat's	Based on $N = x^2 - y^2$; most effective when prime factors are close in value [5].	≤ 30 (close factors)
QS	Uses smooth quadratic residues; the standard method before GNFS [11].	Up to 100 digits
GNFS	State-of-the-art general-purpose method. Factored RSA-250 in February 2020 [10].	> 110 digits

2.5 RSA PKE Algorithm

The RSA algorithm was publicly introduced in 1977 by Ron Rivest, Adi Shamir, and Leonard Adleman at the Massachusetts Institute of Technology, and remains the most widely deployed asymmetric cryptosystem in modern digital infrastructure [2]. While its publication transformed public key cryptography, it was later revealed that Clifford Cocks of the British Government Communications Headquarters (GCHQ) had independently derived an equivalent scheme in 1973, though his work remained classified until 1997. The

security of RSA rests on the presumed computational hardness of the integer factorization problem, specifically the difficulty of factoring the product of two sufficiently large distinct prime numbers.

1. Keygen
   - Choose two large prime numbers p and q

     $$p, q \in \mathbb{P}$$

     where $\mathbb{P}$ is the set of prime numbers. These must be chosen at random and kept secret.
   - Compute the modulus n

     $$n = p \cdot q$$

     The modulus n is used in both encryption and decryption and is made public.
   - Compute Euler's Totient Function $\phi(n)$
     Since p and q are prime:

     $$\phi(n) = (p-1)(q-1)$$

     This value is used to determine the encryption and decryption exponents.
   - Choose the public exponent e

     $$1 < e < \phi(n) \quad \text{such that} \quad \gcd(e, \phi(n)) = 1$$

   - Compute the private exponent d

     $$d \equiv e^{-1} \pmod{\phi(n)} \quad \text{or equivalently} \quad e \cdot d \equiv 1 \pmod{\phi(n)}$$

     That is, d is the modular multiplicative inverse of e modulo $\phi(n)$, computed using the Extended Euclidean Algorithm.

     Key pairs:
     - Public key: (e, n)
     - Private key: (d, n)
2. **Encryption** To encrypt a message m where $0 \leq m < n$:

   $$c = m^e \bmod n$$

   Here c is the ciphertext, and m must be represented as an integer.
3. **Decryption**

To decrypt ciphertext c:

$$m = c^d \bmod n$$

The result m is the original plaintext message.

Summary Table of RSA

Step	Formula	Purpose
Choose primes	p, q	Basis of security
Compute modulus	$n = p \cdot q$	Used in encryption/decryption
Euler's totient	$\phi(n) = (p-1)(q-1)$	Needed to compute d
Public exponent	$\gcd(e, \phi(n)) = 1$	Used for encryption
Private exponent	$d \equiv e^{-1} \bmod \phi(n)$	Used for decryption
Encryption	$c = m^e \bmod n$	Produces ciphertext
Decryption	$m = c^d \bmod n$	Recovers original message

2.5.1 Injectivity of the RSA Encryption Primitive

To establish that the RSA encryption function $f(m) = m^e \pmod n$ is a permutation of $\mathbb{Z}_n$, we must prove its injectivity. A function is injective if for any $m_1, m_2 \in \mathbb{Z}_n$, the condition $f(m_1) = f(m_2)$ implies $m_1 = m_2$.

Proof. Let e be the public exponent and d be the private exponent satisfying:

$$ed \equiv 1 \pmod{\phi(n)} \tag{2.3}$$

This implies the existence of an integer k such that $ed = k\phi(n) + 1$. Suppose two messages $m_1, m_2 \in \mathbb{Z}_n$ produce the same ciphertext c, so that:

$$m_1^e \equiv m_2^e \pmod n \tag{2.4}$$

Raising both sides to the power d:

$$(m_1^e)^d \equiv (m_2^e)^d \pmod n \implies m_1^{ed} \equiv m_2^{ed} \pmod n \tag{2.5}$$

To show $m_1^{ed} \equiv m_1 \pmod n$ for all $m \in \mathbb{Z}_n$, we consider two cases.

Case 1: $\gcd(m, n) = 1$. By Euler's Totient Theorem, $m^{\phi(n)} \equiv 1 \pmod n$. Substituting $ed = k\phi(n) + 1$:

$$m^{ed} = m^{k\phi(n)+1} = (m^{\phi(n)})^k \cdot m \equiv 1^k \cdot m \equiv m \pmod n \tag{2.6}$$

Case 2: $\gcd(m, n) > 1$. In this case m must be a multiple of one of the primes p or q. If m is a multiple of p, then $m \equiv 0 \pmod{p}$, so $m^{ed} \equiv 0 \equiv m \pmod{p}$. Since q is prime and m is not a multiple of q (as $m < n = pq$ and p, q are distinct), Fermat's Little Theorem gives:

$$m^{q-1} \equiv 1 \pmod{q} \implies m^{k(p-1)(q-1)} \equiv 1 \pmod{q} \tag{2.7}$$

Thus $m^{ed} = m^{k\phi(n)+1} \equiv m \pmod{q}$. By the Chinese Remainder Theorem, since the congruence holds modulo both p and q, it follows that $m^{ed} \equiv m \pmod{n}$ for all $m \in [0, n-1]$.

Applying this to the initial assumption:

$$m_1^{ed} \equiv m_2^{ed} \pmod{n} \implies m_1 \equiv m_2 \pmod{n} \tag{2.8}$$

Since $m_1, m_2 \in \mathbb{Z}_n$ with $0 \leq m_1, m_2 < n$, modular equivalence implies $m_1 = m_2$. Therefore $f(m)$ is injective. □

2.5.2 Surjectivity of the RSA Mapping

In addition to injectivity, the RSA encryption function $f : \mathbb{Z}_n \to \mathbb{Z}_n$ must be surjective to ensure that every possible ciphertext $c \in \mathbb{Z}_n$ corresponds to at least one valid plaintext message m.

Proof. Consider any element c in the codomain $\mathbb{Z}_n$. To demonstrate surjectivity, we must identify a pre-image $m \in \mathbb{Z}_n$ such that $f(m) = c$. We define the candidate pre-image using the private exponent d:

$$m \equiv c^d \pmod{n} \tag{2.9}$$

Substituting into the encryption function $f(m) = m^e \pmod{n}$:

$$f(c^d) \equiv (c^d)^e \equiv c^{ed} \pmod{n} \tag{2.10}$$

As established in the proof of injectivity, the relation $ed \equiv 1 \pmod{\phi(n)}$ ensures that $c^{ed} \equiv c \pmod{n}$ for all $c \in \mathbb{Z}_n$, with the Chinese Remainder Theorem applied to handle cases where $\gcd(c, n) > 1$. Therefore:

$$f(m) \equiv c \pmod{n} \tag{2.11}$$

Since an explicit pre-image m can be constructed for any $c \in \mathbb{Z}_n$, the function f is surjective. Combined with the previously established injectivity, this confirms that the RSA encryption function is a bijection on $\mathbb{Z}_n$. □

2.6 Security of RSA

The security of RSA rests on a simple asymmetry: encrypting a message requires only the public key (n, e), which is freely available, but recovering the plaintext from a ciphertext requires the private exponent d. Computing d from e requires knowing $\phi(n) = (p-1)(q-1)$, and computing $\phi(n)$ without knowing p and q separately is believed to be computationally equivalent to factoring n itself [6].

More precisely, given only the public modulus $n = pq$, an adversary attempting to recover the private key must factor n into its prime components p and q. This is the *integer factorization problem*. No classical polynomial-time algorithm is known for this problem, and the best known classical algorithms, including the General Number Field Sieve, run in sub-exponential time

$$O\Big(\exp\Big(c \cdot (\log n)^{1/3} (\log \log n)^{2/3}\Big)\Big) \tag{2.12}$$

for a constant $c > 0$. For RSA-2048, this remains computationally infeasible on any classical machine.

It is important to note that RSA's security is *conditional*: it holds only if integer factorization is hard, and no proof exists that factorization cannot be solved in polynomial time classically. The hardness is an assumption, not a theorem. This distinction becomes critical in the quantum setting: Shor's algorithm, developed in the next chapter, solves integer factorization in polynomial time on a quantum computer, which directly and completely breaks RSA.

CHAPTER 3

QUANTUM COMPUTING VERSUS RSA

The security of RSA, as established in Chapter 2, relies on the presumed computational hardness of the integer factorization problem. This assumption is fundamentally challenged by the advent of quantum computing.

3.1 Introduction

In 1994, Peter Shor introduced a quantum algorithm capable of solving the integer factorization problem with significantly lower complexity than any known classical method [12]. Specifically, Shor's algorithm can factor an integer n using $(\log n)^2 + o(1)$ operations on a quantum computer utilizing $(\log n)^{1+o(1)}$ qubits.

Bit-Length Representation In this context, $\log n$ refers to the base-2 logarithm $\log_2 n$, which represents the number of bits required to encode the integer n. For instance, a standard RSA-2048 key corresponds to $\log n = 2048$. Unlike classical algorithms such as the General Number Field Sieve (GNFS), which exhibit sub-exponential complexity, Shor's algorithm provides a polynomial-time solution.

Time Complexity Analysis The expression $(\log n)^2 + o(1)$ denotes the computational time complexity. The quadratic growth relative to the bit-length implies that for a 2048-bit RSA modulus, the algorithm requires approximately $2048^2 \approx 4.19 \times 10^6$ operations. The $o(1)$ term represents asymptotic corrections that become negligible as the input size n grows toward infinity.

Space Complexity and Qubit Requirements The hardware requirements for Shor's algorithm are characterised by the space complexity $(\log n)^{1+o(1)}$, meaning the number of required qubits grows near-linearly with the bit-length of the modulus. For a 2048-bit key, a fault-tolerant quantum computer would require a qubit count in the low thousands

(logical qubits), although the physical qubit count is significantly higher due to error correction overhead.

3.2 Factoring Capacity of Shor's Algorithm and Historical Timeline

Quantum Resource Estimates for RSA Factorization The practical implementation of Shor's algorithm is constrained by the physical resource overhead required for fault-tolerant quantum computing. Table 3.1 illustrates the relationship between the bit-length of an RSA integer and the corresponding requirements in physical qubits and execution time.

Table 3.1: Quantum Resource Requirements for RSA Factorization

RSA Standard	Physical Qubits (2019)	Physical Qubits (2025)	Time
RSA-1024	$\approx$ 10 Million	$\approx$ 0.5 Million	2–4 Days
RSA-2048	$\approx$ 20 Million	$\approx$ 0.95 Million	$\approx$ 7.2 Days

Scaling and Physical Overhead As the bit-length of the target integer increases, the logical qubit requirement scales linearly; however, the physical qubit count grows more aggressively to account for the code distance needed for error correction during extended runtimes. Traditional estimates for RSA-2048 factorization, such as those from 2019, suggested a requirement of approximately 20 million noisy qubits to complete the computation in 8 hours [13].

Optimized Footprint Estimates Recent advances in approximate residue arithmetic have substantially reduced the projected hardware requirements for Shor's algorithm. While earlier models prioritised execution speed, more recent work by Gidney (2025) shows that RSA-2048 can be factored using approximately 950,000 physical qubits by extending the runtime to roughly one week [14]. As detailed in Table 3.2, this physical footprint supports the operation of the 1,399 logical qubits required for the $n = 2048$ bit-length factorization process [14].

This reduction is achieved primarily through:

- **Approximate Residue Arithmetic:** Truncated modular exponentiation is used to extract significant bits without computing the full value.
- **Yoked Surface Codes:** Storage efficiency of idle logical qubits is improved.
- **Magic State Cultivation:** Significantly less space is allocated for magic state distillation.

Table 3.2: Highlighted parameter choices and resulting logical cost estimates for factoring RSA integers of various sizes.

n	s	ℓ	w_1	w_3	w_4	f	m	P_{deviant}	E(shots)	Toffolis	Qubits
1024	8	18	6	3	6	28	640	2.87%	9.4	1.1e+09	742
1536	8	21	6	3	5	31	960	1.83%	9.3	3.1e+09	1074
2048	8	21	6	3	5	33	1280	1.25%	9.2	6.5e+09	1399
3072	8	21	6	3	5	35	1920	0.91%	9.2	1.9e+10	2043
4096	8	24	6	3	5	36	2560	0.80%	9.2	4.0e+10	2692
6144	8	24	6	3	5	39	3840	0.42%	9.1	1.2e+11	3978
8192	8	24	6	3	5	40	5120	0.40%	9.1	2.7e+11	5261

3.3 The Role of the Quantum Computer in Shor's Algorithm

Shor's algorithm rests on the following key fact: the factoring problem can be reduced to finding the period of a certain function. The algorithm therefore consists of two parts:

- A reduction of the factoring problem to the problem of order-finding, similar to reductions used in other factoring algorithms such as the quadratic sieve.
- A quantum algorithm to solve the order-finding problem.

Let N be the integer to be factored, assumed to be composite rather than prime. A deterministic primality test, such as the AKS algorithm (Agrawal, Kayal, and Saxena, 2004), can confirm this before proceeding. Once N is confirmed composite, the factorization proceeds as follows.

A random integer a is selected such that $1 < a < N$ and a shares no non-trivial factor with N. This is verified by computing $\gcd(a, N)$ using Euclid's algorithm:

- If $\gcd(a, N) = 1$, then a is coprime to N and we proceed.
- Otherwise, a non-trivial factor of N has been found directly and no further steps are needed.

The next step is to compute the powers of a modulo N:

$$a^0 \bmod N, \quad a^1 \bmod N, \quad a^2 \bmod N, \quad \ldots$$

This defines the function:

$$f(r) = a^r \bmod N$$

The objective is to find the **period** of this function, that is, the smallest positive integer

r such that:

$$f(r) = a^r \bmod N = 1$$

By a standard result in number theory, for any a coprime to N, the function f will return 1 for some $r \leq N$, after which the sequence repeats. If $f(r) = 1$, then:

$$f(r+1) = f(1)$$

and in general:

$$f(r+s) = f(s)$$

The Quantum Part of the Algorithm The goal is to find the **period** r of the function:

$$f(x) = a^x \bmod N$$

that is, the number of steps after which the function begins to repeat.

Example 3.3.1. For $f(x) = 2^x \bmod 15$, the output sequence is:

$$1,\ 2,\ 4,\ 8,\ 1,\ 2,\ 4,\ 8,\ \ldots$$

The sequence repeats every 4 steps, so the period is $r = 4$.

Finding this period is computationally hard on a classical computer when the numbers are large. A quantum computer can solve this efficiently by placing all possible values of x into superposition simultaneously, evaluating $f(x)$ across all of them, and then using interference to identify the period.

The term "simultaneously" here refers to **quantum superposition**: a qubit, the basic unit of quantum information, can exist as 0 and 1 at the same time, unlike a classical bit which holds exactly one value at a time.

After preparing a quantum superposition and entangling it with $f(x) = a^x \bmod N$, the Quantum Fourier Transform (QFT) is applied. The QFT exploits **quantum interference** to amplify measurement outcomes that correspond to the correct period. Probability amplitudes associated with the correct period interfere constructively, while those corresponding to incorrect values interfere destructively. Measurement of the transformed state therefore yields information about the period with high probability.

The Period is Even in Shor's Algorithm A result from number theory states that for the majority of choices of a, the period of the function will be even.

Theorem 3.3.1. *If a is an integer such that* $\gcd(a, N) = 1$*, then with high probability the*

order (period) r of $a \bmod N$ is even [15].

Let $N = pq$ where p and q are distinct odd primes, and let a be a randomly chosen integer with $1 < a < N$ and $\gcd(a, N) = 1$.

- The function $f(x) = a^x \bmod N$ is periodic.
- The period r (the order of $a \bmod N$) divides $\phi(N) = (p-1)(q-1)$.
- Since both $p-1$ and $q-1$ are even (as p and q are odd primes), their product $\phi(N)$ is also even.

It is therefore highly likely that r divides an even number and is itself even.

Theoretical Foundation:

- **Euler's Theorem:** If $\gcd(a, N) = 1$, then $a^{\phi(N)} \equiv 1 \pmod{N}$.
- **Group Theory:** The order r of an element a in the multiplicative group $\mathbb{Z}_N^*$ must divide $\phi(N)$. Since $\phi(N)$ is even, most values of r will be even.

From Period to Factors If the chosen a yields an odd period, a different value of a is selected. Once an even r is found such that:

$$a^r \equiv 1 \pmod{N}$$

subtracting 1 from both sides gives:

$$a^r - 1 \equiv 0 \pmod{N} \tag{3.1}$$

or equivalently:

$$N \mid (a^r - 1) \tag{3.2}$$

Applying the difference of squares identity $x^2 - y^2 = (x + y)(x - y)$ with $1 = 1^2$:

$$N \mid \left(\sqrt{a^r} + 1\right)\left(\sqrt{a^r} - 1\right) \tag{3.3}$$

or equivalently:

$$N \mid \left(a^{r/2} + 1\right)\left(a^{r/2} - 1\right) \tag{3.4}$$

(This step requires r to be even so that $r/2$ is an integer.) Any factor of N must therefore divide either $\left(a^{r/2} + 1\right)$ or $\left(a^{r/2} - 1\right)$ or both. A non-trivial factor of N can then be found by computing:

$$\gcd\left(a^{r/2} + 1,\ N\right) \tag{3.5}$$

$$\gcd\left(a^{r/2} - 1,\ N\right) \tag{3.6}$$

using the classical Euclidean algorithm. One condition must be checked: we require that:

$$a^{r/2} \not\equiv -1 \pmod{N} \tag{3.7}$$

because if $a^{r/2} \equiv -1 \pmod{N}$, then the right-hand side of equation (3.4) reduces to zero and no information about N is obtained. In that case, the chosen value of a is discarded and the procedure restarts with a new one.

CHAPTER 4

POST-QUANTUM CRYPTOGRAPHY

Chapter 3 established that Shor's algorithm poses a credible long-term threat to RSA and other systems whose security rests on integer factorization or discrete logarithms. This motivates the study of post-quantum cryptography: the design of cryptographic primitives whose hardness assumptions are not undermined by quantum computation.

The emergence of large-scale quantum computing has fundamentally changed the threat landscape for public-key cryptography. Classical hardness assumptions, including the intractability of integer factorization and the discrete logarithm problem, are not believed to be hard for quantum computers: Shor's algorithm [12] solves both in polynomial time, and as established in Chapter 3, the physical resources required to break RSA-2048 have now been estimated at fewer than one million qubits [14]. The question is therefore not whether quantum computers will eventually break current standards, but whether cryptographic infrastructure can be replaced before they do.

Post-quantum cryptography addresses this by designing primitives whose security rests on problems that are believed to resist both classical and quantum attack. The hardness of decoding random linear codes, for instance, has no known quantum speedup beyond a quadratic Grover factor. The hardness of finding short vectors in a high-dimensional lattice similarly appears to resist quantum algorithms, with the best known quantum attacks offering only modest polynomial improvements over their classical counterparts. Both families have been under structured international evaluation through the NIST Post-Quantum Cryptography Standardization process [16, 17], which concluded its two phases in 2021, 2025 and selected four algorithms for standardization.

A NIST report from April 2016 cites experts who acknowledge the possibility of quantum technology rendering the commonly used RSA algorithm insecure by 2030 [16]. As a result, a need to standardize quantum-secure cryptographic primitives was identified and pursued.

Post-Quantum Cryptography Standardization is a program and competition run by NIST

to update their standards to include post-quantum cryptography. It was announced at PQCrypto 2016. NIST is selecting public-key cryptographic algorithms through a public, competition-like process to specify additional digital signature, public-key encryption, and key-establishment algorithms to supplement FIPS 203, 204, 205 and planned 206, 207, 208 [17]. These algorithms are intended to protect sensitive information well into the foreseeable future, including after the advent of quantum computers. The standardization process went through four rounds of evaluation, beginning in 2016 and concluding second phase(four rounds) in March 2025.

Timeline

Round 1. Of the 82 submissions received, 69 algorithms were accepted for evaluation, comprising 59 Key Encapsulation Mechanism (KEM) algorithms and 26 digital signature algorithms.

Round 2. Twenty-six algorithms advanced from Round 1 for further evaluation. Of these, 17 were KEM algorithms and 9 were digital signature algorithms.

Round 3. Following extensive cryptanalysis in Round 2, NIST launched Round 3 with 7 finalists (4 KEM/PKE algorithms and 3 digital signature algorithms) and 8 alternate candidates (5 KEM/PKE algorithms and 3 digital signature algorithms).

Round 4. After standardizing four main algorithms in July 2022 (Kyber, Dilithium, FALCON, and SPHINCS+), Round 4 was launched to continue evaluating alternate KEM candidates:

- BIKE (code-based)
- Classic McEliece (code-based)
- HQC (Hamming Quasi-Cyclic, code-based)
- SIKE (isogeny-based)

In August 2022, a cryptanalysis attack broke SIKE in under an hour using a classical computer, and it was subsequently eliminated [18].

After Round 4. NIST concluded Round 4 on March 11, 2025, by announcing HQC [19] as an additional algorithm to be standardized as a KEM, alongside Kyber [19]. NIST made the following statement regarding future standardization:

> "NIST does not plan to conduct a fifth round of evaluation. However, the NIST PQC team will continue to monitor developments and will remain open to standardizing new algorithms in the future if warranted."

Final Status of Standardized Algorithms (as of March 2025)

- **KEMs:**

  - CRYSTALS-Kyber (Primary)
  - HQC (Alternate, added March 2025)
- **Digital Signatures:**
  - CRYSTALS-Dilithium
  - FALCON
  - SPHINCS$^+$

4.1 Code-Based Cryptography

4.1.1 Development of the Cryptosystem

In code-based cryptography, the underlying one-way function is built from an error-correcting code C. This primitive may consist of adding an error to a codeword of C, or of computing a syndrome with respect to a parity-check matrix of C. The first such system was a public-key encryption scheme proposed by Robert J. McEliece in 1978. The private key is a random binary irreducible Goppa code and the public key is a generator matrix of a randomly permuted version of that code. The ciphertext is a codeword to which some errors have been added, and only the owner of the private key can remove those errors. Three decades later, some parameter adjustments have been required, but no attack is known to represent a serious threat to the system, even on a quantum computer.

4.1.2 Linear Codes and the Hard Problem

Introduction to Coding Theory and Channel Coding

In practice, communication systems and data storage devices are susceptible to errors due to noise and interference. Coding theory addresses this issue by developing techniques to **detect and correct errors** in transmitted data. Coding is generally classified into two categories: **source coding** and **channel coding**.

- **Source coding** transforms a message into a form suitable for transmission. An example is ASCII encoding, where each character is represented by an 8-bit binary code.
- **Channel coding** adds **redundancy** to the source-encoded message to enable **error detection or correction** during transmission over a noisy channel. The idea is to encode the message a second time after source coding by introducing redundancy so that errors can be detected or even corrected.

Basic Communication Model A simple communication model consists of:

- A **source encoder** that converts the message to binary form.

- A **channel** that transmits the encoded message.
- A **source decoder** that reconstructs the original message.

If noise corrupts the message during transmission, the receiver may not detect the error. For example, if the message "apple" is encoded as `00` but received as `01`, the error remains undetected.

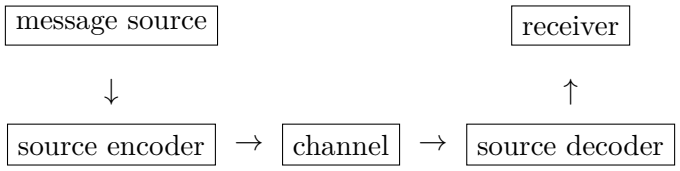


Figure 4.1: Basic communication model without channel coding.

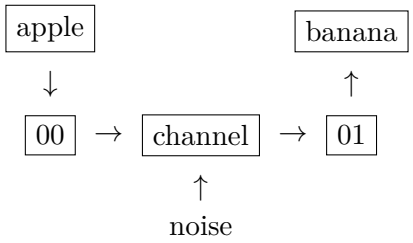


Figure 4.2: An undetected error caused by channel noise.

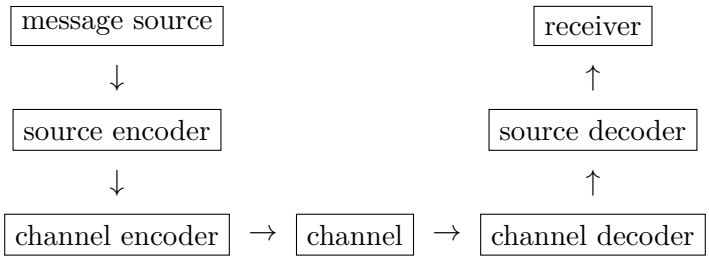


Figure 4.3: Communication model with channel encoding and decoding.

Channel Coding and Redundancy To overcome transmission errors, a **channel encoder** introduces redundancy before transmission. For instance:

- Original codes:
  `00` → apple, `01` → banana, `10` → cherry, `11` → grape
- With 1-bit redundancy:
  `00` → `000`, `01` → `011`, `10` → `101`, `11` → `110`

If a single-bit error occurs (e.g., `000` becomes `100`), the error is detectable because `100` does not correspond to any valid encoded message. However, this scheme only supports **detection**, not **correction**.

Error Correction with More Redundancy By increasing redundancy, error **correction** becomes possible. For example:

- `00` → `00000`, `01` → `01111`, `10` → `10110`, `11` → `11001`

If a single-bit error occurs (e.g., `00000` becomes `10000`), it can be corrected back to `00000` because the received word is closer to `00000` than to any other valid codeword.

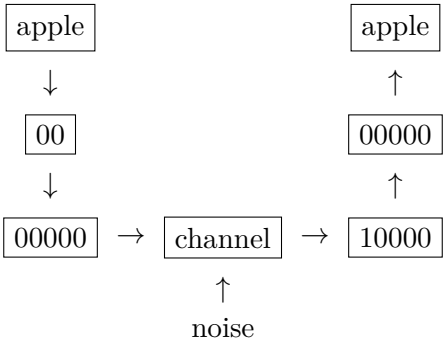


Figure 4.4: Error correction using a redundant code.

Repetition Code: A General Error Correction Strategy A simple method for correcting multiple errors is the **repetition code**: repeat the original bit string $2r + 1$ times. For example, with $r = 2$:

$$\texttt{01} \rightarrow \texttt{0101010101}$$

At the receiver, **majority decoding** is applied:

- Take the most frequent bit at each position group (e.g., positions 1, 3, 5, 7, 9 for the first bit).
- For the second bit, use positions 0, 2, 4, 6, 8.
- This method can **correct up to r errors**.

While effective, repetition codes greatly reduce **transmission efficiency**, making them impractical for high-throughput systems.

Goals of Channel Coding An ideal channel coding scheme aims to:

1. Encode messages quickly.
2. Transmit encoded data efficiently.
3. Decode messages with low computational cost.
4. Maximize information transfer per unit time.
5. Maximize error detection and correction capability.

Vector Spaces over Finite Fields

Definition 4.1.1 (Vector Space)**.** A vector space over $\mathbb{F}_q$ is a nonempty set V with operations of vector addition and scalar multiplication satisfying the usual axioms: associativity, commutativity, distributivity, existence of additive identity and inverses, and compatibility with scalar multiplication.

Standard Vector Space $\mathbb{F}_q^n$ The space $\mathbb{F}_q^n$ consists of n-tuples with entries from $\mathbb{F}_q$. Vector addition and scalar multiplication are defined componentwise. The zero vector is the tuple $(0, 0, \ldots, 0)$.

Subspaces A subset of a vector space is called a subspace if it is also a vector space under the same operations.

Proposition 4.1.1. *A nonempty subset C of a vector space V is a subspace if and only if for all $x, y \in C$ and all scalars $\lambda, \mu \in \mathbb{F}_q$, the combination $\lambda x + \mu y$ also lies in C.*

Linear Combinations, Independence, and Span

- **Linear Combination:** Any expression of the form $\lambda_1 v_1 + \cdots + \lambda_r v_r$.
- **Linear Independence:** A set $\{v_1, \ldots, v_r\}$ is linearly independent if the only way to write 0 as a combination is by setting all coefficients to zero.
- **Span:** The span of a set S is the set of all linear combinations of vectors in S. It is the smallest subspace containing S.

Basis and Dimension A set B is a basis if it is both linearly independent and spans the entire space V.

Remark 4.1.1. Every vector in V can be uniquely expressed as a linear combination of basis vectors. All bases have the same number of elements, which defines the **dimension** of the space, denoted $\dim(V)$.

Theorem 4.1.1 (Counting Theorem)**.** *Let* $\dim(V) = k$*:*

- *The number of elements in V is q^k.*
- *The number of distinct bases for V is:*

$$\frac{1}{k!} \prod_{i=0}^{k-1} (q^k - q^i)$$

Scalar Product and Orthogonality The scalar (dot) product of two vectors in $\mathbb{F}_q^n$ is defined as:

$$v \cdot w = v_1 w_1 + v_2 w_2 + \cdots + v_n w_n$$

- Two vectors are orthogonal if their scalar product is zero.
- The orthogonal complement $S^{\perp}$ of a set S is the set of all vectors orthogonal to every vector in S.

Remark 4.1.2. The dot product defined here is the Euclidean inner product. Other inner products (e.g., Hermitian or symplectic) may also be used in coding theory, but the Euclidean inner product is assumed unless stated otherwise.

Theorem 4.1.2. *Let $S \subseteq \mathbb{F}_q^n$. Then:*

$$\dim(\langle S \rangle) + \dim(S^{\perp}) = n$$

This follows from linear algebra: the null space of a system defined by k linearly independent equations in n variables has dimension $n - k$.

Linear Codes

A linear code is a block code in which the set of codewords forms a subspace of $\mathbb{F}_q^n$, where $\mathbb{F}_q$ is the finite field with q elements. Such codes are referred to as $[n, k]$-linear codes, where k is the dimension of the code and n is the length of each codeword. There are q^k codewords in total, and any linear combination of codewords is also a codeword.

Let C be a linear code over $\mathbb{F}_q$. Then for any $\mathbf{c}_1, \mathbf{c}_2 \in C$ and any scalars $\alpha, \beta \in \mathbb{F}_q$, the combination $\alpha\mathbf{c}_1 + \beta\mathbf{c}_2 \in C$. This closure under addition and scalar multiplication provides the algebraic tools needed to study the code.

Linear codes are important because they admit more efficient encoding and decoding algorithms, and their algebraic structure facilitates the analysis of properties such as minimum distance.

Generator Matrix For a linear $[n, k]$-code C, a generator matrix G is a $k \times n$ matrix over $\mathbb{F}_q$ whose rows form a basis of C. Any codeword in C can be expressed as a linear combination of the rows of G.

Let $\mathbf{u} \in \mathbb{F}_q^k$ be a message vector. The corresponding codeword $\mathbf{c} \in C$ is given by:

$$\mathbf{c} = \mathbf{u}G$$

This process is called encoding.

Parity-Check Matrix and Dual Code For every $[n, k]$-linear code C, there exists an $(n-k) \times n$ matrix H called the *parity-check matrix* such that:

$$C = \{\mathbf{c} \in \mathbb{F}_q^n \mid H\mathbf{c}^T = \mathbf{0}\}$$

A vector is a codeword if and only if it satisfies the linear constraint imposed by H.

The *dual code* $C^\perp$ of a linear code C is defined as:

$$C^\perp = \{\mathbf{x} \in \mathbb{F}_q^n \mid \mathbf{x} \cdot \mathbf{c} = 0 \text{ for all } \mathbf{c} \in C\}$$

This is also a linear code of dimension $n - k$. The parity-check matrix H of C is a generator matrix for $C^\perp$, and vice versa.

Minimum Distance of a Linear Code The Hamming weight of a vector is the number of its non-zero entries. The Hamming distance between two vectors is the number of positions in which they differ.

In a linear code C, the minimum distance d is defined as:

$$d = \min\{\mathrm{wt}(\mathbf{c}) \mid \mathbf{c} \in C,\ \mathbf{c} \neq 0\}$$

The minimum distance therefore equals the minimum weight of any non-zero codeword.

The error detection and correction capabilities of a code depend on d:

- It can detect up to $d - 1$ errors.
- It can correct up to $t = \left\lfloor \frac{d-1}{2} \right\rfloor$ errors.

4.1.3 Syndrome Decoding

In cryptography, a hard problem is one that is computationally infeasible to solve in general, which is precisely what makes it useful for cryptographic security. Syndrome decoding is one such problem.

Cosets

A coset is a shifted version of the code.

- Start with a linear code C, a subspace of $\mathbb{F}_q^n$.
- Pick any vector u in $\mathbb{F}_q^n$.
- Add u to every codeword in C. The resulting set is the coset of C determined by u.

Formally:

$$C + u = \{v + u : v \in C\}$$

In group theory, $\mathbb{F}_q^n$ with vector addition is a finite abelian group. Since C is a subgroup, the definition here coincides exactly with the group-theoretic notion of a coset: if H is a subgroup of G and $g \in G$, the coset of H determined by g is:

$$gH = \{g \cdot h \mid h \in H\} \quad \text{(left coset)}, \qquad Hg = \{h \cdot g \mid h \in H\} \quad \text{(right coset)}.$$

In abelian groups such as $\mathbb{F}_q^n$, left and right cosets coincide.

Example 4.1.1. Let $q = 2$ and $C = \{000, 101, 010, 111\}$. Then:

- $C + 000 = \{000, 101, 010, 111\}$ (this is C itself)
- $C + 001 = \{001, 100, 011, 110\}$
- $C + 010 = \{010, 111, 000, 101\}$
- $C + 011 = \{011, 110, 001, 100\}$
- $C + 100 = \{100, 001, 110, 011\}$
- $C + 101 = \{101, 000, 111, 010\}$
- $C + 110 = \{110, 011, 100, 001\}$
- $C + 111 = \{111, 010, 101, 000\}$

Note that some cosets are identical (e.g., $C + 000 = C + 010 = C + 101 = C + 111$).

Theorem 4.1.3 (Properties of Cosets)**.** *If C is an $[n, k, d]$-linear code over $\mathbb{F}_q$, then:*

1. *Every vector in $\mathbb{F}_q^n$ belongs to some coset of C.*
2. *Each coset has exactly $|C| = q^k$ elements.*
3. *If u is in the coset $C + v$, then $C + u = C + v$.*
4. *Two cosets are either identical or disjoint.*
5. *There are q^{n-k} distinct cosets.*
6. *Two vectors are in the same coset if and only if their difference is a codeword in C.*

Example 4.1.2. Cosets in a Binary $[4, 2]$ Code Let $C = \{0000, 1011, 0101, 1110\}$. The cosets are:

- $0000 + C$: 0000 1011 0101 1110
- $0001 + C$: 0001 1010 0100 1111
- $0010 + C$: 0010 1001 0111 1100
- $1000 + C$: 1000 0011 1101 0110

Standard Array Arranging all cosets in a table, where each row is a coset and the first element of each row is a coset leader, yields what is called a (Slepian) standard array.

Coset Leaders A coset leader is the vector in a coset with the smallest Hamming weight (fewest non-zero entries). It represents the most likely error pattern for that coset.

Non-Uniqueness of Coset Leaders In Example 4.2 above, the coset $0001 + C$ has two minimum-weight vectors: 0001 and 0100. Both are equally valid coset leaders, showing that coset leaders are not always unique.

Nearest Neighbour Decoding for Linear Codes

When a codeword v is transmitted and a word w is received, the *error pattern* is:

$$e = w - v.$$

By property (vi) of cosets, the error pattern e and the received word w lie in the same coset of C. Since small-weight errors are most likely in practice, *nearest neighbour decoding* proceeds as follows:

1. Find the coset containing w.
2. Choose the *coset leader* e (the minimum-weight vector in that coset).
3. Decode to $v = w - e$.

Example 4.1.3. Let $q = 2$ and $C = \{0000, 1011, 0101, 1110\}$, with standard array:

$$\begin{array}{llllll} 0000 + C: & 0000 & 1011 & 0101 & 1110 \\ 0001 + C: & 0001 & 1010 & 0100 & 1111 \\ 0010 + C: & 0010 & 1001 & 0111 & 1100 \\ 1000 + C: & 1000 & 0011 & 1101 & 0110 \end{array}$$

Case (i): $w = 1101$ The word w lies in the fourth row. The coset leader is 1000. Subtracting:

$$v = 1101 - 1000 = 0101.$$

The decoded codeword is 0101.

Case (ii): $w = 1111$ The word w lies in the second row, which has two minimum-weight vectors: 0001 and 0100.

- **Incomplete decoding:** The decoder cannot decide between them and requests retransmission.
- **Complete decoding:** One leader is chosen arbitrarily.

If $e = 0001$: $v = 1111 - 0001 = 1110$.
If $e = 0100$: $v = 1111 - 0100 = 1011$.

Both are valid codewords but different, illustrating the ambiguity that arises when coset leaders are not unique.

Syndrome Decoding

The standard array method works well for small n, but becomes impractical for large codes because:

- The number of cosets grows as q^{n-k}.
- Searching the full array takes too long.

To make decoding faster, the *syndrome* of a received word is used to identify its coset directly.

Definition 4.1.2 (Syndrome)**.** Let C be an $[n, k, d]$-linear code over $\mathbb{F}_q$, H a parity-check matrix for C, and $w \in \mathbb{F}_q^n$ any word. The *syndrome* of w is:

$$S(w) = wH^T \in \mathbb{F}_q^{n-k}$$

Key Properties For $u, v \in \mathbb{F}_q^n$:

1. **Linearity:** $S(u + v) = S(u) + S(v)$.
2. **Codeword test:** $S(u) = 0 \iff u \in C$.
3. **Coset test:** $S(u) = S(v) \iff u$ and v lie in the same coset of C.

Property (3) means that each coset has a unique syndrome. Knowing $S(w)$ therefore identifies exactly which coset w belongs to.

Cosets and Syndromes There is a one-to-one correspondence between cosets and syndromes. There are q^{n-k} cosets and exactly q^{n-k} possible syndromes, one for each vector in $\mathbb{F}_q^{n-k}$.

Definition 4.1.3 (Syndrome Look-Up Table)**.** A *syndrome look-up table* pairs each coset leader with its syndrome. It is also called a Standard Decoding Array (SDA).

Building a Syndrome Look-Up Table (Complete Decoding)

1. List all cosets and select a coset leader (minimum-weight vector) from each.
2. For each coset leader u, compute $S(u) = uH^T$.

Incomplete Decoding If a coset has more than one minimum-weight vector, place an asterisk $(*)$ in the table to signal that retransmission is needed.

Example 4.1.4 (Complete Decoding)**.** Given $C = \{0000, 1011, 0101, 1110\}$ with coset leaders $0000, 0001, 0010, 1000$ and parity-check matrix:

$$H = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 \end{bmatrix}$$

The syndrome look-up table is:

Coset Leader u	Syndrome $S(u)$
0000	00
0001	01
0010	10
1000	11

Example 4.1.5 (Incomplete Decoding)**.** Same code C, but if a coset has multiple minimum-weight leaders, replace with $*$:

Coset Leader u	Syndrome $S(u)$
0000	00
$*$	01
0010	10
1000	11

Remark 4.1.3. In incomplete decoding: a unique coset leader means the error can be corrected; multiple leaders means retransmission is needed. As a practical shortcut, if d is the minimum distance, generate all error patterns e satisfying:

$$\mathrm{wt}(e) \leq \frac{d-1}{2}$$

as coset leaders, then compute their syndromes.

Example 4.1.6 (Larger Code)**.** Given:

$$H = \begin{bmatrix} 1 & 0 & 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 0 & 1 \end{bmatrix}$$

From H, we find $d = 3$, so the code can correct 1 error. All weight-0 and weight-1 words are coset leaders, with one remaining coset whose leader is chosen as 000101. The complete syndrome look-up table is:

Coset Leader u	Syndrome $S(u)$
000000	000
100000	110
010000	011
001000	111
000100	100
000010	010
000001	001
000101	101

Syndrome Decoding Procedure

1. Compute $S(w)$.
2. Look up $S(w)$ in the table to find the coset leader u.
3. Decode as $v = w - u$.

Key Takeaways

- Syndromes provide a fast way to find the coset of a received word.
- Each syndrome corresponds to exactly one coset.
- A syndrome table is much smaller than a full standard array, making decoding practical for large n.

4.1.4 Code-Based Cryptosystems: McEliece

The McEliece Cryptosystem

The McEliece Public-Key Cryptosystem was first proposed by Robert McEliece in 1978 [20]. It remains one of the earliest and most enduring candidates for post-quantum secure public-key encryption because its underlying decoding problem is hard even for quantum computers.

Unlike RSA or ECC, McEliece relies on the *syndrome decoding problem for general linear codes*, which is **NP-hard** [20]. The original scheme uses *binary Goppa codes*, which support efficient decoding, but the public matrix appears as a random linear code so its structure is hidden. Despite decades of cryptanalysis, no efficient attack has been found against the scheme when parameters are chosen correctly.

The Hard Problem Behind McEliece

The security of McEliece rests on the *syndrome decoding problem*: given a parity-check matrix H and a syndrome s, find a vector e of small weight such that $He^T = s$. This

problem is computationally intractable for large n and small weight bounds, even with quantum resources.

The McEliece scheme hides an efficiently decodable code (a Goppa code) within a random-looking generator matrix. An attacker must decode this random-looking code, which is equivalent to solving the general syndrome decoding problem. Quantum algorithms provide only a generic Grover-type quadratic speedup against this problem, not a polynomial-time solution.

Main Components of the McEliece Cryptosystem

Private Key

- An efficiently decodable linear code, typically a binary Goppa code capable of correcting up to t errors.
- Two secret invertible transformations:
  - A random $k \times k$ invertible matrix S (used to scramble codewords).
  - A random $n \times n$ permutation matrix P (used to disguise the code structure).
- Together, the secret code and these two matrices enable fast decoding while ensuring that the public key reflects only a scrambled version of the code.

Public Key

- The public key is the matrix:
$$G' = S \cdot G \cdot P$$
where G is a generator matrix of the secret Goppa code.
- G' appears indistinguishable from a generator matrix of a random linear code, hiding the underlying structured code from an attacker.

Key Generation, Encryption, and Decryption

Key Generation

- Bob selects a binary Goppa code C of length n and dimension k, capable of correcting up to t errors, and knows a decoding algorithm for it.
- He chooses a random invertible matrix $S \in \mathbb{F}_2^{k \times k}$ and a random permutation matrix $P \in \mathbb{F}_2^{n \times n}$.
- He computes the permuted generator matrix $\hat{G} = S \cdot G \cdot P$.
- **Public key:** $(\hat{G}, t)$; **Private key:** (S, P, G).

Encryption

1. Alice represents the message as a binary vector $\mathbf{m} \in \mathbb{F}_2^k$.

2. She computes $\mathbf{c}' = \mathbf{m} \cdot \hat{G}$.
3. She adds a random error vector $\mathbf{e} \in \mathbb{F}_2^n$ of Hamming weight t:

$$\mathbf{c} = \mathbf{c}' + \mathbf{e}$$

4. Alice sends $\mathbf{c}$ to Bob.

Decryption

1. Bob applies P^{-1} to the ciphertext:

$$\mathbf{c} \cdot P^{-1} = \mathbf{m}SG + \mathbf{e}P^{-1}$$

2. Since $\mathbf{m}SG$ is a codeword and $\mathbf{e}P^{-1}$ has Hamming weight t, he uses the Goppa decoding algorithm to correct up to t errors and recover $\mathbf{m}S$.
3. He then multiplies by S^{-1} to obtain the original message:

$$\mathbf{m} = (\mathbf{m}S)S^{-1}$$

An adversary Eve sees $\mathbf{m} \cdot \hat{G} + \mathbf{e}$, which is indistinguishable from a random vector. Decoding a random linear code is hard, so she cannot efficiently recover $\mathbf{m}$.

Security and Hardness of McEliece

The security of the McEliece cryptosystem rests on the **general decoding problem** [20]. Unlike traditional cryptosystems that rely on number-theoretic assumptions (such as factoring or discrete logarithms), McEliece relies on a **combinatorial problem** believed to remain hard even for quantum computers.

General Decoding Problem

> Given a random generator matrix $G' \in \mathbb{F}_2^{k \times n}$ of an $[n, k \geq 2t + 1]$ code, a received word $y \in \mathbb{F}_2^n$, and an integer t, find a message $m \in \mathbb{F}_2^k$ and error vector $e \in \mathbb{F}_2^n$ with $\mathrm{wt}(e) \leq t$ such that $y = m \cdot G' + e$.

This problem is **NP-hard** [20] and remains intractable for randomly chosen linear codes.

Attacks The main known attacks fall into two categories:

- **Structural attacks:** Attempt to recover the private key by exploiting weaknesses in the code structure. These have consistently failed against well-chosen binary irreducible Goppa codes.

- **Decoding attacks:** Attempt to solve the decoding problem directly using algorithms such as Information Set Decoding (ISD). These have exponential time complexity and are mitigated by choosing sufficiently large parameters (e.g., $n \geq 2048$).

4.2 Lattice-Based Cryptography

A *lattice* is a set of points in n-dimensional space with a periodic structure. More formally, given n linearly independent vectors $b_1, \ldots, b_n \in \mathbb{R}^n$, the lattice generated by them is:

$$\mathcal{L}(b_1, \ldots, b_n) = \left\{ \sum_{i=1}^{n} x_i b_i : x_i \in \mathbb{Z} \right\}.$$

The vectors $b_1, \ldots, b_n$ are called a *basis* of the lattice. The way lattices can be used in cryptography is not immediately obvious. The key insight was established in a landmark paper by Ajtai, which identified important connections between lattice problems and laid the foundations for lattice-based cryptography.

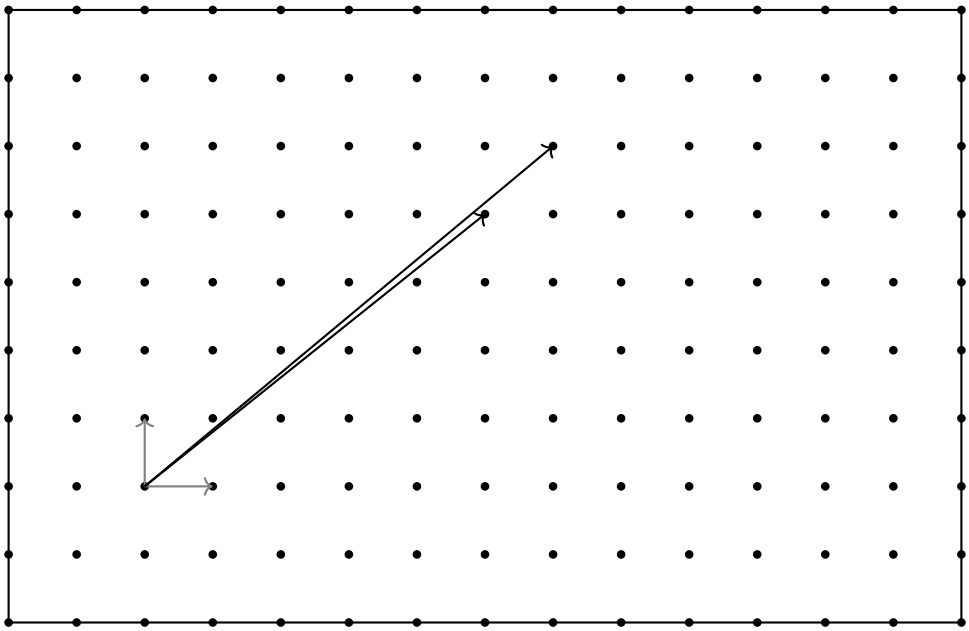


Figure 4.5: A two-dimensional lattice with two possible bases shown.

4.2.1 Mathematical Background

Determinant of a Lattice

The **determinant** $\det(L)$ measures the *density of lattice points*. It is defined as the volume of the **fundamental parallelepiped** generated by the basis B:

$$P(B) = \left\{ \sum_{i=1}^{n} \alpha_i b_i : 0 \leq \alpha_i < 1 \right\}, \qquad \det(L) = \mathrm{vol}(P(B))$$

- **Basis-invariance:** $\det(L)$ is the same for all bases of L.
- **Full-rank case** ($m = n$): $\det(L) = |\det(B)|$.
- **General case:** $\det(L) = \sqrt{\det(B^T B)}$, where $B^T B$ is the Gram matrix.

Gram-Schmidt view: If $b_1^*, \ldots, b_n^*$ are the Gram-Schmidt orthogonalized vectors, then:

$$\det(L) = \prod_{i=1}^{n} \|b_i^*\|$$

A small determinant corresponds to a dense lattice (points close together); a large determinant corresponds to a sparse lattice.

Gram-Schmidt Orthogonalization

The **Gram-Schmidt orthogonalization** process transforms a linearly independent set of vectors into an orthogonal set spanning the same subspace.

Definition 4.2.1. Let $B = \{b_1, b_2, \ldots, b_n\}$ be a basis of a lattice $L \subseteq \mathbb{R}^m$. The Gram-Schmidt orthogonalization produces a sequence $B^* = \{b_1^*, b_2^*, \ldots, b_n^*\}$ defined recursively as:

$$b_1^* = b_1,$$

and for $i \geq 2$:

$$b_i^* = b_i - \sum_{j=1}^{i-1} \mu_{i,j} b_j^*, \qquad \mu_{i,j} = \frac{\langle b_i, b_j^* \rangle}{\langle b_j^*, b_j^* \rangle}.$$

The vectors b_i^* are mutually orthogonal: $\langle b_i^*, b_j^* \rangle = 0$ for all $i \neq j$.

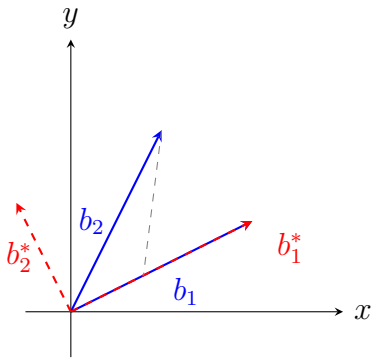


Figure 4.6: Gram-Schmidt orthogonalization: transforming a skewed basis $\{b_1, b_2\}$ into an orthogonal set $\{b_1^*, b_2^*\}$.

Example 4.2.1. Consider the basis $B = \{b_1 = (2, 1),\ b_2 = (1, 2)\} \subseteq \mathbb{R}^2$.

- The first orthogonal vector is $b_1^* = b_1 = (2, 1)$.
- The coefficient is:

$$\mu_{2,1} = \frac{\langle b_2, b_1^* \rangle}{\langle b_1^*, b_1^* \rangle} = \frac{(1, 2) \cdot (2, 1)}{(2, 1) \cdot (2, 1)} = \frac{4}{5}.$$

- The second orthogonal vector is:

$$b_2^* = b_2 - \mu_{2,1} b_1^* = (1,2) - \tfrac{4}{5}(2,1) = \left(-\tfrac{3}{5},\ \tfrac{6}{5}\right).$$

The orthogonal basis is $B^* = \left\{(2,1),\ \left(-\frac{3}{5}, \frac{6}{5}\right)\right\}$.

Importance in Lattice Cryptography Gram-Schmidt orthogonalization is central to lattice-based cryptography because the lengths of the orthogonalized vectors b_i^* provide a quantitative measure of basis quality. Lattice reduction algorithms, such as the LLL algorithm, rely on Gram-Schmidt orthogonalization to iteratively improve a basis by reducing the size of the coefficients $\mu_{i,j}$. The distinction between a nearly orthogonal (good) secret basis and a highly skewed (bad) public basis underpins the security of many lattice-based cryptosystems.

Norms

To measure distances in lattices, we use **norms**. A norm $\|\cdot\|$ must satisfy:

1. $\|x\| \geq 0$, and $\|x\| = 0 \iff x = 0$.
2. $\|\alpha x\| = |\alpha| \cdot \|x\|$ for all scalars α.
3. $\|x + y\| \leq \|x\| + \|y\|$ (triangle inequality).

Common norms:

- ℓ_1: $\|x\|_1 = \sum |x_i|$.
- ℓ_2 (Euclidean): $\|x\|_2 = \sqrt{\sum x_i^2}$.
- ℓ_∞: $\|x\|_\infty = \max_i |x_i|$.

In lattice cryptography, the **Euclidean norm** is standard, as it aligns naturally with geometric intuition and is used in complexity reductions.

Successive Minima

The **successive minima** $\lambda_1, \lambda_2, \ldots, \lambda_n$ describe how lattice vectors grow in length as more independent ones are required:

$$\lambda_i(L) = \inf\left\{r : \dim\left(\operatorname{span}(L \cap B(0,r))\right) \geq i\right\}$$

- λ_1: length of the shortest nonzero lattice vector.
- λ_2: smallest radius containing two linearly independent lattice vectors.
- λ_n: smallest radius containing n independent lattice vectors.

Properties:

- Each λ_i is achieved by some lattice vector.
- Not every set of vectors realizing the minima forms a basis.

Minkowski's Theorems

Two central results link determinants with successive minima:

1. **Minkowski's First Theorem:** Every lattice L of rank n contains a nonzero vector v such that:
$$\|v\|_2 \leq \sqrt{n} \cdot \det(L)^{1/n}$$
This guarantees that short vectors exist, but does not provide a method for finding them.
2. **Minkowski's Second Theorem:** The product of successive minima is bounded:
$$\prod_{i=1}^{n} \lambda_i(L) \leq 2^n \cdot \det(L)$$
This is a stronger structural relationship between the geometry of short vectors and the density of the lattice.

These theorems are critical in establishing worst-case hardness results for lattice problems.

Sublattices and Equivalence of Bases

- If $L' \subseteq L$, then L' is a **sublattice**, and its determinant is an integer multiple of $\det(L)$.
- Two bases B and B' generate the same lattice if and only if $B' = BU$ for some unimodular matrix U (i.e., $\det(U) = \pm 1$).

This algebraic property explains why lattices are basis-independent objects.

4.2.2 Hard Problems for Lattice Cryptography

In lattice-based cryptography, a *good basis* refers to a set of basis vectors that are relatively short and nearly orthogonal, making certain lattice problems computationally difficult. A *bad basis* has long, non-orthogonal vectors, which allows those problems to be solved more easily. The choice of basis significantly affects both the security and efficiency of lattice-based cryptographic schemes.

Shortest Vector Problem (SVP)

Definition Given a basis $B = \{b_1, b_2, \ldots, b_n\}$ of an n-dimensional lattice $L(B) \subseteq \mathbb{R}^n$, the **Shortest Vector Problem (SVP)** asks for a nonzero lattice vector of minimal

Euclidean length:

$$\lambda_1(L) = \min_{v \in L \setminus \{0\}} \|v\|_2,$$

where for a vector $x = (x_1, x_2, \ldots, x_n)$:

$$\|x\|_2 = \sqrt{x_1^2 + x_2^2 + \cdots + x_n^2}.$$

Hardness

- SVP is **NP-hard under randomized reductions** [21].
- Approximating SVP within factors less than $\sqrt{2}$ is also NP-hard.
- No polynomial-time exact algorithm is known for general lattices.
- The problem remains hard even in the presence of quantum algorithms, which distinguishes it from factoring and discrete logarithms.

Approximation Factor In practice, attention is often restricted to the **approximate SVP problem**, denoted SVP_α. The goal is to find a vector $v \in L \setminus \{0\}$ such that:

$$\|v\|_2 \leq \alpha \cdot \lambda_1(L)$$

for some approximation factor $\alpha \geq 1$. When $\alpha = 1$ this is exact SVP; larger α yields an easier but less precise approximation.

Types of SVP *1. Exact SVP.* Find the shortest nonzero vector in $\mathcal{L}(B)$:

$$\lambda_1(\mathcal{L}(B)) = \min_{v \in \mathcal{L}(B) \setminus \{0\}} \|v\|.$$

A structural property noted by Micciancio and Goldwasser: for any shortest vector $v = \sum_i c_i b_i$, at least one coefficient c_i must be odd. This observation underlies reductions between SVP and CVP.

2. Approximate SVP. The approximate (gap) version, denoted GAPSVP_γ, is a promise problem: given a lattice basis B and radius r, decide whether there exists a nonzero $v \in \mathcal{L}(B)$ with $\|v\| \leq r$ (YES), or all nonzero vectors satisfy $\|v\| > \gamma r$ (NO). Approximate SVP is NP-hard under randomized reductions for constant factors $\gamma < \sqrt{2}$ [23].

3. Decisional SVP. The decisional SVP is essentially the gap version. In the ℓ_∞ norm, GAPSVP_1 is **NP-complete** [24]. For the Euclidean norm, NP-hardness was first established via randomized reductions by Ajtai [21], with later work improving the approximation factors.

Algorithms

- **Approximation algorithms:**
  - **LLL (Lenstra-Lenstra-Lovász, 1982):** Runs in polynomial time and outputs a reduced basis where the shortest vector found is within a factor of approximately $2^{(n-1)/2}$ of optimal. Widely used as a preprocessing step [25].
  - **BKZ (Block Korkine-Zolotarev, 1994):** Extends LLL by applying stronger reduction locally to blocks of vectors. Offers significantly tighter approximations at higher computational cost. State-of-the-art for practical cryptanalysis [26, 27].
- **Exact algorithms:**
  - **Enumeration methods:** Explore lattice vectors within a bounded radius. Exact but exponential in dimension [28].
  - **Sieving methods:** Generate many short vectors and combine them to find shorter ones. Exact solutions but require exponential time and memory [29].
- **Decisional algorithms:**
  - **Reduction from approximate SVP:** Any SVP approximation algorithm (such as LLL or BKZ) can be adapted for decisional SVP by comparing the output vector length against the threshold [30].
  - **Threshold testing:** Once a short vector is found, its norm is checked to decide whether $\lambda_1(L) \leq d$ or $\lambda_1(L) > \gamma(n) \cdot d$ [22].

Using LLL for Approximating SVP A standard practical approach to approximate SVP is to apply LLL as a preprocessing step and extract the shortest vector from the reduced basis. Given a basis $B = \{b_1, \ldots, b_n\}$, LLL produces a reduced basis in which the first vector b_1 satisfies:

$$\|b_1\| \leq 2^{(n-1)/2} \cdot \lambda_1(L)$$

where $\lambda_1(L)$ denotes the length of the shortest nonzero lattice vector [31]. In practice, this allows one to approximate the shortest vector by selecting the shortest vector among the LLL-reduced basis vectors. While LLL does not solve SVP exactly in high dimensions, it provides an efficient, provably bounded approximation in polynomial time.

BKZ as an Extension of LLL The Block Korkine-Zolotarev (BKZ) algorithm generalizes LLL by considering k vectors at a time [32].

- While LLL performs local reductions on pairs of vectors, BKZ uses larger block sizes to achieve tighter approximations.
- By increasing the block size k, BKZ provides a $2^{n/k}$ approximation to SVP in 2^k time, offering a scalable trade-off between computational cost and approximation quality.

Variants A closely related variant is the **Shortest Independent Vectors Problem (SIVP)**, which asks for a set of n linearly independent lattice vectors each of length at most $\alpha \cdot \lambda_n(L)$. SIVP is central in reductions from worst-case lattice problems to average-case cryptographic assumptions.

Geometric Interpretation SVP can be interpreted as finding the radius of the smallest Euclidean ball centered at the origin that contains a nonzero lattice point. In low dimensions this corresponds to visually identifying the shortest lattice vector, but in high dimensions the problem becomes computationally intractable.

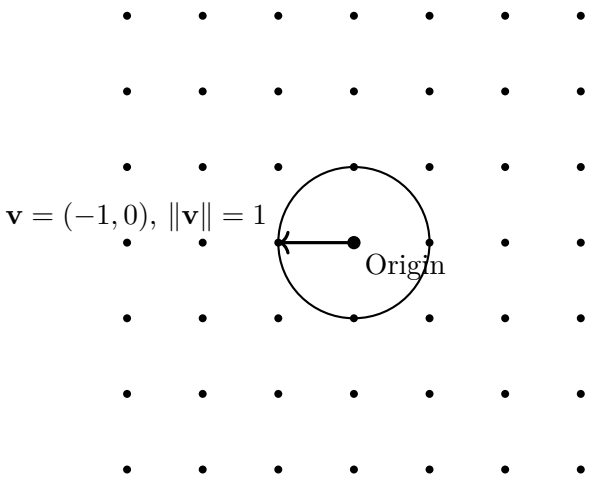


Figure 4.7: Geometric interpretation of SVP: the smallest ball centered at the origin enclosing a nonzero lattice point. The shortest vector here is $(-1, 0)$ with Euclidean length 1.

Importance in Post-Quantum Cryptography

- SVP has known **worst-case to average-case reductions** (Ajtai, 1996), meaning that breaking certain cryptosystems is as hard as solving SVP on all lattices.
- Parameters in NTRU [33], Ring-LWE [34], and Module-LWE [35] are chosen to ensure that even approximate SVP cannot be solved efficiently with current algorithms.
- This hardness underpins the security of many encryption, signature, and identification schemes considered for post-quantum standardization.

Closest Vector Problem (CVP)

Definition 4.2.2. Given a lattice $L(B) \subseteq \mathbb{R}^n$ with basis $B = \{b_1, \ldots, b_n\}$ and a target vector $t \in \mathbb{R}^n$, the **Closest Vector Problem (CVP)** asks for the lattice vector closest to t:

$$\text{Find } v \in L \text{ such that } \|t - v\|_2 = \min_{u \in L} \|t - u\|_2.$$

Definition 4.2.3 (Voronoi Cell)**.** The Voronoi cell $\mathcal{V}(v)$ associated with a lattice vector $v \in L$ is the set of all points in $\mathbb{R}^n$ closer to v than to any other lattice vector $u \in L \setminus \{v\}$.

Geometrically, CVP corresponds to determining the lattice point inside whose Voronoi cell the target vector lies.

Hardness

- CVP is **NP-hard** [24].
- Approximate versions of CVP are also NP-hard within any constant factor.
- CVP is considered harder than SVP in general, since the target vector may be far from the lattice and no short structure necessarily exists.

Approximation Factor The approximate CVP problem, CVP_α, seeks a lattice vector $v \in L$ such that:

$$\|t - v\|_2 \leq \alpha \cdot d(L, t), \qquad d(L, t) = \min_{u \in L} \|t - u\|_2.$$

When $\alpha = 1$ this is exact CVP; larger α yields an easier approximation.

Types of CVP *1. Exact CVP.* Micciancio and Goldwasser (2002) present exact CVP in three equivalent forms:

- **Search version:** Find a lattice point $Bx \in \mathcal{L}(B)$ minimizing $\|Bx - t\|$.
- **Optimization version:** Compute $\text{dist}(t, \mathcal{L}(B)) = \min_{y \in \mathcal{L}(B)} \|t - y\|$.
- These are closely tied to the decisional version below.

The search, optimization, and decisional versions of CVP are polynomially equivalent: a search oracle yields optimization directly, and a decisional oracle can reconstruct the search solution bit-by-bit.

2. Approximate CVP (GapCVP_γ*).* Given (B, t, r) and approximation factor γ, decide whether $\text{dist}(t, \mathcal{L}(B)) \leq r$ (YES) or $\text{dist}(t, \mathcal{L}(B)) > \gamma r$ (NO). CVP remains NP-hard even when approximation is allowed within constant or polylogarithmic factors.

3. Decisional CVP. Given (B, t, r), decide whether $\text{dist}(t, \mathcal{L}(B)) \leq r$. Micciancio and Goldwasser establish that decisional CVP in the ℓ_p norm is **NP-complete** via a reduction from the Subset Sum problem.

Algorithms

- **Approximation algorithms:**
  - **Babai's nearest plane algorithm (1986):** Runs in polynomial time by rounding coordinates with respect to almost a Gram-Schmidt-reduced basis.

Combined with LLL reduction, it guarantees a solution within a factor of $2^{n/2}$ of the optimum.

- **Exact algorithms:**
  - **Enumeration techniques:** Similar to those for SVP but generally more computationally demanding.
  - **Sieving approaches:** Can be adapted from SVP sieving but are less efficient for CVP.
- **Decisional algorithms:**
  - **Reduction-based approaches:** GapCVP is often solved via reductions to approximate CVP or SVP.
  - **Lattice reduction with distance testing:** LLL/BKZ combined with norm computations can probabilistically decide GapCVP instances for small approximation factors.
  - In cryptographic practice, GapCVP hardness is assumed rather than directly attacked, based on known worst-case to average-case reductions.

Approximating CVP Using Babai's Algorithm A commonly used method for approximating CVP is Babai's algorithm, which works well when the lattice basis has been reduced using LLL. Given an LLL-reduced basis $B = \{b_1, \ldots, b_n\}$ and a target vector $y \in \mathbb{R}^n$, Babai's rounding variant computes $v = B\lfloor B^{-1}y \rceil$. The nearest-plane variant improves accuracy by iteratively projecting the target onto lower-dimensional affine subspaces spanned by the basis vectors [37].

Numerical Example of CVP Using Babai's Algorithm Yang *et al.* [36] report a numerical example for a full-rank lattice with basis $B \in \mathbb{Z}^{6\times 6}$ and target vector $t \in \mathbb{R}^6$. Babai's rounding procedure computes the lattice point $v = B\lfloor B^{-1}t \rceil$, yielding a result at Euclidean distance $\|v - t\| = 33.46$ from the target. This demonstrates that Babai's method efficiently produces a reasonably close lattice point despite its coarse rounding strategy [36].

Variants

- **Bounded Distance Decoding (BDD):** A restricted version of CVP where the target is guaranteed to be very close to the lattice (within a fraction of the minimum distance). This variant is particularly relevant in cryptography.
- **Unique-CVP:** A variant where the closest lattice vector is guaranteed to be unique.

Geometric Interpretation CVP asks: given a point in space, which lattice point is closest to it? This is equivalent to partitioning $\mathbb{R}^n$ into Voronoi cells centered at lattice points and determining which cell contains the target t.

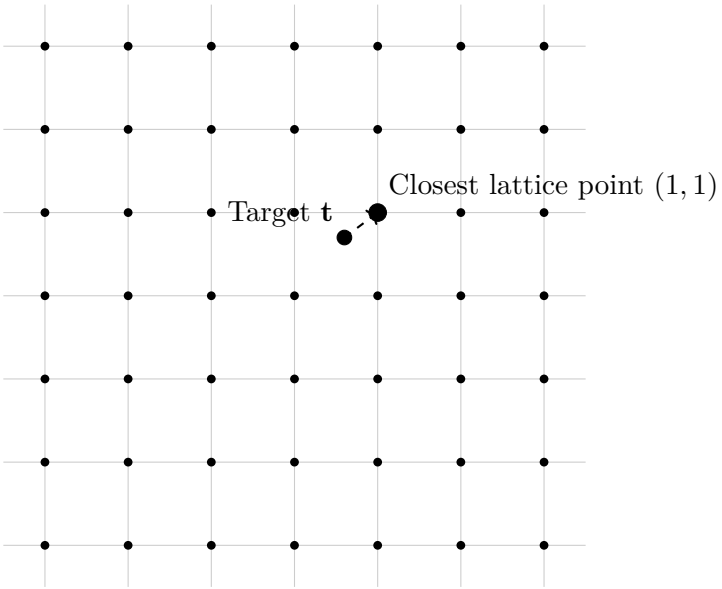


Figure 4.8: Geometric interpretation of CVP: the target vector $\mathbf{t} = (0.6, 0.7)$, which does not lie on the lattice, is mapped to its nearest lattice point $(1, 1)$.

Importance in Post-Quantum Cryptography

- CVP is directly connected to **decoding problems in coding theory**: finding the lattice vector closest to a noisy input corresponds to decoding in a lattice code.
- Trapdoor functions in cryptography are often built around CVP: with a good (nearly orthogonal) basis, CVP can be solved efficiently; with a bad (skewed) basis, it is infeasible.
- This asymmetry forms the basis for constructing encryption and digital signature schemes. Many primitives in the NIST post-quantum standardization process rely on hardness assumptions connected to CVP.

Relative Hardness of SVP and CVP SVP $\leq_T$ CVP: the Shortest Vector Problem is Turing-reducible to the Closest Vector Problem. This means that if CVP can be solved (exactly or approximately), then SVP can also be solved. The reduction is dimension-preserving, approximation-preserving, and deterministic.

Conversely, the reduction from CVP to SVP exists only in a weak form and does not preserve dimension or approximation guarantees, which suggests CVP is the more general and challenging problem. The decision version of CVP is NP-hard, while the natural decision version of SVP lies in NP. This asymmetry supports the intuition that CVP is harder. However, it has not been proven that CVP is strictly harder than SVP; what can be stated with certainty is that SVP reduces cleanly to CVP, so CVP is at least as hard as SVP.

Theorem 4.2.1. *Given an oracle for* CVP_γ*, one can solve* SVP_γ*. Given an oracle for*

OptCVP_γ, *one can solve* OptSVP_γ *[22, 30].*

Proof. Let $B = [b_1, b_2, \ldots, b_n]$. For each $1 \leq i \leq n$, define:

$$B^{(i)} = [b_1, \ldots, b_{i-1}, 2b_i, b_{i+1}, \ldots, b_n].$$

Call $\text{CVP}_\gamma(B^{(i)}, b_i)$ and let the returned vector be x_i. Select from the set $\{x_i - b_i \mid 1 \leq i \leq n\}$ the vector of minimum length and output it as the shortest vector.

For each i, we have $x_i - b_i \in \mathcal{L}(B)$ since b_i is a lattice vector, and $x_i - b_i \neq 0$ because $b_i \notin \mathcal{L}(B^{(i)})$.

Let $v = \sum_{i=1}^{n} a_i b_i$ be the shortest vector in $\mathcal{L}(B)$ with $\|v\| = \lambda_1$. At least one coefficient a_i must be odd, since otherwise $\frac{1}{2}v \in \mathcal{L}(B)$ and $\|\frac{1}{2}v\| = \lambda_1/2$, contradicting minimality. Fix such an i with odd a_i and write $a_i = 2a_i' - 1$. Then:

$$\|a_1 b_1 + \cdots + 2a_i' b_i - b_i + \cdots + a_n b_n\| = \|v\| = \lambda_1.$$

Since $a_1 b_1 + \cdots + 2a_i' b_i + \cdots + a_n b_n \in \mathcal{L}(B^{(i)})$, it follows that $\text{dist}(b_i, \mathcal{L}(B^{(i)})) = \lambda_1$. Therefore:

$$\|x_i - b_i\| \leq \gamma \lambda_1.$$

The reduction for $\text{OptSVP}_\gamma \to \text{OptCVP}_\gamma$ is analogous: instead of returning vectors x_i, we obtain distances d_i and output the minimum. □

Reduction from CVP to SVP via Embedding Any CVP instance in dimension n can be converted into an SVP instance in dimension $n + 1$ [38]. Given a basis $B \in \mathbb{R}^{n \times n}$ and a target vector $t \in \mathbb{R}^n$, construct the $(n + 1)$-dimensional lattice with basis:

$$B' = \begin{bmatrix} B & -t \\ 0 & \alpha \end{bmatrix}$$

where α is a suitably chosen scaling parameter.

A vector in $\mathcal{L}(B')$ has the form $(By - kt,\ k\alpha)$ for $y \in \mathbb{Z}^n$ and $k \in \mathbb{Z}$. For $k = 1$, this becomes $(By - t,\ \alpha)$, whose squared Euclidean norm is:

$$\|(By - t, \alpha)\|^2 = \|By - t\|^2 + \alpha^2.$$

Minimizing this norm is equivalent to minimizing $\|By - t\|$, which is precisely the CVP objective. Thus, the shortest vector in the $(n+1)$-dimensional lattice encodes the solution to the original CVP instance.

4.3 Learning With Errors and Structured Variants

Classical lattice problems such as SVP and CVP are central to complexity theory but are not directly suitable as foundations for efficient cryptographic constructions. A major advance was made by Regev (2005) [39] with the introduction of the Learning With Errors (LWE) problem, which reformulates lattice hardness in terms of learning linear relations corrupted by small additive errors. Formally, the problem is to distinguish between pairs of the form:

$$(A,\ As + e \bmod q),$$

where $A \in \mathbb{Z}_q^{m \times n}$ is a random matrix, $s \in \mathbb{Z}_q^n$ is a secret vector, and e is a small error vector whose components are sampled from a discrete Gaussian distribution χ with small standard deviation $\sigma \ll q$. Without the error, recovering s is trivial by linear algebra; with error, the problem is provably as hard as approximating worst-case lattice problems in high dimension.

To address the efficiency limitations of standard LWE, structured variants have been developed. The Ring-LWE (RLWE) problem (Lyubashevsky, Peikert, and Regev, 2010) replaces vectors and matrices with polynomials over quotient rings, enabling compact representation and efficient multiplication via the Number Theoretic Transform. The Module-LWE (MLWE) problem provides a further balance between the general hardness of LWE and the efficiency of RLWE by working with modules of ring elements rather than single polynomials.

These learning problems now serve as the principal hardness assumptions for lattice-based cryptography, providing both strong worst-case security guarantees and practical efficiency. They form the foundation for modern post-quantum schemes such as Kyber, Saber, and Dilithium.

4.3.1 Learning With Errors (LWE)

Gaussian and Discrete Gaussian Distributions

The **Gaussian distribution** (also called the Normal distribution) describes how values are spread around a mean. Mathematically, for a continuous variable $x \in \mathbb{R}$:

$$f(x) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

where μ is the mean (center of the curve), σ is the standard deviation (spread), and $f(x)$ is the probability density at x.

In cryptography (for LWE), the mean is taken as $\mu = 0$, meaning the noise is centered at

zero. The standard deviation σ controls the width of the distribution: small σ produces a narrow peak with most samples close to 0, while larger σ produces more widely spread noise.

Since cryptography works over integers modulo q, integer-valued noise is required rather than continuous samples. This leads to the **Discrete Gaussian distribution**.

From Continuous to Discrete Gaussian The continuous Gaussian density centered at 0 is:

$$f(x) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{x^2}{2\sigma^2}\right), \quad x \in \mathbb{R}.$$

Evaluating this only at integer points yields non-negative weights:

$$w(x) = \exp\left(-\frac{x^2}{2\sigma^2}\right), \quad x \in \mathbb{Z}.$$

These do not yet form a valid probability distribution, so they are normalized by:

$$Z = \sum_{k\in\mathbb{Z}} \exp\left(-\frac{k^2}{2\sigma^2}\right).$$

This gives the *discrete Gaussian distribution* over $\mathbb{Z}$:

$$D_\sigma(x) = \frac{\exp\left(-\frac{x^2}{2\sigma^2}\right)}{Z}, \quad x \in \mathbb{Z}.$$

Values close to 0 have the highest probability, decreasing exponentially as $|x|$ increases. For integer x, the probability is proportional to:

$$P(X = x) \propto e^{-\frac{x^2}{2\sigma^2}}$$

This discrete distribution is used to generate the small integer noise values in LWE, RLWE, and MLWE.

Example 4.3.1 (Sampling discrete gaussian distribution with $\sigma = 2$)**.** Consider $x \in \{-3, -2, -1, 0, 1, 2, 3\}$.

Step 1: Compute weights $w(x_i) = e^{-x_i^2/8}$:

x	Calculation	$w(x)$
-3	$e^{-9/8}$	0.3247
-2	$e^{-4/8}$	0.6065
-1	$e^{-1/8}$	0.8825
0	e^{0}	1.0000
1	$e^{-1/8}$	0.8825
2	$e^{-4/8}$	0.6065
3	$e^{-9/8}$	0.3247

Step 2: Normalize: $\rho_\sigma = \sum w(x_i) = 4.6274$.

Step 3: Compute probabilities $D_\sigma(x_i) = w(x_i)/\rho_\sigma$:

x	$w(x_i)$	$D_\sigma(x_i)$	Cumulative Prob.
-3	0.3247	0.0702	0.0702
-2	0.6065	0.1311	0.2013
-1	0.8825	0.1907	0.3920
0	1.0000	0.2161	0.6081
1	0.8825	0.1907	0.7988
2	0.6065	0.1311	0.9299
3	0.3247	0.0702	1.0000

Step 4: Sampling. Generate $u \in [0, 1)$ uniformly and find the smallest x_i whose cumulative probability exceeds u. For $u = \{0.04, 0.38, 0.61, 0.90\}$:

u_i	x_i
0.04	-3
0.38	-1
0.61	0
0.90	2

Repeating this over many samples produces a discrete Gaussian bell curve with approximate frequencies:

x_i	Frequency (approx.)
± 3	7% each
± 2	13% each
± 1	19% each
0	22%

Therefore, if we want a discrete Gaussian sampling over S with the given standard

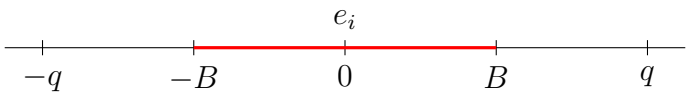
deviation and generate a sample of size 100, we consider ± 3 in that sample approximately 7 times each, and so on.

LWE Definition and Parameters

Notation:

- $\mathbb{Z}_q = \{0, 1, 2, \ldots, q-1\}$.
- $x \overset{R}{\in} S$ means x is selected uniformly at random from S.
- All vectors are column vectors.

Definition 4.3.1 (Learning With Errors: LWE(m, n, q, B))**.** Let $s \overset{R}{\in} \mathbb{Z}_q^n$ and $e \overset{R}{\in} [-B, B]^m$ where $B \ll q/2$. Given $A \overset{R}{\in} \mathbb{Z}_q^{m \times n}$ and $b = As + e \pmod q \in \mathbb{Z}_q^m$, find s.



Example 4.3.2. Let $m = 5$, $n = 3$, $q = 31$, and $B = 2$. The LWE instance is:

$$A = \begin{bmatrix} 11 & 3 & 27 \\ 12 & 21 & 7 \\ 6 & 23 & 30 \\ 5 & 6 & 2 \\ 21 & 0 & 14 \end{bmatrix}, \qquad b = \begin{bmatrix} 25 \\ 25 \\ 12 \\ 29 \\ 17 \end{bmatrix}.$$

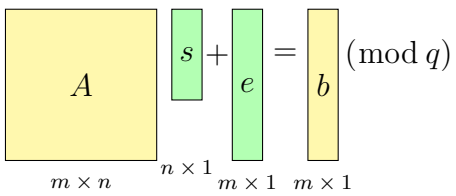
We need to find $s \in \mathbb{Z}_{31}^3$ and $e \in [-2, 2]^5$ with $As + e \equiv b \pmod{31}$. Three solutions exist:

$$s = (2, 11, 7)^T, \quad e = (-2, 0, 2, 1, 1)^T,$$

$$s = (27, 13, 16)^T, \quad e = (1, -2, 1, 1, 1)^T,$$

$$s = (30, 9, 5)^T, \quad e = (-2, -1, 2, 1, -1)^T.$$

In general, uniqueness of the LWE solution is not guaranteed. The parameters must therefore be chosen carefully so that the probability of more than one solution is negligibly small.



LWE Parameter B

1. If $B = 0$ (so $e = 0$), then $As = b \pmod{q}$ can be solved efficiently by linear algebra.
2. If $B = (q-1)/2$, finding s is information-theoretically impossible. We therefore assume $B < q/4$.
3. (Arora-Ge) If B is asymptotically smaller than $\sqrt{n}$, LWE can be solved in subexponential time for sufficiently large $m \gg n$.

LWE Parameters m and n

- When the number of equations m is much larger than the dimension n, the system is overdetermined. The abundance of equations ensures that error regions corresponding to different secrets are well-separated, making the solution unique with high probability.
- If m is not significantly larger than n, error regions may overlap and a single observation could be explained by multiple candidate secrets. The condition $m \gg n$ is therefore typically assumed to guarantee uniqueness of the pair (s, e).

Decisional LWE and Search-Decision Equivalence

Definition 4.3.2 (Decisional LWE (DLWE))**.** The DLWE problem asks one to distinguish between **samples** of the form:

$$(\mathbf{a},\ b = \langle \mathbf{a}, \mathbf{s} \rangle + e \bmod q),$$

where $\mathbf{a}$ is uniformly random in $\mathbb{Z}_q^n$, $\mathbf{s}$ is a fixed secret vector, e is drawn from an error distribution χ, and **samples** drawn uniformly at random from $\mathbb{Z}_q^n \times \mathbb{Z}_q$. The DLWE assumption states that no efficient algorithm can distinguish these two distributions with non-negligible advantage [39].

Equivalence of Search-LWE and Decision-LWE

Theorem 4.3.1. *Claim 1:* DLWE $\leq$ LWE*. The decision variant is no harder than the search variant.*

Proof. Let (A, c) be a DLWE instance. Invoke an LWE oracle $\mathcal{O}_{\text{LWE}}$ with input (A, c).

1. If $c = As + e$, then (A, c) is a valid LWE instance. Under parameters (n, m, q, χ) chosen to ensure a unique solution with high probability [39], the oracle returns the pair (s, e) with $\|e\| \leq B$.
2. If c is sampled uniformly from $\mathbb{Z}_q^m$, the probability that a pair (s, e) satisfying $As + e = c \pmod{q}$ with $\|e\| \leq B$ exists is negligible.

If the oracle returns a valid solution, the reduction concludes that c is an LWE sample. Otherwise, it concludes that c is uniform. □

Theorem 4.3.2. *Claim 2:* LWE $\leq$ DLWE. *An efficient DLWE solver can be used to recover the secret s coordinate-wise.*

Proof. Let (A, b) be a Search-LWE instance with $b = As+e$. We determine each coordinate s_j by iterating through all values $d \in \mathbb{Z}_q$. To test whether $s_1 = d$:

1. Sample a uniformly random vector $\Delta \in \mathbb{Z}_q^m$.
2. Construct (A', b') where $A' = A + [\Delta|0|\dots|0]$ and $b' = b + d\Delta$.
3. If $s_1 = d$, then $b' = A's + e$, which remains a valid LWE instance.
4. If $s_1 \neq d$, the term $(d - s_1)\Delta$ introduces uniform randomness, making b' independent of A' and thus uniformly distributed.

Querying the DLWE oracle on (A', b') identifies the correct d. The full secret s is recovered by repeating this for all n coordinates. □

Definition 4.3.3 (Small-Secret Decisional LWE (ss-DLWE))**.** Let $n, q \in \mathbb{N}$ and let χ denote an error distribution over $\mathbb{Z}_q$ (typically a discrete Gaussian with parameter σ). The ss-DLWE problem is to distinguish between:

- **LWE distribution:** Sample $A \xleftarrow{\$} \mathbb{Z}_q^{n\times m}$, a small secret $s \xleftarrow{\$} \chi^n$, and an error vector $e \xleftarrow{\$} \chi^m$, and output $(A, A^T s + e)$.
- **Uniform distribution:** Sample $A \xleftarrow{\$} \mathbb{Z}_q^{n\times m}$ and $u \xleftarrow{\$} \mathbb{Z}_q^m$ uniformly, and output (A, u).

The ss-DLWE assumption states that these two distributions are computationally indistinguishable. The key feature is that the secret s is drawn from a small distribution rather than uniformly from $\mathbb{Z}_q^n$. Despite this restriction, the combined term $A^T s + e$ still appears pseudorandom modulo q, because the linear term $A^T s$ is effectively masked by the error e.

LWE-Based Public-Key Encryption

The construction of efficient public-key encryption from LWE was advanced by Lindner and Peikert [40], who introduced a more compact version of Regev's cryptosystem using a discrete Gaussian distribution for better performance and tighter security analysis we present here Regev's PKE.

Key Generation Alice does:

1. Select $s \overset{R}{\in} [-B, B]^n$ and $e \overset{R}{\in} [-B, B]^n$.
2. Select $A \overset{R}{\in} \mathbb{Z}_q^{n\times n}$.
3. Compute $b = As + e \pmod q$.
4. **Public key:** (A, b); **Private key:** s.

Encryption To encrypt a message $m \in \{0,1\}$ for Alice, Bob does:

1. Obtain an authentic copy of Alice's public key (A, b).
2. Select $r, z \overset{R}{\in} [-B, B]^n$ and $z' \overset{R}{\in} [-B, B]$.
3. Compute $c_1 = A^T r + z$ and $c_2 = b^T r + z' + m\lfloor q/2 \rfloor$.
4. Output ciphertext $c = (c_1, c_2) \in \mathbb{Z}_q^n \times \mathbb{Z}_q$.

Decryption To decrypt $c = (c_1, c_2)$, Alice computes:

$$m = \text{Round}_q(c_2 - s^T c_1),$$

where:

$$\text{Round}_q(x) = \begin{cases} 0, & \text{if } -q/4 < x \bmod q < q/4, \\ 1, & \text{otherwise.} \end{cases}$$

Example 4.3.3. Domain parameters: $n = 3$, $q = 229$, $B = 2$.

Key generation: Alice selects:

$$A = \begin{bmatrix} 101 & 173 & 27 \\ 192 & 121 & 7 \\ 116 & 223 & 30 \end{bmatrix}, \quad s = \begin{bmatrix} 2 \\ -2 \\ 1 \end{bmatrix}, \quad e = \begin{bmatrix} 0 \\ -2 \\ 1 \end{bmatrix}$$

and computes $b = As + e \pmod{229} = (112, 147, 17)^T$.

Encryption: To encrypt $m = 1$, Bob selects:

$$r = (2, -2, -1)^T, \quad z = (0, 1, -2)^T, \quad z' = -2$$

and computes:

$$c_1 = A^T r + z \pmod{229} = (160, 111, 37)^T, \qquad c_2 = b^T r + z' + 115m \pmod{229} = 26.$$

Decryption: Alice computes:

$$c_2 - s^T c_1 \pmod{229} = 120.$$

Since $120 \bmod 229 = -109$ and $\text{Round}_{229}(-109) = 1$, Alice recovers $m = 1$.

Security of the Lindner-Peikert PKE

- **Claim:** The Lindner-Peikert PKE [40] is indistinguishable under chosen-plaintext attack, assuming that ss-DLWE is hard.

- **Proof:** The encryption can be written as:

$$\begin{bmatrix} c_1 \\ c_2 \end{bmatrix} = \begin{bmatrix} A^T \\ b^T \end{bmatrix} r + \begin{bmatrix} z \\ z' \end{bmatrix} + \begin{bmatrix} 0 \\ \lceil q/2 \rceil m \end{bmatrix}.$$

  By the ss-DLWE assumption, the matrix/vector $[A^T \mid b^T]^T$ is indistinguishable from random. Therefore, $[A^T r + z \mid b^T r + z']^T$ is indistinguishable from random, and from the adversary's perspective, c_2 is the sum of a random element and the scaled plaintext $\lceil q/2 \rceil m$. The adversary therefore learns nothing about m.

Example 4.3.4 (LWE as CVP on a Lattice)**.** Let $q = 7$, $s = (3, 1) \in \mathbb{Z}_7^2$, and $e = 1$. Choose $a = (2, 4)$.

Compute $\langle a, s \rangle = (2)(3) + (4)(1) = 10 \equiv 3 \pmod 7$.

Adding the error: $b = 3 + 1 = 4 \pmod 7$.

The LWE sample is $(a, b) = ((2, 4),\ 4)$.

If $e = 0$, the sample satisfies $b = \langle a, s \rangle \pmod q$ exactly, lying on a q-ary lattice. With nonzero e, the sample is displaced off the lattice by the error. For multiple samples (a_i, b_i), stacking rows gives:

$$b = As + e \pmod q, \qquad b \approx As.$$

Recovering s requires finding the lattice vector As closest to b, which is a **Bounded Distance Decoding (BDD)** instance, a special case of CVP.

4.3.2 Ring Learning With Errors (RLWE)

Algebraic Foundations: Rings and Fields

Lattice-based cryptography increasingly uses advanced algebraic structures to improve performance. A *ring* $(R, +, \times)$ is an algebraic structure where $(R, +)$ forms an abelian group and $(R, \times)$ is a semigroup satisfying the distributive property. Unlike fields, rings do not require multiplicative commutativity or the existence of inverses for all non-zero elements. For example, the integers $\mathbb{Z}$ form a ring because the quotient of two integers is not always an integer.

A *field* $(\mathbb{F}, +, \times)$ is a commutative ring in which every non-zero element has a multiplicative inverse. Common examples include $\mathbb{Q}$, $\mathbb{R}$, and $\mathbb{C}$.

RLWE Construction and Polynomial Rings

In cryptographic applications, efficiency is achieved by working over the quotient ring:

$$R_q = \mathbb{Z}_q[x]/(x^n + 1) \tag{4.1}$$

where n is typically a power of two. This ring consists of polynomials of degree less than n with coefficients in $\mathbb{Z}_q$.

The Ring Learning With Errors (RLWE) problem, introduced as an optimization of standard LWE [34], replaces matrix-vector multiplications with polynomial multiplications in R_q. A sample from the RLWE distribution is:

$$b(x) = a(x) \cdot s(x) + e(x) \in R_q \tag{4.2}$$

where $s(x)$ is the secret polynomial, $a(x)$ is uniformly random, and $e(x)$ is a small error polynomial. Storing a single polynomial $a(x)$ replaces the need for a full $n \times n$ random matrix A, significantly reducing communication overhead.

The Ring-LWE Public-Key Cryptosystem

The RLWE encryption scheme, an optimization of the Lindner-Peikert construction [40], uses polynomial arithmetic over $R_q = \mathbb{Z}_q[x]/(x^n + 1)$. Let χ be a discrete Gaussian distribution over R_q producing polynomials with small-norm coefficients.

Key Generation

1. Sample secret $s(x) \leftarrow \chi$ and error $e(x) \leftarrow \chi$.
2. Sample a uniformly random polynomial $a(x) \in R_q$.
3. Compute $b = a \cdot s + e \in R_q$.
4. **Public key:** (a, b); **Secret key:** s.

Encryption To encrypt a message $m \in \{0,1\}^n$, viewed as a polynomial $m(x) \in R_q$:

1. Sample ephemeral secret $r \leftarrow \chi$ and error terms $e_1, e_2 \leftarrow \chi$.
2. Compute $u = a \cdot r + e_1 \pmod q$.
3. Compute $v = b \cdot r + e_2 + \lfloor q/2 \rfloor \cdot m \pmod q$.
4. Output ciphertext $c = (u, v)$.

Decryption To recover the message from $c = (u, v)$:

1. Compute $w = v - u \cdot s \pmod q$.
2. Note that $w = \lfloor q/2 \rfloor m + (e \cdot r + e_2 - e_1 \cdot s)$.

3. Output 1 if w is closer to $\lfloor q/2 \rfloor$ than to 0 $\pmod q$, and 0 otherwise.

Example 4.3.5. Consider $R_7 = \mathbb{Z}_7[x]/(x^2+1)$ with $\lfloor q/2 \rfloor = 3$. Let $a(x) = 4 + x$, $s(x) = 1 + x$, and $e(x) = 1$.

- **Key generation:** $b = (4+x)(1+x) + 1 = 4 + 5x + x^2 + 1$. Since $x^2 \equiv -1 \equiv 6 \pmod 7$, this gives $b = 11 + 5x \equiv 4 + 5x \pmod 7$.
- **Encryption (for $m = 1$, $r = 1$, $e_1 = e_2 = 0$):** $u = (4+x)(1) = 4 + x$ and $v = (4+5x)(1) + 3 = 7 + 5x \equiv 5x \pmod 7$.
- **Decryption:** $w = 5x - (4+x)(1+x) = 5x - (4+5x+x^2) = 5x - (3+5x) = -3 \equiv 4 \pmod 7$. Since 4 is closer to $\lfloor q/2 \rfloor = 3$ than to 0, we correctly recover $m = 1$.

4.3.3 Module Learning With Errors (MLWE) based PKE

The Module Learning With Errors (MLWE) problem generalizes both LWE and RLWE, providing a flexible trade-off between security and efficiency. While LWE operates over vectors in $\mathbb{Z}_q^n$ and RLWE over a single polynomial ring R_q, MLWE is defined over modules R_q^k, where elements are vectors of k polynomials. This structure is the mathematical foundation for ML-KEM (formerly Kyber), standardized by NIST [35].

Parameterization and Structure

MLWE PKE is characterized by the ring $R_q = \mathbb{Z}_q[x]/(x^n+1)$, a module rank k, and an error distribution χ. In ML-KEM, $n = 256$ and $q = 3329$. The security level is adjusted by varying k (e.g., $k = 2$ for security level 1, $k = 4$ for level 5).

Scheme Construction

Key Generation The secret key is a vector of small-norm polynomials $s \in R_q^k$ sampled from χ. A public matrix $A \in R_q^{k \times k}$ is drawn uniformly at random. The public key is (A, t) where:

$$t = As + e \pmod q, \quad e \in R_q^k \text{ small.} \tag{4.3}$$

Encryption To encrypt a message $m \in \{0,1\}^n$ (mapped to a polynomial in R_q), sample an ephemeral vector $r \in R_q^k$ and error terms $e_1 \in R_q^k$, $e_2 \in R_q$ from discrete gaussian sampling over R_q. Compute:

$$u = A^T r + e_1 \pmod q \tag{4.4}$$

$$v = t^T r + e_2 + \lfloor \tfrac{q}{2} \rfloor m \pmod q \tag{4.5}$$

Decryption Compute $w = v - s^T u \pmod{q}$:

$$v - s^T u = (t^T r + e_2 + \lfloor \tfrac{q}{2} \rfloor m) - s^T (A^T r + e_1) \tag{4.6}$$

$$= \lfloor \tfrac{q}{2} \rfloor m + (e^T r + e_2 - s^T e_1) \tag{4.7}$$

The message m is recovered by testing whether the coefficients of w are closer to $\lfloor q/2 \rfloor$ or to 0.

Example 4.3.6. Consider $q = 7$, $R_q = \mathbb{Z}_7[x]/(x^2+1)$, and $k = 2$. Let $s = (1+x,\ 2)^T$ and $A = \begin{bmatrix} 2 & 3+x \\ 4+x & 5 \end{bmatrix}$. With encoding $\lfloor q/2 \rfloor = 3$, for $m = 1$, $r = (1, x)^T$, and negligible error, the ciphertext (u, v) enables recovery of m since the noise-adjusted result $w \equiv 3 \pmod 7$ aligns with the scaled message bit.

4.4 The ML-KEM (Kyber) Cryptosystem

The Kyber cryptosystem, standardized by NIST as ML-KEM, is a lattice-based post-quantum public-key encryption and key-encapsulation mechanism. Its security is derived from the hardness of the Module Learning With Errors (MLWE) problem, which is conjectured to be resistant to both classical and quantum cryptanalysis [35]. By operating over structured module lattices, Kyber balances the high security of standard LWE with the computational efficiency of Ring-LWE.

4.4.1 Mathematical Framework

Kyber executes arithmetic within the cyclotomic ring $R_q = \mathbb{Z}_q[x]/(x^n+1)$, where $n = 256$ and $q = 3329$. The plaintext space is $\mathcal{M} \in \{0,1\}^n$, where a binary message $m = (m_0, m_1, \ldots, m_{n-1})$ is represented as a polynomial $m(x) = \sum_{i=0}^{n-1} m_i x^i$.

4.4.2 Algorithmic Structure

The cryptosystem comprises three primary procedures:

1. **Key Generation:** Generates a public-private key pair (pk, sk) using a public matrix $A \in R_q^{k \times k}$ and small-norm noise polynomials using centred binomial distribution.
2. **Encapsulation (Encryption):** Given the recipient's public key, samples randomness, encodes it as a message $m \in \{0,1\}^n$, and computes a ciphertext $c = (u, v)$ together with a shared secret.
3. **Decapsulation (Decryption):** Recovers the original message or shared secret using the private key.

4.4.3 Decoding and Error Management

To ensure robustness against encryption noise, Kyber uses a coefficient rounding function during decryption. For $q = 3329$, coefficients are first converted to a centered representation $x' \in [-(q-1)/2,\ (q-1)/2]$. The decision threshold for bit recovery is:

$$\text{decode}_q(x') = \begin{cases} 0, & \text{if } x' \in (-q/4,\ q/4) \\ 1, & \text{otherwise.} \end{cases} \tag{4.8}$$

For $q = 3329$, the boundary is ± 832.25. Table 4.1 illustrates decoding for a sample polynomial.

Term	Original Coefficient	Centered Mod 3329	Decoded Bit
1	3000	$3000 - 3329 = -329$	0
x	1500	1500	1
x^2	2010	$2010 - 3329 = -1319$	1
x^3	37	37	0

Table 4.1: Coefficient rounding and decoding for $q = 3329$.

4.4.4 Efficiency and Optimization

Practical Kyber implementations use several optimizations:

- **Number Theoretic Transform (NTT):** Accelerates polynomial multiplication from $O(n^2)$ to $O(n \log n)$.
- **Centered Binomial Distribution (CBD):** Samples small-norm error polynomials by computing differences of Hamming weights of random bitstrings:

  $$e = \sum_{i=1}^{\eta} b_i - \sum_{i=1}^{\eta} b'_i, \quad b_i, b'_i \in \{0, 1\}.$$

  This avoids the floating-point arithmetic and rejection sampling required for true discrete Gaussian sampling, making it efficient and suitable for constant-time hardware implementations.
- **Compression:** Reduces the bandwidth requirements for public keys and ciphertexts.

Example 4.4.1. Consider a pedagogical instance with $q = 137$, $n = 4$, and $k = 2$. The secret key S and error vector e are sampled from a small-norm distribution, and the public key t is computed via $As + e$.

- **Encryption:** For message vector m, the ciphertext (u, v) is generated using ephemeral secret r and noise terms e_1, e_2, where v incorporates the message scaled by $\lfloor q/2 \rfloor = 68$.

- **Decryption:** The plaintext is recovered by computing $m' = v - uS^T$. In this instance, $m' = 44 + 60x + 79x^2 + 66x^3$. Applying the decoding threshold $q/4 \approx 34$ recovers the bitstring 0111, since the coefficients 60, 79, and 66 all fall in the "1" decision region.

4.4.5 Bandwidth Optimization via Public Key Compression

A practical challenge in lattice-based cryptography is the relatively large size of public keys compared to RSA or ECC. In ML-KEM-768, efficiency is achieved by replacing the explicit transmission of the large public matrix A with a compact cryptographic seed ρ.

Analysis of Public Key Dimensions

For ML-KEM-768, $q = 3329$, $n = 256$, and $k = 3$. Each coefficient requires $\lceil \log_2(q) \rceil = 12$ bits. The matrix $A \in R_q^{k \times k}$ contains $k^2 n = 9 \times 256 = 2304$ coefficients, giving an uncompressed size of $2304 \times 12 = 27{,}648$ bits. Combined with the public vector t (which has $kn = 768$ coefficients at 12 bits each), the total uncompressed public key size is:

$$|PK_{\text{uncompressed}}| = \frac{27{,}648 + (768 \times 12)}{8} = 4{,}608 \text{ bytes.} \tag{4.9}$$

Seed-Based Reconstruction

ML-KEM uses a 256-bit seed ρ to deterministically generate A via an extendable-output function (XOF) such as SHAKE-128 [39, 35]. Transmitting (ρ, t) instead of (A, t) reduces the public key size to:

$$|PK_{\text{compressed}}| = \frac{256 + 9{,}216}{8} = 1{,}184 \text{ bytes,} \tag{4.10}$$

a compression ratio of approximately 3.9:1, which is significant for constrained and embedded systems.

4.4.6 Ciphertext Compression

ML-KEM reduces ciphertext size by discarding the least significant bits of coefficients. This lossy compression is parameterized to balance size against decryption failure probability.

Compression and Decompression Functions

The compression of $x \in \mathbb{Z}_q$ to d bits is:

$$\text{Compress}_q(x, d) = \lfloor (2^d/q) \cdot x \rceil \pmod{2^d} \tag{4.11}$$

The corresponding decompression function is:

$$\text{Decompress}_q(y, d) = \lfloor (q/2^d) \cdot y \rceil \pmod{q} \tag{4.12}$$

Error Bound Analysis

The compression-decompression cycle introduces a bounded rounding error. Let $x' = \text{Decompress}_q(\text{Compress}_q(x, d), d)$. The deviation satisfies:

$$|x - x'| \leq \left\lceil \frac{q}{2^{d+1}} \right\rfloor \tag{4.13}$$

Larger d reduces the rounding noise but increases ciphertext size. These functions are applied coefficient-wise to polynomials in R_q.

Implementation in ML-KEM-768

In ML-KEM-768, the ciphertext components (u, v) are compressed using bit-depths $d_u = 10$ and $d_v = 4$. For $k = 3$ and $n = 256$, the compressed ciphertext size is:

$$\text{Size} = \frac{(k \cdot n \cdot d_u) + (n \cdot d_v)}{8} = \frac{(3 \times 256 \times 10) + (256 \times 4)}{8} = 1{,}088 \text{ bytes.} \tag{4.14}$$

This is a significant reduction from the uncompressed size of 1536 bytes [34, 40].

4.4.7 Central Binomial Distribution for Noise Sampling

ML-KEM requires small-norm error polynomials $e \in R_q$. The Central Binomial Distribution (CBD), denoted β_η, is used as a discrete approximation of a centered Gaussian [41].

Probabilistic Definition

For a given parameter η, let $(a_1, \ldots, a_\eta)$ and $(b_1, \ldots, b_\eta)$ be independently and uniformly sampled bits from $\{0, 1\}$. An element $c \sim \beta_\eta$ is:

$$c = \sum_{i=1}^{\eta} (a_i - b_i), \quad c \in [-\eta, \eta]. \tag{4.15}$$

The probability mass function for outcome $j \in [-\eta, \eta]$ is:

$$P(c = j) = \frac{\binom{2\eta}{\eta + j}}{2^{2\eta}} \tag{4.16}$$

Distributional Characteristics

For $\eta = 2$, used in several ML-KEM security levels, the distribution has five possible outcomes with a symmetric, bell-shaped profile:

Table 4.2: Probability Distribution for β_2

Outcome (j)	Frequency Count	Probability (P)	Approx. %
-2	$\binom{4}{0} = 1$	1/16	6.25%
-1	$\binom{4}{1} = 4$	4/16	25.0%
0	$\binom{4}{2} = 6$	6/16	37.5%
$+1$	$\binom{4}{3} = 4$	4/16	25.0%
$+2$	$\binom{4}{4} = 1$	1/16	6.25%

Operational Implementation

In practice, ML-KEM generates a noise polynomial by sampling $n = 256$ coefficients independently from β_η. This approach is more computationally efficient than sampling from a true discrete Gaussian, as it avoids transcendental function evaluations and rejection sampling. It therefore supports constant-time implementations that are resistant to timing-based side-channel attacks [42].

CHAPTER 5

ATTRIBUTE-BASED ENCRYPTION

Public-key encryption (PKE) enables secure communication over open networks; however, classical PKE binds ciphertexts to individual public keys and therefore requires the sender to know the exact recipient in advance. Identity-Based Encryption (IBE) simplifies key management by allowing arbitrary strings to serve as public keys. In such a setting, a trusted authority generates a private key corresponding to a given identity, and correctness requires exact identity matching during decryption.

Sahai and Waters introduced *Fuzzy Identity-Based Encryption* (Fuzzy IBE), where an identity is interpreted as a *set* of descriptive attributes rather than a single string [44]. Let $\mathcal{U}$ denote the universe of attributes. An identity is represented as a subset

$$\omega \subseteq \mathcal{U}.$$

Encryption is performed with respect to an attribute set $\omega' \subseteq \mathcal{U}$, and decryption succeeds if and only if the overlap between the two sets meets a threshold d, that is,

$$|\omega \cap \omega'| \geq d. \tag{5.1}$$

This relaxed matching condition introduces error tolerance into the encryption process and allows the scheme to function even when identities are not identical but sufficiently similar [44]. Such a formulation is particularly suitable for biometric or descriptive identity systems.

Bilinear Pairings. The construction of Fuzzy IBE relies on bilinear pairings. Let G_1 and G_T be cyclic groups of prime order p, generated by $g \in G_1$. Let

$$e : G_1 \times G_1 \to G_T$$

be a bilinear map satisfying:

1. **Bilinearity:**
$$e(g^a, g^b) = e(g, g)^{ab} \quad \text{for all } a, b \in \mathbb{Z}_p.$$
2. **Non-degeneracy:** $e(g, g) \neq 1$.
3. **Efficient computability.**

These properties allow exponent multiplication to be transferred into the target group G_T, which is fundamental for the correctness of the scheme [44].

Shamir's Secret Sharing Scheme. A central technique used in Fuzzy IBE is Shamir's secret sharing applied in the exponent. During setup, a master secret $y \in \mathbb{Z}_p$ is selected. To generate a private key for an attribute set ω, the authority chooses a random polynomial

$$q(x) \in \mathbb{Z}_p[x]$$

of degree $d - 1$ such that

$$q(0) = y. \tag{5.2}$$

For each attribute $i \in \omega$, the user receives a key component

$$D_i = g^{q(i)}. \tag{5.3}$$

Because a polynomial of degree $d - 1$ is uniquely determined by any d distinct points, any subset $S \subseteq \omega \cap \omega'$ with $|S| = d$ enables reconstruction of the value $q(0)$ using Lagrange interpolation:

$$q(0) = \sum_{i \in S} q(i)\Delta_{i,S}(0), \tag{5.4}$$

where the Lagrange coefficients are defined as

$$\Delta_{i,S}(0) = \prod_{\substack{j \in S \\ j \neq i}} \frac{0 - j}{i - j}.$$

During encryption, a random value $s \in \mathbb{Z}_p$ is chosen and the ciphertext contains a component of the form

$$E' = M \cdot e(g, g)^{ys}.$$

Using bilinearity, the pairing computation with d valid shares yields

$$\prod_{i \in S} e(D_i, g^s)^{\Delta_{i,S}(0)} = e(g, g)^{s \sum_{i \in S} q(i)\Delta_{i,S}(0)} = e(g, g)^{ys},$$

which cancels the masking term and recovers M [44]. This establishes correctness.

From Fuzzy IBE to Attribute-Based Encryption. Goyal, Pandey, Sahai, and Waters generalized the threshold-based notion into Attribute-Based Encryption (ABE), where access control is defined by more expressive access structures rather than simple set-overlap thresholds [45]. An access structure $\mathbb{A}$ over the universe $\mathcal{U}$ is a collection of authorized subsets:

$$\mathbb{A} \subseteq 2^{\mathcal{U}},$$

with the monotonicity property that if $B \in \mathbb{A}$ and $B \subseteq C$, then $C \in \mathbb{A}$ [45].

In this formulation, decryption succeeds precisely when the user's attribute set satisfies the specified access structure. The construction embeds secret-sharing techniques within the bilinear pairing framework so that only authorized attribute combinations can reconstruct the shared secret.

Formally, an ABE scheme consists of the polynomial-time algorithms

$$(\mathsf{Setup},\ \mathsf{KeyGen},\ \mathsf{Encrypt},\ \mathsf{Decrypt}),$$

where correctness requires that decryption outputs the original message whenever the attribute set satisfies the access structure [45].

To summarize the progression so far: Fuzzy IBE introduced the idea of threshold-based set overlap for decryption, while Attribute-Based Encryption extended this to general monotone access structures. The mathematical framework is built upon bilinear pairings and polynomial secret sharing in the exponent, ensuring correctness and provable security under standard computational assumptions [44, 45].

Building on this foundation, Goyal, Pandey, Sahai, and Waters formalized Key-Policy ABE (KP-ABE) in 2006, where ciphertexts are labeled with attribute sets and users' secret keys embed access structures [47, 45]. In 2008, Waters presented Ciphertext-Policy ABE (CP-ABE), reversing the roles so that encryptors specify access policies directly over attributes [47]. Subsequent work refined security models and expressiveness, notably Waters' fully expressive CP-ABE construction in the standard model and numerous extensions for hierarchical, dual-policy, and searchable encryption [46, 48]. Today, ABE underpins applications ranging from cloud data sharing and health-record protection to broadcast encryption and verifiable computation, offering a versatile framework for attribute-driven cryptographic access control [49].

This thesis adopts the Key-Policy Attribute-Based Encryption (KP-ABE) paradigm for its primary construction, as it more naturally models the intended university document management application: a document is encrypted with respect to a set of descriptive attributes (e.g., course, department, or access

level), while users receive secret keys embedding access policies defined by a trusted institutional authority. In this setting, the authority determines which combinations of attributes a user is permitted to decrypt, thereby enforcing fine-grained access control at the level of key distribution.

The fundamental algebraic tools used in the lattice-based construction presented later in this chapter include the gadget matrix $\mathbf{G}$, the tensor product encoding $\mathbf{x}^\top \otimes \mathbf{G}$, and the homomorphic evaluation algorithms EvalF and EvalFX. These are developed in detail in Sections 5.5.1 through 5.6, before the complete BGG+14 scheme is presented in Section 5.7.

5.1 Key-Policy Attribute-Based Encryption

Key-Policy Attribute-Based Encryption is a public-key primitive in which each ciphertext is labeled by a set of descriptive attributes, while a user's private key embeds an access structure that determines which ciphertexts the user can decrypt [45]. This section develops the necessary mathematical prerequisites and then presents the formal construction.

5.1.1 Preliminaries

Access Structures

An *access structure* specifies which sets of attributes are authorized to recover a secret. Formally, let $P = \{P_1, \ldots, P_n\}$ be a collection of parties (or attributes). An access structure $\mathcal{A} \subseteq 2^P$ (power set of P) is *monotone* if $B \in \mathcal{A}$ and $B \subseteq C$ together imply $C \in \mathcal{A}$ [45].

Example 5.1.1. Let the universe of parties be $P = \{\text{Professor}, \text{TA}, \text{Student}\}$. Define the access structure

$$\mathcal{A} = \{\, S \subseteq P \mid \text{Professor} \in S \,\}.$$

That is, any subset of parties that includes the Professor is authorized.

This access structure is monotone: if $B \in \mathcal{A}$, then Professor $\in B$. For any $C \supseteq B$, it follows that Professor $\in C$, and hence $C \in \mathcal{A}$. For example, $\{\text{Professor}\} \in \mathcal{A}$ implies $\{\text{Professor}, \text{TA}\} \in \mathcal{A}$ and $\{\text{Professor}, \text{TA}, \text{Student}\} \in \mathcal{A}$.

In the context of KP-ABE, the parties correspond to attributes, and a private key embeds a monotone access tree describing the authorized attribute sets [45].

A common representation is a *threshold tree*: interior nodes are AND/OR (or k-of-t) gates, and leaves correspond to individual attributes [45]. A k-of-t threshold gate is satisfied if at least k out of its t child nodes are satisfied. An AND gate corresponds to a t-of-t

gate (all children must be satisfied), while an OR gate corresponds to a 1-of-t gate (at least a child must be satisfied). Decryption succeeds when the attribute set attached to a ciphertext satisfies the tree, that is, there exists an assignment of attributes to leaves that makes every gate evaluate to true [45]. This model generalizes classic secret-sharing schemes, where a secret can be reconstructed only by a qualified subset of parties [45].

More expressive access structures can be captured by Linear Secret-Sharing Schemes (LSSS). Any monotone access structure realizable by an LSSS admits a matrix representation, enabling efficient key generation and delegation [45]. KP-ABE constructions support such structures, allowing fine-grained control over encrypted data—for example, audit-log entries labeled with attributes such as date, user, or IP subnet [45].

Bilinear Maps and the Decisional BDH Assumption

The security of many pairing-based schemes, including KP-ABE, relies on the hardness of the *Decisional Bilinear Diffie-Hellman (BDH)* problem. Let G_1 and G_2 be multiplicative cyclic groups of prime order p, with a bilinear map $e : G_1 \times G_1 \to G_2$ that is efficiently computable, bilinear, and non-degenerate [45].

Given a random generator $g \in G_1$ and random exponents $a, b, c \in \mathbb{Z}_p^*$, an adversary receives the tuple

$$(g,\ g^a,\ g^b,\ g^c).$$

The Decisional BDH assumption states that no polynomial-time algorithm can distinguish the element $e(g,g)^{abc}$ from a uniformly random element $Z \in G_2$ with non-negligible advantage [45]. Formally,

$$\Pr\Big[\mathcal{A}(g, g^a, g^b, g^c, e(g,g)^{abc}) = 1\Big] - \Pr\Big[\mathcal{A}(g, g^a, g^b, g^c, Z) = 1\Big] \leq \mathrm{negl}(p).$$

Intuitively, even though the adversary sees g^a, g^b, g^c, the pairing $e(g,g)^{abc}$ hides the product abc in a way that cannot be efficiently extracted. This hardness underpins the selective-set security proof of the KP-ABE construction: if an adversary could break the encryption scheme, it would yield an algorithm that solves the BDH problem, contradicting the assumption [45].

5.1.2 Mathematical Construction [45]

- **Setup.** Choose random $t_i \in \mathbb{Z}_p$ for each attribute $i \in \mathcal{U}$ and a master secret $y \in \mathbb{Z}_p$. Publish $\mathsf{PK} = \{g, g^y, \{T_i = g^{t_i}\}_{i\in\mathcal{U}}\}$ and keep $\mathsf{MK} = \{y, \{t_i\}_{i\in\mathcal{U}}\}$.
- **Key Generation.** For an access tree T, set $q_{\mathrm{root}}(0) = y$ and for each node x choose a random polynomial q_x of degree $k_x - 1$ with $q_x(0) = q_{\mathrm{parent}(x)}(\mathrm{index}(x))$. For a leaf x labeled by attribute i, output the key component $D_i = g^{q_x(0)/t_i}$.

- **Encryption.** To encrypt $M \in \mathbb{Z}_p$ under attribute set γ, pick $s \xleftarrow{\$} \mathbb{Z}_p$ and compute

$$E_0 = M \cdot e(g,g)^{ys}, \qquad E_i = T_i^s \quad \text{for } i \in \gamma.$$

  The ciphertext is $E = (E_0, \{E_i\}_{i\in\gamma})$.
- **Decryption.** Using D, recursively evaluate $\mathsf{DecryptNode}(E, D, x)$. At a leaf x with attribute i, compute $e(D_i, E_i) = e(g,g)^{q_x(0)s}$. At an internal node, combine child values $\{F_z\}$ via Lagrange coefficients $\Delta_{i,S}(0)$ to obtain $F_x = e(g,g)^{q_x(0)s}$. The root value yields $e(g,g)^{ys}$, which is divided out of E_0 to recover M.

Correctness follows from the secret-sharing reconstruction property, and security follows from the selective-set model under the DBDH assumption.

5.2 Ciphertext-Policy Attribute-Based Encryption

Ciphertext-Policy ABE is a public-key primitive in which the encryptor embeds an access policy into the ciphertext, while each user's private key is associated with a set of attributes [47]. In contrast to KP-ABE, which attaches the policy to the private key and the ciphertext carries only attributes, CP-ABE reverses this arrangement [47]. This inversion makes CP-ABE more natural for scenarios such as fine-grained data sharing, where a data owner wishes to specify who may decrypt directly at encryption time.

5.2.1 Mathematical Construction [47]

- **Access Structure.** Let $\mathcal{U} = \{1, \ldots, U\}$ be the universe of attributes. A monotone access structure $\mathcal{A}$ is a collection of authorized attribute subsets; it can be represented by a Linear Secret-Sharing Scheme (LSSS) matrix $M \in \mathbb{Z}_p^{\ell\times n}$ together with a mapping $\rho : \{1, \ldots, \ell\} \to \mathcal{U}$.
- **Setup.** Choose bilinear groups G_1, G_2 of prime order p with generator g and pairing $e : G_1 \times G_1 \to G_2$. For each attribute $x \in \mathcal{U}$, pick $h_x \in G_1$; publish $\mathsf{PK} = \{g, h_x\}$ and keep $\mathsf{MSK} = \{\alpha, a\}$, where g^α and g^a are also published.
- **Key Generation.** Given a set of attributes S, the authority chooses a random $t \in \mathbb{Z}_p$ and outputs

$$K_0 = g^\alpha g^{at}, \qquad K_x = g^t h_x \quad (x \in S).$$

  .
- **Encryption.** To encrypt message $M \in G_2$ under policy (M, ρ), pick a random vector $\mathbf{v} = (s, y_2, \ldots, y_n)$ and compute shares $\lambda_i = \mathbf{M}_i \cdot \mathbf{v}$ for each row i. The

ciphertext consists of

$$C' = M \cdot e(g,g)^{\alpha s}, \quad C_i = g^{\lambda_i} h_{\rho(i)}^{r_i}, \quad D_i = g^{r_i},$$

where $r_i \in \mathbb{Z}_p$ are random.

- **Decryption.** If the attribute set S satisfies the LSSS matrix, the user obtains constants $\{\omega_i\}$ such that $\sum_{i \in I} \omega_i \lambda_i = s$ (where I indexes rows whose attributes belong to S). By pairing the ciphertext components with the key components and raising each to ω_i, the user recovers $e(g,g)^{\alpha s}$ and thus M.

5.3 Correctness of Decryption

This section provides a formal verification of the correctness of both the KP-ABE and CP-ABE constructions presented above. The proofs follow directly from the properties of bilinear pairings and polynomial secret sharing.

5.3.1 Key-Policy ABE

We follow the notation from Section 5.1.2. Let $e : G_1 \times G_1 \to G_T$ be a bilinear map, and let $g \in G_1$ be a generator.

Encryption. Given a message $M \in G_T$ and attribute set γ, the ciphertext is:

$$E_0 = M \cdot e(g,g)^{ys}, \quad E_i = T_i^s = g^{t_i s} \quad \forall\, i \in \gamma.$$

Key Generation. For an access tree T, each node x is assigned a polynomial q_x such that:

$$q_{\text{root}}(0) = y.$$

For a leaf node x associated with attribute i, the key component is:

$$D_x = g^{q_x(0)/t_i}.$$

Decryption. At a leaf node x corresponding to attribute $i \in \gamma$, compute:

$$e(D_x, E_i) = e\Big(g^{q_x(0)/t_i},\ g^{t_i s}\Big) = e(g,g)^{q_x(0)s}.$$

For an interior node x, combine child nodes using Lagrange coefficients $\{\Delta_j\}$:

$$F_x = \prod_{j \in S_x} F_j^{\Delta_j} = e(g,g)^{q_x(0)s}.$$

At the root node:

$$F_{\text{root}} = e(g,g)^{ys}.$$

Finally, recover the message:

$$M = \frac{E_0}{F_{\text{root}}} = \frac{M \cdot e(g,g)^{ys}}{e(g,g)^{ys}} = M.$$

Thus, correctness follows from the secret-sharing reconstruction property of the access tree.

Example 5.3.1 (KP-ABE)**.** Let the universe of attributes be $\{1,2,3\}$ and consider an access tree T defined as:

$$(1 \wedge 2) \vee 3.$$

Suppose the ciphertext is associated with the attribute set $\gamma = \{1,2\}$.

During encryption, let the random secret be $s \in \mathbb{Z}_p$, so:

$$E_0 = M \cdot e(g,g)^{ys}, \quad E_1 = g^{t_1 s}, \quad E_2 = g^{t_2 s}.$$

During key generation, the authority assigns polynomials such that $q_{\text{root}}(0) = y$. The root is an OR gate with two children: the left child is an AND gate for attributes 1 and 2, and the right child is a leaf for attribute 3.

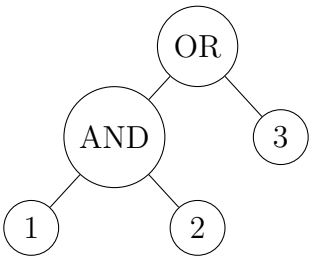


Figure 5.1: Access tree for the policy $(1 \wedge 2) \vee 3$.

Shares are distributed so that the leaves for attributes 1 and 2 receive values $q_1(0)$ and $q_2(0)$ satisfying the reconstruction of y. At decryption:

$$e(D_1, E_1) = e(g,g)^{q_1(0)s}, \quad e(D_2, E_2) = e(g,g)^{q_2(0)s}.$$

Since both attributes 1 and 2 are present, the AND node is satisfied, and by Lagrange interpolation:

$$F_{\text{AND}} = e(g,g)^{(q_1(0)+q_2(0))s} = e(g,g)^{ys}.$$

Thus the root is satisfied and $M = E_0 / F_{\text{root}} = M$.

5.3.2 Ciphertext-Policy ABE

We follow the notation from Section 5.2.1.

Encryption. Given access structure (M, ρ) and random vector $\vec{v} = (s, y_2, \ldots, y_n)$, define shares $\lambda_i = M_i \cdot \vec{v}$. Ciphertext components are:

$$C' = M \cdot e(g,g)^{\alpha s}, \quad C_i = g^{\lambda_i}, \quad D_i = h_{\rho(i)}^{\lambda_i}.$$

Key Generation. Given attribute set S, choose random $t \in \mathbb{Z}_p$ and compute:

$$K_0 = g^{\alpha} g^{at}, \quad K_x = g^t h_x^t \quad \forall\, x \in S.$$

Decryption. If S satisfies the access structure, there exist constants $\{w_i\}$ such that $\sum_{i \in I} w_i \lambda_i = s$. Compute:

$$\prod_{i \in I} \left(\frac{e(C_i, K_0)}{e(D_i, K_{\rho(i)})} \right)^{w_i}.$$

Expanding the pairings:

$$e(C_i, K_0) = e(g^{\lambda_i},\, g^{\alpha} g^{at}) = e(g,g)^{\alpha \lambda_i} \cdot e(g,g)^{at\lambda_i},$$

$$e(D_i, K_{\rho(i)}) = e(h_{\rho(i)}^{\lambda_i},\, h_{\rho(i)}^{t}) = e(h_{\rho(i)}, h_{\rho(i)})^{\lambda_i t}.$$

Using bilinearity and cancellation of attribute-dependent terms, one obtains:

$$\prod_{i \in I} e(g,g)^{at\lambda_i w_i} = e(g,g)^{ats},$$

which, combined with the remaining terms, yields $e(g,g)^{\alpha s}$. Finally:

$$M = \frac{C'}{e(g,g)^{\alpha s}}.$$

Thus, correctness follows from the LSSS reconstruction and bilinear pairing properties.

Example 5.3.2 (CP-ABE)**.** Let the access structure be $(1 \wedge 2)$, represented as an LSSS matrix:

$$M = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, \quad \rho(1) = 1,\ \rho(2) = 2.$$

Let $\vec{v} = (s, r)$, giving $\lambda_1 = s$, $\lambda_2 = r$. Ciphertext components:

$$C' = M \cdot e(g,g)^{\alpha s}, \quad C_1 = g^s,\ D_1 = h_1^s, \quad C_2 = g^r,\ D_2 = h_2^r.$$

Suppose a user holds attributes $S = \{1, 2\}$. Then there exist constants $w_1 = 1$, $w_2 = 0$ (or appropriate reconstruction coefficients) such that $w_1\lambda_1 + w_2\lambda_2 = s$.

Computing the ratio and combining using w_i:

$$\prod_{i \in I} \left(\frac{e(C_i, K_0)}{e(D_i, K_{\rho(i)})} \right)^{w_i} = e(g, g)^{\alpha s},$$

and therefore $M = C'/e(g, g)^{\alpha s} = M$.

5.4 Applications of Attribute-Based Encryption and the Case for Post-Quantum Constructions

Having established the mathematical foundations and constructions of both KP-ABE and CP-ABE in the preceding sections, this section examines where each paradigm finds its natural application domain, how the two paradigms diverge at the point of implementation, and why neither classical construction is sufficient for long-term security. This discussion motivates the transition to lattice-based ABE constructions presented in the sections that follow.

5.4.1 Policy Placement: The Defining Implementation Difference

The fundamental distinction between KP-ABE and CP-ABE lies not in the cryptographic hardness assumption each relies upon, but in *where* the access policy is embedded during the encryption and key-generation phases. This architectural choice determines which entity controls access and, consequently, which real-world deployment scenarios each scheme is suited for.

In **KP-ABE** [45], the encryptor labels a ciphertext with a descriptive attribute set, while the trusted authority embeds the access policy within the user's secret key. Formally, a ciphertext is associated with an attribute vector $\mathbf{x} \in \{0, 1\}^\ell$, and a secret key sk_C is generated for a policy circuit $C : \{0, 1\}^\ell \to \{0, 1\}$. Decryption succeeds if and only if $C(\mathbf{x}) = 0$. A critical consequence is that the encryptor need not be aware of who will eventually decrypt the message or under what policy; the encryptor need only correctly characterize the data being encrypted. This makes KP-ABE particularly suitable for scenarios in which data is produced continuously and access decisions are determined retrospectively by the authority.

In **CP-ABE** [47], this arrangement is inverted. The encryptor embeds the access policy C directly into the ciphertext, while the authority issues secret keys associated with user

attribute sets $S \subseteq \mathcal{U}$. A user whose attribute set satisfies the ciphertext policy can decrypt. This gives the encryptor direct and explicit control over access at the time of encryption, making CP-ABE more natural for scenarios in which the data owner wishes to enforce access at the point of publication.

Table 5.1: Structural Comparison of KP-ABE and CP-ABE

Component	**KP-ABE** [45]	**CP-ABE** [47]
Setup	Generates master public/secret key pair	Generates master public/secret key pair
KeyGen	Authority embeds policy C into sk_C	Authority embeds attribute set S into sk_S
Encrypt	Encryptor labels ciphertext with attribute vector $\mathbf{x}$	Encryptor embeds access policy C into ciphertext
Decrypt	Succeeds iff $C(\mathbf{x}) = 0$	Succeeds iff $C(S) = 1$
Policy Control	Authority (key-generation time)	Encryptor (encryption time)
Encryptor's Knowledge	Must describe attributes of the data	Must specify the intended access structure

The structural comparison in Table 5.1 indicates that KP-ABE is most suitable in settings where access decisions are centrally determined and the data producer is not required to define recipient-specific policies during encryption. In this model, ciphertexts are labeled only with descriptive attributes, while the authority embeds the access structure directly into the user's secret key. This arrangement is particularly appropriate for institutional environments in which access privileges are governed by administrative rules rather than by individual encryptors. **For the university document management system considered in this thesis, documents generated by academic or administrative units may be encrypted under attributes such as department, designation, or document category, while the university authority issues policy-bearing keys corresponding to approved access rights.** A faculty member, examination controller, or departmental administrator may therefore decrypt only when the attribute set attached to the ciphertext satisfies the policy encoded in the issued key. This motivates the transition from the classical constructions reviewed above to the LWE-based KP-ABE construction adopted in Section 5.7.

5.4.2 Application Domains of KP-ABE

KP-ABE is most naturally deployed in settings where data is tagged with observable, objective attributes and access control decisions are centralized, determined by a system

authority rather than the data producer. The following domains represent the primary application contexts.

Audit Log Systems and Forensic Data Management. In enterprise and governmental audit infrastructures, log records are continuously generated and labeled with metadata attributes such as timestamp, originating subsystem, event type, and severity classification. Administrators or compliance officers are issued secret keys embedded with policies reflecting their authorization level—for example, a policy permitting access to all `CRITICAL`-severity logs generated by the financial subsystem within a specified date range. The log producer (an automated system process) need only correctly tag each entry; the policy governing readability is encoded in the recipients' keys. This architecture aligns precisely with the KP-ABE model, as the encryptor neither knows nor needs to know the set of potential readers at encryption time [45].

Healthcare and Clinical Data Systems. In electronic health record (EHR) systems, patient records are labeled with clinical attributes such as diagnosis classification, ward identifier, attending physician code, and record sensitivity level. Healthcare professionals receive secret keys encoding policies aligned with their clinical role and departmental jurisdiction. The system generating and encrypting records need not anticipate the full set of future authorized readers, making KP-ABE a natural fit for continuously produced clinical data.

University Information Ecosystems. A further advantage of KP-ABE in academic information systems arises when encrypted records are produced in large volume by institutional platforms rather than by individual users. In such environments, documents may be tagged automatically with attributes reflecting source, category, or administrative status, while differentiated access is enforced later through policy-specific secret keys issued by the authority. This is particularly relevant for archival repositories, internal audit trails, and departmental databases where access requirements evolve over time without requiring re-encryption of stored material. The same framework extends naturally to examination records and restricted academic datasets, since a single attribute-labeled ciphertext may remain usable across multiple administrative roles provided that each issued key encodes the corresponding institutional policy.

5.4.3 Application Domains of CP-ABE

CP-ABE is most appropriate when the data owner is the primary decision-maker regarding access and when the intended access structure is known and expressible at the time of encryption [47].

Cloud Storage with Owner-Defined Access Control. When an individual or organization uploads sensitive documents to a cloud storage provider, they wish to specify directly who may access each file. Using CP-ABE, a document may be encrypted under the policy

$$C = \Big(\texttt{Role = Manager} \wedge \texttt{Dept = Finance}\Big) \vee \texttt{Role = CFO},$$

ensuring that only users whose attribute credentials satisfy C can decrypt. This data-owner-centric model is not readily achievable with KP-ABE without additional infrastructure.

Broadcast and Pay-Per-View Content Distribution. In subscription media services and pay-per-view broadcasting, content is encrypted under a policy specifying the subscriber class permitted to view it, such as $\texttt{Tier = Premium} \wedge \texttt{Region = India}$. Individual subscribers receive attribute-bearing keys from the service provider. The broadcaster, who determines the access policy at the time of content preparation, naturally operates in the CP-ABE model.

E-Government and Public Administration. Government agencies encrypting inter-departmental communications under CP-ABE can specify that a document is accessible to $\texttt{Role = TaxOfficer} \wedge \texttt{Jurisdiction = StateTax}$, with officials holding attribute keys issued by a central identity authority.

University Examination and Document Management. A university document system also illustrates the distinction between architectural suitability and implementation feasibility in attribute-based encryption. In principle, a faculty member preparing a mid-semester examination paper may wish to restrict access to a precisely defined recipient class—for example, through conditions involving departmental affiliation and examination responsibility. Such encryptor-defined access structures align naturally with the CP-ABE model, where the policy is embedded directly in the ciphertext. However, in lattice-based constructions this requires policy encoding during encryption and leads to significantly greater structural overhead when Boolean access rules must be translated into ciphertext components. For the present work, where the objective is a small-scale LWE-based demonstrative implementation, the KP-ABE formulation provides a more tractable route: examination documents are labeled with institutional attributes, while the university authority issues policy-bearing keys corresponding to approved academic roles. This preserves the same practical access semantics while allowing the lattice construction to remain closer to the key-generation framework developed in the BGG construction adopted in Section 5.7.

5.4.4 Why Classical ABE Is Insufficient for Future-Proof Systems

The KP-ABE and CP-ABE constructions discussed in the preceding sections—notably those of Goyal et al. [45] and Bethencourt et al. [47], which rely on bilinear map assumptions and the hardness of the Decisional Bilinear Diffie-Hellman (DBDH) problem—are computationally secure only against classical adversaries. The emergence of large-scale quantum computing poses a fundamental threat to these constructions.

Specifically, Shor's algorithm [12], when executed on a sufficiently powerful quantum computer, solves the discrete logarithm problem in groups supporting bilinear maps in polynomial time, directly breaking the DBDH assumption that underpins pairing-based ABE constructions. The timeline for cryptographically relevant quantum hardware remains an active area of research; however, the principle of *harvest now, decrypt later*—in which adversaries collect ciphertexts today for decryption once quantum capability matures—means that data with long-term confidentiality requirements is already at risk under classical ABE schemes.

The National Institute of Standards and Technology (NIST) has, through its Post-Quantum Cryptography (PQC) standardization process [17], explicitly identified lattice-based cryptography as a leading candidate family for quantum-resistant primitives. The hardness of the *Learning With Errors* (LWE) problem, introduced by Regev [39], is not known to be efficiently solvable by any quantum algorithm. This makes LWE-based ABE a principled and timely replacement for pairing-based constructions in security-critical deployments.

5.4.5 KP-ABE vs. CP-ABE in the LWE Setting

When transitioning from pairing-based to LWE-based constructions, the structural distinction between KP-ABE and CP-ABE is preserved but manifests differently, owing to the algebraic nature of lattice operations. Both paradigms share the same core mathematical machinery—gadget matrices, tensor product attribute encodings, and homomorphic circuit evaluation—but differ in which component carries the policy.

In **LWE-based KP-ABE** [50], the attribute vector $\mathbf{x} \in \{0,1\}^\ell$ is encoded into the ciphertext via the matrix term

$$\mathbf{B} - \mathbf{x}^T \otimes \mathbf{G} \ \in \ \mathbb{Z}_q^{n \times \ell m},$$

where $\mathbf{B} \in \mathbb{Z}_q^{n \times \ell m}$ is a public attribute-encoding matrix and $\mathbf{G} = \mathbf{I}_n \otimes \mathbf{g}^T$ is the gadget matrix with $\mathbf{g}^T = [1, 2, 4, \ldots, 2^{\lceil \log q \rceil - 1}]$. The policy circuit C is evaluated homomorphically

during key generation via

$$\mathbf{B}_C = \mathsf{EvalF}(\mathbf{B}, C) \ \in \ \mathbb{Z}_q^{n \times m},$$

and the user's secret key is a short lattice vector $\mathbf{y} \in \mathbb{Z}^{2m}$ satisfying

$$[\mathbf{A} \mid \mathbf{B}_C]\,\mathbf{y} = \mathbf{p} \pmod{q}.$$

Decryption exploits the homomorphic evaluation witness $\mathbf{H} = \mathsf{EvalFX}(\mathbf{B}, C, \mathbf{x})$, which satisfies the fundamental identity

$$\left(\mathbf{B} - \mathbf{x}^T \otimes \mathbf{G}\right) \cdot \mathbf{H} = \mathbf{B}_C - C(\mathbf{x}) \cdot \mathbf{G},$$

and succeeds precisely when $C(\mathbf{x}) = 0$, causing the $C(\mathbf{x}) \cdot \mathbf{G}$ term to vanish.

In **LWE-based CP-ABE** [51], this arrangement is inverted at the algebraic level. The policy circuit C is encoded into the ciphertext structure, while key generation produces a short vector $\mathbf{y}$ satisfying $[\mathbf{A} \mid \mathbf{B}_S]\mathbf{y} = \mathbf{p}$, where $\mathbf{B}_S = \mathsf{EvalF}(\mathbf{B}, S)$ encodes the user's attribute set S. The homomorphic evaluation at decryption then checks whether the ciphertext policy C is satisfied by the key's attribute set S.

Table 5.2 summarizes where the policy and attribute appear in the LWE setting for each paradigm.

Table 5.2: Policy and Attribute Placement in LWE-Based ABE

Scheme	**Ciphertext Contains**	**Secret Key Contains**	**Hom. Eval. at KeyGen**	**Hom. Eval. at Decrypt**
LWE KP-ABE [50]	Attribute $\mathbf{x}$ via $\mathbf{B} - \mathbf{x}^T \otimes \mathbf{G}$	Policy C via short $\mathbf{y}$ s.t. $[\mathbf{A} \mid \mathbf{B}_C]\mathbf{y} = \mathbf{p}$	$\mathbf{B}_C = \mathsf{EvalF}(\mathbf{B}, C)$	$\mathbf{H} = \mathsf{EvalFX}(\mathbf{B}, C, \mathbf{x})$
LWE CP-ABE [51]	Policy C encoded into ciphertext structure	Attribute set S via short $\mathbf{y}$ s.t. $[\mathbf{A} \mid \mathbf{B}_S]\mathbf{y} = \mathbf{p}$	$\mathbf{B}_S = \mathsf{EvalF}(\mathbf{B}, S)$	$\mathbf{H} = \mathsf{EvalFX}(\mathbf{B}, S, C)$

This thesis adopts the LWE-based KP-ABE construction of Boneh, Gentry, and Gorbunov [50], since it provides a structurally direct framework for representing institutional access policies through key generation while remaining suitable for small-scale numerical implementation. In the university document model considered here, encrypted records are labeled with descriptive attributes at creation time, whereas the institutional authority derives policy-bearing secret keys for authorized users according to administrative access requirements. The algebraic mechanisms underlying the construction—including the

gadget matrix $\mathbf{G}$, the encoding of the tensor product $\mathbf{x}^\top \otimes \mathbf{G}$, and the homomorphic evaluation procedures EvalF and EvalFX—are developed in the following sections and enter the KP-ABE setting through policy-dependent key derivation and attribute-based decryption, as summarized in Table 5.2. Section 5.7 presents the complete construction, establishes the correctness, and develops a numerical example consistent with the university deployment framework.

5.5 Mathematical Preliminaries for Lattice-Based ABE

The BGG+14 construction relies on three foundational tools: the gadget matrix and its inverse operator, the tensor product encoding of attribute vectors, and the lattice trapdoor mechanism. Each is developed below before the full scheme is presented.

5.5.1 The Gadget Matrix and $\mathbf{G}^{-1}$ Operator

The gadget matrix, introduced by Micciancio and Peikert [52], is a structured matrix whose primary role in lattice-based cryptography is to provide a deterministic, efficiently invertible encoding of integer vectors into their binary representations. It serves as a foundational component in the construction of attribute-based encryption from the Learning With Errors assumption, enabling homomorphic evaluation of Boolean circuits over encrypted data.

Definition 5.5.1 (Gadget Vector and Gadget Matrix)**.** Let $n \geq 1$ be a lattice dimension and let $q \geq 2$ be a modulus. Set $k = \lceil \log_2 q \rceil$. The *gadget vector* is defined as:

$$\mathbf{g}^T = \left[1,\ 2,\ 4,\ \ldots,\ 2^{k-1}\right] \ \in\ \mathbb{Z}_q^{1\times k}.$$

The *gadget matrix* is defined as:

$$\mathbf{G}_n = \mathbf{I}_n \otimes \mathbf{g}^T \ \in\ \mathbb{Z}_q^{n\times nk},$$

where $\mathbf{I}_n$ denotes the $n \times n$ identity matrix and $\otimes$ denotes the Kronecker product (defined in Section 5.5.1).

The explicit block structure of $\mathbf{G}_n$ is:

$$\mathbf{G}_n = \begin{bmatrix} 1 & 2 & \cdots & 2^{k-1} & 0 & 0 & \cdots & 0 & \cdots & 0 \\ 0 & 0 & \cdots & 0 & 1 & 2 & \cdots & 2^{k-1} & \cdots & 0 \\ \vdots & & & & & & & & \ddots & \vdots \\ 0 & 0 & \cdots & 0 & 0 & 0 & \cdots & 0 & \cdots & 2^{k-1} \end{bmatrix}.$$

Each row i of $\mathbf{G}_n$ contains the vector $\mathbf{g}^T$ placed at columns $((i-1)k+1)$ through ik, with zeros elsewhere. Consequently, multiplication by $\mathbf{G}_n$ recovers an n-dimensional integer vector from its binary representation.

The Gadget Matrix as a Linear Map. The gadget matrix can be viewed explicitly as a linear map:

$$G_n : \{0,1\}^{nk} \longrightarrow \mathbb{Z}_q^n.$$

The input vector is partitioned into n blocks of size k, and each block represents a binary expansion:

$$u_i = \sum_{j=0}^{k-1} b_{i,j} \cdot 2^j \quad \text{for each } i \in [n].$$

Thus, multiplication by G_n maps the binary vector back to its integer representation in $\mathbb{Z}_q^n$.

The $\mathbf{G}^{-1}$ Operator

The operator $\mathbf{G}^{-1} : \mathbb{Z}_q^n \to \{0,1\}^{nk}$ is a deterministic, component-wise binary decomposition map. For a vector $\mathbf{u} = [u_1, u_2, \ldots, u_n]^T \in \mathbb{Z}_q^n$, the map $\mathbf{G}^{-1}(\mathbf{u})$ expands each component u_i into its k-bit binary representation:

$$\mathbf{G}^{-1}(\mathbf{u}) = \Big[b_{1,0},\ b_{1,1},\ \ldots,\ b_{1,k-1},\ b_{2,0},\ \ldots,\ b_{n,k-1}\Big]^T \ \in\ \{0,1\}^{nk},$$

where $u_i = \sum_{j=0}^{k-1} b_{i,j} \cdot 2^j$ for each $i \in [n]$. The defining property of the pair $(\mathbf{G}_n, \mathbf{G}^{-1})$ is:

$$\mathbf{G}_n \cdot \mathbf{G}^{-1}(\mathbf{u}) = \mathbf{u} \quad \forall\, \mathbf{u} \in \mathbb{Z}_q^n. \tag{5.5}$$

This identity holds exactly over $\mathbb{Z}_q$ and follows from the definition of binary representation: writing u_i in binary and multiplying by the powers-of-two vector $\mathbf{g}^T$ recovers u_i.

The operator extends naturally to matrices: for a matrix $\mathbf{M} \in \mathbb{Z}_q^{n\times t}$, the map $\mathbf{G}^{-1}(\mathbf{M})$ applies $\mathbf{G}^{-1}$ column-wise, yielding a matrix in $\{0,1\}^{nk\times t}$. An important algebraic consequence of (5.5) used throughout lattice-based ABE constructions is the following. For any matrix $\mathbf{M} \in \mathbb{Z}_q^{n\times t}$:

$$\mathbf{G}_n \cdot \mathbf{G}^{-1}(\mathbf{M}) = \mathbf{M}. \tag{5.6}$$

Example 5.5.1 (Gadget Matrix Construction and Inversion)**.** Let $n = 2$, $q = 8$. Then $k = \lceil \log_2 8 \rceil = 3$ and $\mathbf{g}^T = [1, 2, 4]$.

Gadget matrix:

$$\mathbf{G}_2 = \mathbf{I}_2 \otimes \mathbf{g}^T = \begin{bmatrix} 1 & 2 & 4 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 2 & 4 \end{bmatrix} \in \mathbb{Z}_8^{2\times 6}.$$

Computing $\mathbf{G}^{-1}(\mathbf{u})$ for $\mathbf{u} = [5,\ 3]^T$:

Writing each component in 3-bit binary:

$$5 = 1\cdot 1 + 0\cdot 2 + 1\cdot 4 \ \Rightarrow\ [1,0,1], \qquad 3 = 1\cdot 1 + 1\cdot 2 + 0\cdot 4 \ \Rightarrow\ [1,1,0].$$

Therefore:

$$\mathbf{G}^{-1}([5,3]^T) = [1,0,1,1,1,0]^T \ \in\ \{0,1\}^6.$$

Verification of (5.5):

$$\mathbf{G}_2 \cdot [1,0,1,1,1,0]^T = \begin{bmatrix} 1 & 2 & 4 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 2 & 4 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 1 \\ 1 \\ 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1+0+4 \\ 1+2+0 \end{bmatrix} = \begin{bmatrix} 5 \\ 3 \end{bmatrix}. \quad \square$$

Remark 5.5.1. The entries of $\mathbf{G}^{-1}(\mathbf{u})$ are binary (i.e., elements of $\{0,1\}$) and thus have small norm. This property is critical in the security analysis of LWE-based constructions: multiplying by $\mathbf{G}^{-1}(\cdot)$ does not significantly amplify the norm of associated error vectors, preserving the correctness of decryption.

The Tensor Product $\mathbf{x}^T \otimes \mathbf{G}$

The Kronecker (tensor) product is a binary operation on matrices that constructs a larger matrix from two input matrices by systematically scaling and tiling one matrix by each scalar entry of the other. In LWE-based ABE, this operation is used to embed an attribute vector $\mathbf{x}$ into a matrix of the same dimensional structure as the attribute-encoding matrix $\mathbf{B}$, enabling the ciphertext to carry an encoding of the access attribute.

Definition 5.5.2 (Kronecker Product)**.** Let $\mathbf{P} \in \mathbb{Z}_q^{a\times b}$ and $\mathbf{Q} \in \mathbb{Z}_q^{c\times d}$. The *Kronecker*

product $\mathbf{P} \otimes \mathbf{Q} \in \mathbb{Z}_q^{ac \times bd}$ is defined by:

$$\mathbf{P} \otimes \mathbf{Q} = \begin{bmatrix} P_{11}\mathbf{Q} & P_{12}\mathbf{Q} & \cdots & P_{1b}\mathbf{Q} \\ P_{21}\mathbf{Q} & P_{22}\mathbf{Q} & \cdots & P_{2b}\mathbf{Q} \\ \vdots & \vdots & \ddots & \vdots \\ P_{a1}\mathbf{Q} & P_{a2}\mathbf{Q} & \cdots & P_{ab}\mathbf{Q} \end{bmatrix},$$

where each scalar entry P_{ij} of $\mathbf{P}$ is replaced by the block $P_{ij} \cdot \mathbf{Q}$.

Proposition 5.5.1. *The Kronecker product satisfies the* mixed-product property*: for matrices* $\mathbf{P}, \mathbf{R}$ *and* $\mathbf{Q}, \mathbf{S}$ *such that the products* $\mathbf{PR}$ *and* $\mathbf{QS}$ *are defined,*

$$(\mathbf{P} \otimes \mathbf{Q})(\mathbf{R} \otimes \mathbf{S}) = (\mathbf{PR}) \otimes (\mathbf{QS}). \tag{5.7}$$

This identity is used extensively in the correctness proofs of homomorphic evaluation.

Proof. The Kronecker product $\mathbf{P} \otimes \mathbf{Q}$ is defined as the block matrix with (i, j)-th block $p_{ij}\mathbf{Q}$, and $\mathbf{R} \otimes \mathbf{S}$ has (i, j)-th block $r_{ij}\mathbf{S}$. The (i, j)-th block of the product $(\mathbf{P} \otimes \mathbf{Q})(\mathbf{R} \otimes \mathbf{S})$ is:

$$\sum_k (p_{ik}\mathbf{Q})(r_{kj}\mathbf{S}) = \left(\sum_k p_{ik}r_{kj}\right)(\mathbf{QS}) = (\mathbf{PR})_{ij}(\mathbf{QS}).$$

Assembling all blocks:

$$(\mathbf{P} \otimes \mathbf{Q})(\mathbf{R} \otimes \mathbf{S}) = (\mathbf{PR}) \otimes (\mathbf{QS}). \quad \square$$

$\square$

The Attribute Encoding $\mathbf{x}^T \otimes \mathbf{G}$

In a KP-ABE scheme with attribute space $\{0, 1\}^\ell$, the parameter ℓ denotes the total number of attributes in the system. An attribute vector is given by $\mathbf{x} = [x_1, \ldots, x_\ell]^T \in \{0, 1\}^\ell$.

The public parameters include matrices

$$\mathbf{B} = [\mathbf{B}_1 \mid \mathbf{B}_2 \mid \cdots \mid \mathbf{B}_\ell] \in \mathbb{Z}_q^{n \times \ell m},$$

where each block $\mathbf{B}_i \in \mathbb{Z}_q^{n \times m}$ is sampled uniformly at random during setup. These matrices form the attribute-encoding component of the public key.

The ciphertext associated with attribute $\mathbf{x}$ contains the term:

$$\mathbf{x}^T \otimes G_n = [x_1 G_n \mid x_2 G_n \mid \cdots \mid x_\ell G_n] \ \in \ \mathbb{Z}_q^{n \times \ell m}.$$

Since $x_i \in \{0, 1\}$, each block $x_i G_n$ is either G_n (if $x_i = 1$) or the zero matrix (if $x_i = 0$).

Thus, the product $\mathbf{x}^T \otimes G_n$ selects the active attributes.

The matrix

$$\mathbf{B} - \mathbf{x}^T \otimes G_n$$

is called the *attribute-shifted matrix*, and it encodes the attribute vector $\mathbf{x}$ relative to the public parameter $\mathbf{B}$.

Example 5.5.2 (Computing $\mathbf{x}^T \otimes G_n$ and the Attribute-Shifted Matrix)**.** Let $n = 1$ and $q = 7$. Then $k = \lceil \log_2 q \rceil = 3$, and the gadget vector is $G_1 = \mathbf{g}^T = [1, 2, 4]$. Let $\ell = 3$ and $\mathbf{x} = [1, 0, 1]^T \in \{0, 1\}^3$.

Step 1: Compute $\mathbf{x}^T \otimes G_1$.

$$\mathbf{x}^T \otimes G_1 = [x_1 G_1 \mid x_2 G_1 \mid x_3 G_1] = [1, 2, 4 \mid 0, 0, 0 \mid 1, 2, 4] \in \mathbb{Z}_7^{1\times 9}.$$

Step 2: Define the public matrix B. Let $\mathbf{B}_1 = [5, 3, 6]$, $\mathbf{B}_2 = [2, 4, 1]$, $\mathbf{B}_3 = [6, 1, 5]$, so $\mathbf{B} = [5, 3, 6 \mid 2, 4, 1 \mid 6, 1, 5]$.

Step 3: Compute the attribute-shifted matrix.

$$\mathbf{B}-\mathbf{x}^T\otimes G_1 = [5, 3, 6 \mid 2, 4, 1 \mid 6, 1, 5]-[1, 2, 4 \mid 0, 0, 0 \mid 1, 2, 4] = [4, 1, 2 \mid 2, 4, 1 \mid 5, 6, 1] \pmod 7.$$

The first block $\mathbf{B}_1 - G_1 = [4, 1, 2]$ encodes $x_1 = 1$. The second block $\mathbf{B}_2 - \mathbf{0} = [2, 4, 1]$ encodes $x_2 = 0$ (unchanged). The third block $\mathbf{B}_3 - G_1 = [5, 6, 1]$ encodes $x_3 = 1$. The attribute vector $\mathbf{x}$ is thus embedded into the structure of $\mathbf{B} - \mathbf{x}^T \otimes G_1$, and recovery of the message during decryption depends on whether the decryption policy C is satisfied by $\mathbf{x}$.

5.5.2 Lattice Trapdoors

A lattice trapdoor is auxiliary information associated with a matrix $\mathbf{A} \in \mathbb{Z}_q^{n\times m}$ that enables efficient inversion of linear relations modulo q. While $\mathbf{A}$ appears pseudorandom (and hence hard to invert), the trapdoor allows one to find *short* solutions to equations of the form $\mathbf{Av} = \mathbf{y} \pmod q$.

Two fundamental algorithms underlie the trapdoor framework used in LWE-based ABE constructions [52, 53].

Definition 5.5.3 (Gadget Trapdoor [52])**.** A matrix $\mathbf{T}$ is called a gadget trapdoor for $\mathbf{A}$ if it enables efficient recovery of short preimages relative to the gadget matrix $\mathbf{G}_n$, typically via the relation

$$\mathbf{AT} = \mathbf{G}_n \pmod q,$$

with $\|\mathbf{T}\|$ bounded by a polynomial in n and $\log q$.

The two main algorithms used in the BGG+14 scheme [50] are as follows.

- $\mathsf{TrapGen}(1^n, q, m) \to (\mathbf{A}, \mathbf{T})$:
  Generates a matrix $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$ together with a trapdoor $\mathbf{T}$ such that:
  - $\mathbf{A}$ is statistically close to uniformly random, meaning it is computationally indistinguishable from a random matrix in $\mathbb{Z}_q^{n \times m}$,
  - $\mathbf{AT} = \mathbf{G}_n \pmod q$,
  - $\|\mathbf{T}\| = O(\sqrt{n \log q})$.

  Thus, $\mathbf{A}$ looks random to an adversary, but the trapdoor $\mathbf{T}$ enables structured inversion.
- $\mathsf{SamplePre}(\mathbf{A}, \mathbf{T}, \mathbf{y}, \sigma) \to \mathbf{v}$:
  Given $\mathbf{A}$, its trapdoor $\mathbf{T}$, a target vector $\mathbf{y} \in \mathbb{Z}_q^n$ in the column span of $\mathbf{A}$, and a Gaussian parameter σ, this algorithm outputs a vector $\mathbf{v} \in \mathbb{Z}^m$ such that

  $$\mathbf{Av} = \mathbf{y} \pmod q.$$

  Moreover, $\mathbf{v}$ is sampled from a discrete Gaussian distribution, meaning shorter vectors are exponentially more likely than longer ones. Formally, the output distribution is close to the discrete Gaussian $D_{\Lambda,\sigma}^{\mathbf{y}}$, that is, a Gaussian over all solutions to $\mathbf{Av} = \mathbf{y}$.

The preimage sampling algorithm is central to secret key generation in ABE schemes, where short vectors satisfying policy-dependent constraints must be produced without revealing the trapdoor.

The following standard result characterizes the output of $\mathsf{SamplePre}$.

Theorem 5.5.1 (Preimage Sampling [53])**.** *Let n, m, q be parameters with q prime and $m \geq 2n \log q$. For all but a q^{-n}-fraction of matrices $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$ and trapdoors $\mathbf{T}$ satisfying $\mathbf{AT} = \mathbf{G}_n$, the following holds: for any $\sigma \geq \|\mathbf{T}\| \cdot \sqrt{m} \log m$, the output of $\mathsf{SamplePre}(\mathbf{A}, \mathbf{T}, \mathbf{y}, \sigma)$ is statistically close to the discrete Gaussian over all solutions to $\mathbf{Av} = \mathbf{y}$, and satisfies $\|\mathbf{v}\| \leq \sigma\sqrt{m}$ with overwhelming probability.*

Example 5.5.3 (Trapdoor-Based Preimage Sampling.)**.** The following example illustrates how a lattice trapdoor enables efficient recovery of a short preimage. The parameters are deliberately small for clarity and do not provide security.

Parameter selection. Let $n = 1$, $q = 17$, $m = 2$. Define $\mathbf{G}_1 = [1,\ 2] \in \mathbb{Z}_{17}^{1 \times 2}$ and $\mathbf{A} = [3\ \ 5] \in \mathbb{Z}_{17}^{1 \times 2}$.

Trapdoor construction. Let

$$\mathbf{T} = \begin{bmatrix} 6 & 13 \\ 0 & 1 \end{bmatrix}.$$

Verification:

$$\mathbf{AT} = [3 \;\; 5] \begin{bmatrix} 6 & 13 \\ 0 & 1 \end{bmatrix} = [18,\ 44] \equiv [1,\ 2] = \mathbf{G}_1 \pmod{17}.$$

Preimage problem. Given target $y = 4 \in \mathbb{Z}_{17}$, find $\mathbf{v} = [v_1, v_2]^T$ such that $\mathbf{Av} = 4 \pmod{17}$.

Solution using the trapdoor. Since $4 = 4 \cdot 1 + 0 \cdot 2$, take $\mathbf{u} = [4,\ 0]^T$. Then:

$$\mathbf{v} = \mathbf{Tu} = \begin{bmatrix} 6 & 13 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 4 \\ 0 \end{bmatrix} = \begin{bmatrix} 24 \\ 0 \end{bmatrix} \equiv \begin{bmatrix} 7 \\ 0 \end{bmatrix} \pmod{17}.$$

Verification. $\mathbf{Av} = 3 \cdot 7 + 5 \cdot 0 = 21 \equiv 4 \pmod{17}$.

This example demonstrates how the trapdoor $\mathbf{T}$ enables efficient computation of a short preimage by reducing the inversion problem under $\mathbf{A}$ to a simple decomposition in the gadget basis. Without knowledge of $\mathbf{T}$, finding such a short vector is computationally hard, as it corresponds to solving an instance of the Short Integer Solution (SIS) problem.

Remark 5.5.2. The trapdoor $\mathbf{T}$ is kept secret by the authority. The matrix $\mathbf{A}$ is published. An adversary seeing only $\mathbf{A}$ cannot efficiently compute short preimages of arbitrary targets under $\mathbf{A}$, by the hardness of SIS. The trapdoor thus provides the authority with a capability that is computationally infeasible without it.

5.6 Homomorphic Evaluation over Lattice Encodings

The central mechanism enabling LWE-based KP-ABE to support general Boolean circuit policies is the lattice homomorphic evaluation procedure due to Gentry, Sahai, and Waters [54], further developed by Boneh et al. [50]. This procedure allows the decryptor to evaluate a Boolean circuit C on the attribute $\mathbf{x}$ *homomorphically* over the lattice encodings embedded in the ciphertext, without learning the underlying plaintext.

5.6.1 The Evaluation Algorithms: EvalF and EvalFX

The homomorphic evaluation framework consists of two efficient algorithms. Let $\mathbf{B} = [\mathbf{B}_1 \mid \cdots \mid \mathbf{B}_\ell] \in \mathbb{Z}_q^{n \times \ell m}$ be the attribute-encoding matrix from the master public key, and let $C : \{0,1\}^\ell \to \{0,1\}$ be a Boolean circuit of depth at most d.

The matrix $\mathbf{B}$ is called the *attribute-encoding matrix* because each block $\mathbf{B}_i \in \mathbb{Z}_q^{n \times m}$ corresponds to a specific attribute. For example, with $\ell = 3$ attributes representing

Student, Researcher, and Admin, the matrix is constructed as $\mathbf{B} = [\mathbf{B}_1 \mid \mathbf{B}_2 \mid \mathbf{B}_3]$, where each block encodes one attribute. This block-wise structure is what enables homomorphic evaluation over attributes, since Boolean circuits can act on the encoded components corresponding to each x_i.

- **EvalF**$(\mathbf{B}, C) \to \mathbf{B}_C$: An *input-independent* algorithm that takes the matrix $\mathbf{B}$ and circuit C and returns a matrix $\mathbf{B}_C \in \mathbb{Z}_q^{n \times m}$ that depends only on $\mathbf{B}$ and the structure of C, not on any specific input $\mathbf{x}$. This computation is performed once during key generation. For example, if $C(x_1, x_2) = x_1 \wedge x_2$, then $\mathbf{B}_C$ encodes only the AND structure of the circuit and remains the same regardless of whether the input is $(1, 1)$, $(1, 0)$, or $(0, 0)$.
- **EvalFX**$(\mathbf{B}, C, \mathbf{x}) \to \mathbf{H}_{\mathbf{B},C,\mathbf{x}}$: An *input-dependent* algorithm that takes $\mathbf{B}$, C, and an attribute $\mathbf{x} \in \{0,1\}^\ell$ and returns a matrix $\mathbf{H}_{\mathbf{B},C,\mathbf{x}} \in \mathbb{Z}_q^{\ell m \times m}$ with bounded norm $\|\mathbf{H}_{\mathbf{B},C,\mathbf{x}}\| \leq m^{O(d)}$. Unlike EvalF, this output explicitly depends on the chosen input $\mathbf{x}$. It is computed during decryption and is publicly deterministic given $(\mathbf{B}, C, \mathbf{x})$.

The norm bound on $\mathbf{H}_{\mathbf{B},C,\mathbf{x}}$ grows polynomially in m and exponentially in the circuit depth d. The modulus q in practical parameter choices is therefore set to accommodate this growth while maintaining the hardness of LWE.

5.6.2 The Key Identity

The correctness of homomorphic evaluation rests on the following algebraic identity, which holds for all matrices $\mathbf{B} \in \mathbb{Z}_q^{n \times \ell m}$, circuits $C : \{0,1\}^\ell \to \{0,1\}$, and inputs $\mathbf{x} \in \{0,1\}^\ell$:

$$\left(\mathbf{B} - \mathbf{x}^T \otimes \mathbf{G}_n\right) \cdot \mathbf{H}_{\mathbf{B},C,\mathbf{x}} = \mathbf{B}_C - C(\mathbf{x}) \cdot \mathbf{G}_n, \tag{5.8}$$

where $\mathbf{B}_C = \mathsf{EvalF}(\mathbf{B}, C)$ and $\mathbf{H}_{\mathbf{B},C,\mathbf{x}} = \mathsf{EvalFX}(\mathbf{B}, C, \mathbf{x})$.

The significance of identity (5.8) is most clearly seen in two cases:

(i) **Policy satisfied:** $C(\mathbf{x}) = 0$. The right-hand side reduces to $\mathbf{B}_C$, and the decryptor can use the secret key $\mathbf{y}$ satisfying $[\mathbf{A} \mid \mathbf{B}_C]\mathbf{y} = \mathbf{p}$ to recover the LWE-masked value $\mathbf{s}^T\mathbf{p}$ and thus the message.

(ii) **Policy not satisfied:** $C(\mathbf{x}) = 1$. The right-hand side becomes $\mathbf{B}_C - \mathbf{G}_n$. The extra $-\mathbf{G}_n$ term prevents recovery of $\mathbf{s}^T\mathbf{p}$: the linear equation the decryptor must solve shifts by $\mathbf{s}^T\mathbf{G}_n$, which encodes nontrivial information that cannot be removed without the secret key for a different policy.

In the BGG construction, the Boolean circuit C is defined as the logical negation of the intended access policy. Consequently, decryption succeeds if and only if $C(\mathbf{x}) = 0$, ensuring that the attribute vector $\mathbf{x}$ satisfies the original policy, while $C(\mathbf{x}) = 1$ introduces a

perturbation term that prevents successful decryption. Table 5.3 illustrates this convention for a two-attribute AND policy.

Table 5.3: Negation-based policy interpretation in the BGG scheme.

x_1	x_2	$x_1 \wedge x_2$	$C(\mathbf{x}) = \neg(x_1 \wedge x_2)$	Decryption
0	0	0	1	Fails
0	1	0	1	Fails
1	0	0	1	Fails
1	1	1	0	Succeeds

5.6.3 Gate-by-Gate Evaluation: AND, OR, and NOT

Any Boolean circuit C is composed of AND, OR, and NOT gates. The homomorphic evaluation algorithms process the circuit gate by gate, propagating both the evaluated matrix $\mathbf{B}_C$ and the evaluation hint $\mathbf{H}$ through each gate. Let $\mathbf{B}_1, \mathbf{B}_2 \in \mathbb{Z}_q^{n \times m}$ be the evaluated matrices corresponding to two wire values with inputs $x_1, x_2 \in \{0, 1\}$, satisfying:

$$\mathbf{B}_i - x_i\, \mathbf{G}_n = \Big(\mathbf{B} - x_i \mathbf{G}_n\Big) \cdot \mathbf{H}_i \quad \text{for } i \in \{1, 2\},$$

for appropriate hint matrices $\mathbf{H}_1, \mathbf{H}_2$.

NOT Gate. For a NOT gate with input x_1, the output is $\overline{x_1} = 1 - x_1$. The evaluated matrix is:

$$\mathbf{B}_{\mathsf{NOT}} = \mathbf{G}_n - \mathbf{B}_1.$$

Verification:

$$\mathbf{B}_{\mathsf{NOT}} - (1 - x_1)\mathbf{G}_n = (\mathbf{G}_n - \mathbf{B}_1) - \mathbf{G}_n + x_1\mathbf{G}_n = x_1\mathbf{G}_n - \mathbf{B}_1 = -(\mathbf{B}_1 - x_1\mathbf{G}_n).$$

The hint matrix for NOT is $\mathbf{H}_{\mathsf{NOT}} = -\mathbf{H}_1$.

AND Gate. For an AND gate with inputs x_1, x_2, the output is $x_1 \cdot x_2$. The evaluated matrix is:

$$\mathbf{B}_{\mathsf{AND}} = \mathbf{B}_1 \cdot \mathbf{G}^{-1}(\mathbf{B}_2).$$

The hint matrix is:

$$\mathbf{H}_{\mathsf{AND}} = \begin{bmatrix} \mathbf{G}^{-1}(\mathbf{B}_2) \\ x_1\, \mathbf{G}^{-1}(\mathbf{B}_2 - x_2\mathbf{G}_n) \end{bmatrix}.$$

Verification:

$$(\mathbf{B}_1 - x_1\mathbf{G}_n)\mathbf{G}^{-1}(\mathbf{B}_2) + x_1(\mathbf{B}_2 - x_2\mathbf{G}_n)$$

$$
\begin{aligned}
&= \mathbf{B}_1\mathbf{G}^{-1}(\mathbf{B}_2) - x_1\mathbf{G}_n\mathbf{G}^{-1}(\mathbf{B}_2) + x_1\mathbf{B}_2 - x_1x_2\mathbf{G}_n \\
&= \mathbf{B}_{\mathsf{AND}} - x_1\mathbf{B}_2 + x_1\mathbf{B}_2 - x_1x_2\mathbf{G}_n \\
&= \mathbf{B}_{\mathsf{AND}} - x_1x_2\,\mathbf{G}_n.
\end{aligned}
$$

Since $\mathsf{AND}(x_1, x_2) = x_1x_2$, this confirms identity (5.8) for the AND gate.

OR Gate. For an OR gate with inputs x_1, x_2, the output is $x_1 + x_2 - x_1x_2$. The evaluated matrix is:

$$\mathbf{B}_{\mathsf{OR}} = \mathbf{B}_1 + \mathbf{B}_2 - \mathbf{B}_1 \cdot \mathbf{G}^{-1}(\mathbf{B}_2).$$

Verification:

$$
\begin{aligned}
\mathbf{B}_{\mathsf{OR}} - (x_1 \vee x_2)\mathbf{G}_n &= \mathbf{B}_1 + \mathbf{B}_2 - \mathbf{B}_1\mathbf{G}^{-1}(\mathbf{B}_2) - (x_1 + x_2 - x_1x_2)\mathbf{G}_n \\
&= (\mathbf{B}_1 - x_1\mathbf{G}_n) + (\mathbf{B}_2 - x_2\mathbf{G}_n) - (\mathbf{B}_{\mathsf{AND}} - x_1x_2\mathbf{G}_n),
\end{aligned}
$$

from which the hint matrix is constructed accordingly.

Table 5.4 summarizes the evaluated matrices and their gate outputs.

Table 5.4: Summary of gate-by-gate homomorphic evaluation.

Gate	Output $C(\mathbf{x})$	Evaluated Matrix $\mathbf{B}_C$	Norm Bound
NOT	$1 - x_1$	$\mathbf{G}_n - \mathbf{B}_1$	$O(m)$
AND	$x_1 \cdot x_2$	$\mathbf{B}_1 \cdot \mathbf{G}^{-1}(\mathbf{B}_2)$	$O(m^2)$ per level
OR	$x_1 \vee x_2$	$\mathbf{B}_1 + \mathbf{B}_2 - \mathbf{B}_1\mathbf{G}^{-1}(\mathbf{B}_2)$	$O(m^2)$ per level

5.6.4 Numerical Example: Homomorphic Evaluation of a Two-Input AND Policy

This subsection verifies the homomorphic evaluation identity

$$(\mathbf{B} - \mathbf{x}^T \otimes \mathbf{G})\mathbf{H}_{\mathbf{B},C,\mathbf{x}} = \mathbf{B}_C - C(\mathbf{x})\mathbf{G}$$

for a simple AND policy, demonstrating how Boolean evaluation is embedded into lattice operations.

Parameters. $n = 1$, $q = 7$, $k = 3$, $m = 3$, $\mathbf{G} = [1, 2, 4] \in \mathbb{Z}_7^{1\times 3}$, $\ell = 2$, $C(\mathbf{x}) = x_1 \cdot x_2$.

Attribute-Encoding Matrix. $\mathbf{B}_1 = [1, 0, 0]$, $\mathbf{B}_2 = [2, 0, 0]$, $\mathbf{B} = [1, 0, 0,\ 2, 0, 0]$.

Step 1: Compute $\mathbf{B}_C = \mathsf{EvalF}(\mathbf{B}, C)$. Since $2 = 0 \cdot 1 + 1 \cdot 2 + 0 \cdot 4$, the gadget decomposition gives $\mathbf{G}^{-1}(2) = [0, 1, 0]^T$, and:

$$\mathbf{G}^{-1}(\mathbf{B}_2) = \begin{bmatrix} 0 & 0 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}.$$

Then:

$$\mathbf{B}_C = \mathbf{B}_1 \cdot \mathbf{G}^{-1}(\mathbf{B}_2) = [1, 0, 0] \begin{bmatrix} 0 & 0 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} = [0, 0, 0].$$

Step 2: Case $\mathbf{x} = (1, 1)$, so $C(\mathbf{x}) = 1$.

$$\mathbf{x}^T \otimes \mathbf{G} = [1, 2, 4,\ 1, 2, 4], \quad \mathbf{B} - \mathbf{x}^T \otimes \mathbf{G} = [0, 5, 3,\ 1, 5, 3] \pmod 7.$$

Multiplying by the hint matrix yields $[6, 5, 3] \pmod 7$. Right-hand side: $\mathbf{B}_C - \mathbf{G} = [0, 0, 0] - [1, 2, 4] = [6, 5, 3] \pmod 7$. The identity holds.

Step 3: Case $\mathbf{x} = (1, 0)$, so $C(\mathbf{x}) = 0$.

$$\mathbf{x}^T \otimes \mathbf{G} = [1, 2, 4,\ 0, 0, 0], \quad \mathbf{B} - \mathbf{x}^T \otimes \mathbf{G} = [0, 5, 3,\ 2, 0, 0] \pmod 7.$$

Multiplying by the hint matrix yields $[0, 0, 0]$. Right-hand side: $\mathbf{B}_C = [0, 0, 0]$. The identity holds.

When $C(\mathbf{x}) = 1$, the term $-\mathbf{G}$ appears, altering the relation. When $C(\mathbf{x}) = 0$, the expression reduces to $\mathbf{B}_C$. This confirms that homomorphic evaluation correctly encodes the AND policy within lattice algebra.

5.7 The BGG+14 Key-Policy Attribute-Based Encryption Scheme

The construction presented in this section is the Key-Policy Attribute-Based Encryption scheme due to Boneh, Gentry, Gorbunov, Halevi, Nikolaenko, Segev, Vaikuntanathan, and Vinayagamurthy [50], referred to hereafter as BGG+14. This scheme realizes, for the first time from lattice assumptions, a KP-ABE scheme supporting arbitrary Boolean circuit policies of bounded depth. The construction builds directly on the homomorphic evaluation procedure of Gentry, Sahai, and Waters [54] (reviewed in Section 5.6) and the lattice trapdoor machinery of Gentry, Peikert, and Vaikuntanathan [53] and Micciancio and Peikert [52] (Section 5.5.2).

The scheme operates in the following setting. An authority runs a one-time setup to produce a master public key and a master secret key. Individual users are associated with *decryption policies*, expressed as Boolean circuits, and the authority issues a corresponding secret key using the master secret key. Any party may encrypt a message with respect to a set of attributes. A user holding a secret key for policy C can decrypt a ciphertext associated with attribute vector $\mathbf{x}$ if and only if $C(\mathbf{x}) = 0$, following the convention of [50] in which a satisfied policy evaluates to zero.

Parameter Setting. Let λ be the security parameter. Let $n = n(\lambda)$ denote the lattice dimension, $q = q(\lambda)$ a prime modulus, $m = m(\lambda) \geq 2n \log q$ the number of LWE samples, $\ell = \ell(\lambda)$ the number of attributes, and $d = d(\lambda)$ the maximum depth of the policy circuit. Set $k = \lceil \log_2 q \rceil$. The attribute space is $\mathcal{X} = \{0,1\}^\ell$ and the policy space $\mathcal{C}$ consists of Boolean circuits $C : \{0,1\}^\ell \to \{0,1\}$ of depth at most d.

5.7.1 Setup

The setup algorithm is executed once by the authority. Its output consists of a master public key MPK and a master secret key MSK. The security of the scheme rests on the hardness of the LWE problem with respect to the matrix $\mathbf{A}$ generated in this step [50, 39].

Algorithm 1 $\mathsf{Setup}(1^\lambda, 1^\ell, 1^d)$

Require: Security parameter λ, attribute length ℓ, circuit depth bound d.
Ensure: Master public key MPK, master secret key MSK.

1: Run $(\mathbf{A}, \mathbf{T}) \leftarrow \mathsf{TrapGen}(1^n, q, m)$ to obtain a matrix $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$ statistically close to uniform, together with its gadget trapdoor $\mathbf{T}$ [52].
2: Sample $\mathbf{B}_i \xleftarrow{\$} \mathbb{Z}_q^{n \times m}$ independently and uniformly at random for each $i \in [\ell]$.
3: Set $\mathbf{B} = [\mathbf{B}_1 \mid \mathbf{B}_2 \mid \cdots \mid \mathbf{B}_\ell] \in \mathbb{Z}_q^{n \times \ell m}$.
4: Sample $\mathbf{p} \xleftarrow{\$} \mathbb{Z}_q^n$ uniformly at random.
5: Output $\mathsf{MPK} = (\mathbf{A}, \mathbf{B}, \mathbf{p})$ and $\mathsf{MSK} = \mathbf{T}$.

The matrix $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$ is the base LWE matrix. The matrices $\mathbf{B}_1, \ldots, \mathbf{B}_\ell \in \mathbb{Z}_q^{n \times m}$ encode the ℓ individual attributes: $\mathbf{B}_i$ is associated with the i-th attribute bit x_i. The vector $\mathbf{p} \in \mathbb{Z}_q^n$ serves as the target for the key generation preimage problem. The trapdoor $\mathbf{T}$ is held secret by the authority and enables the computation of short preimages under $\mathbf{A}$, which is the core operation in key generation. As noted in [50], the joint distribution $(\mathbf{A}, \mathbf{B}_1, \ldots, \mathbf{B}_\ell, \mathbf{p})$ is computationally indistinguishable from a tuple of independently and uniformly random matrices and a vector, by the trapdoor distribution property of $\mathsf{TrapGen}$ [52].

5.7.2 Key Generation

The key generation algorithm is executed by the authority on behalf of a user whose decryption rights correspond to the Boolean circuit $C : \{0,1\}^\ell \to \{0,1\}$. The algorithm produces a secret key sk_C that permits decryption of any ciphertext whose attribute satisfies $C(\mathbf{x}) = 0$.

The central step is the computation of the *policy-evaluated matrix* $\mathbf{B}_C = \mathsf{EvalF}(\mathbf{B}, C)$, introduced in Section 5.6.1. Recall from the gate-by-gate analysis (Section 5.6.3) that $\mathbf{B}_C \in \mathbb{Z}_q^{n \times m}$ is derived by propagating the attribute matrices $\mathbf{B}_1, \ldots, \mathbf{B}_\ell$ through the circuit C using gadget-based homomorphic operations on each gate. This computation depends only on $\mathbf{B}$ and C, not on any specific attribute value $\mathbf{x}$.

Algorithm 2 $\mathsf{KeyGen}(\mathsf{MPK}, \mathsf{MSK}, C)$

Require: Master public key $\mathsf{MPK} = (\mathbf{A}, \mathbf{B}, \mathbf{p})$, master secret key $\mathsf{MSK} = \mathbf{T}$, policy circuit $C \in \mathcal{C}$.

Ensure: Secret key sk_C.

1: Compute $\mathbf{B}_C \leftarrow \mathsf{EvalF}(\mathbf{B}, C) \in \mathbb{Z}_q^{n \times m}$ using the homomorphic evaluation procedure of [50, 54] (Section 5.6.1).

2: Form the augmented matrix $[\mathbf{A} \mid \mathbf{B}_C] \in \mathbb{Z}_q^{n \times 2m}$.

3: Using the trapdoor $\mathbf{T}$ for $\mathbf{A}$, run $\mathbf{y} \leftarrow \mathsf{SamplePre}(\mathbf{A}, \mathbf{T}, \mathbf{p}, \sigma)$ for an appropriate Gaussian width σ, to obtain a short vector $\mathbf{y} \in \mathbb{Z}^{2m}$ satisfying:

$$[\mathbf{A} \mid \mathbf{B}_C] \cdot \mathbf{y} = \mathbf{p} \pmod{q}. \tag{5.9}$$

4: Output $\mathsf{sk}_C = \mathbf{y}$.

A few points merit attention. First, the short vector $\mathbf{y}$ satisfying (5.9) is obtained via $\mathsf{SamplePre}$ [53], with width parameter $\sigma \geq \|\mathbf{T}\| \cdot \sqrt{m} \log m$ to ensure statistical closeness to a discrete Gaussian over the appropriate lattice coset [53, 52]. Second, there exist many short vectors satisfying (5.9); the trapdoor allows the authority to sample one with bounded norm, while an adversary without the trapdoor cannot efficiently find such a vector – doing so would require solving a SIS instance [56]. Third, the matrix $\mathbf{B}_C$ depends on the circuit C, so different users with different policies receive secret keys whose preimage relation involves different augmented matrices. This is what prevents a user with policy C from decrypting ciphertexts for which $C(\mathbf{x}) \neq 0$.

5.7.3 Encryption

The encryption algorithm is executed by any party in possession of MPK. It takes a message $\mu \in \{0,1\}$ and an attribute vector $\mathbf{x} = [x_1, \ldots, x_\ell]^T \in \{0,1\}^\ell$ and produces

a ciphertext CT. The construction follows the dual-Regev encryption paradigm [53], extended with the attribute encoding $\mathbf{x}^T \otimes \mathbf{G}_n$ as introduced in [50].

Algorithm 3 $\mathsf{Encrypt}(\mathsf{MPK}, \mathbf{x}, \mu)$

Require: Master public key $\mathsf{MPK} = (\mathbf{A}, \mathbf{B}, \mathbf{p})$, attribute $\mathbf{x} \in \{0,1\}^\ell$, message $\mu \in \{0,1\}$.
Ensure: Ciphertext $\mathsf{CT} = (\mathbf{c}_0, \mathbf{c}_1, c_2)$.

1: Sample $\mathbf{s} \xleftarrow{\$} \mathbb{Z}_q^n$, and small error vectors $e_0, e_1 \leftarrow D^m_{\mathbb{Z},\sigma_{\mathsf{LWE}}}$, and a small scalar $e_2 \leftarrow D_{\mathbb{Z},\sigma_{\mathsf{LWE}}}$, where σ_{LWE} is the LWE error width [39].

2: Compute:

$$\mathbf{c}_0 = \mathbf{s}^T\mathbf{A} + e_0^T \ \in \ \mathbb{Z}_q^m. \tag{5.10}$$

3: Compute the attribute-encoded component:

$$\mathbf{c}_1 = \mathbf{s}^T(\mathbf{B} - \mathbf{x}^T \otimes \mathbf{G}_n) + e_1^T \ \in \ \mathbb{Z}_q^{\ell m}. \tag{5.11}$$

4: Compute the message-encoding component:

$$c_2 = \mathbf{s}^T\mathbf{p} + e_2 + \mu \cdot \lfloor q/2 \rfloor \ \in \ \mathbb{Z}_q. \tag{5.12}$$

5: Output $\mathsf{CT} = (\mathbf{c}_0, \mathbf{c}_1, c_2)$.

Each component of the ciphertext carries a specific role. The component $\mathbf{c}_0$ is a standard LWE sample under the matrix $\mathbf{A}$: it commits the encryptor to the secret $\mathbf{s}$ relative to $\mathbf{A}$ while hiding $\mathbf{s}$ under the LWE hardness assumption [39]. The component $\mathbf{c}_1$ encodes the attribute $\mathbf{x}$ via the tensor term $\mathbf{x}^T \otimes \mathbf{G}_n$ (see Section 5.5.1): it is an LWE sample under the shifted matrix $\mathbf{B} - \mathbf{x}^T \otimes \mathbf{G}_n$, which interacts with the homomorphic evaluation hint $\mathbf{H}_{\mathbf{B},C,\mathbf{x}}$ during decryption. The component c_2 masks the message μ under the LWE-hard value $\mathbf{s}^T\mathbf{p}$: since $\mu \in \{0,1\}$, the message is encoded as $\mu \cdot \lfloor q/2 \rfloor$, placing it either near 0 or near $q/2$ in $\mathbb{Z}_q$, which permits recovery by rounding [50, 53].

5.7.4 Decryption

The decryption algorithm is executed by a user holding $\mathsf{sk}_C = \mathbf{y}$ for a policy C and a ciphertext $\mathsf{CT} = (\mathbf{c}_0, \mathbf{c}_1, c_2)$ associated with attribute $\mathbf{x}$. The algorithm succeeds in recovering μ if and only if $C(\mathbf{x}) = 0$.

The procedure proceeds in three phases. In the first phase, the decryptor evaluates the circuit C on $\mathbf{x}$ homomorphically over the lattice encodings in $\mathbf{c}_1$ using EvalFX. In the second phase, it applies the secret key $\mathbf{y}$ to compute an approximation of $\mathbf{s}^T\mathbf{p}$. In the third phase, it subtracts this approximation from c_2 and rounds to recover μ. The correctness argument follows directly from identity (5.8) of Section 5.6.2 [50].

Algorithm 4 $\mathsf{Decrypt}(\mathsf{sk}_C, \mathsf{CT}, \mathbf{x})$

Require: Secret key $\mathsf{sk}_C = \mathbf{y} \in \mathbb{Z}^{2m}$, ciphertext $\mathsf{CT} = (\mathbf{c}_0, \mathbf{c}_1, c_2)$, attribute $\mathbf{x} \in \{0,1\}^\ell$.
Ensure: Message $\mu \in \{0,1\}$ if $C(\mathbf{x}) = 0$; otherwise decryption fails.

1: Compute the evaluation hint:

$$\mathbf{H} \leftarrow \mathsf{EvalFX}(\mathbf{B}, C, \mathbf{x}) \in \mathbb{Z}_q^{\ell m \times m}.$$

2: Apply $\mathbf{H}$ to compress the attribute component:

$$\mathbf{w} = \mathbf{c}_1 \cdot \mathbf{H} \in \mathbb{Z}_q^m. \tag{5.13}$$

3: Using the key identity (5.8) with $C(\mathbf{x}) = 0$, note that $\mathbf{w} \approx \mathbf{s}^T\mathbf{B}_C$. Form the concatenated vector $[\mathbf{c}_0 \mid \mathbf{w}] \in \mathbb{Z}_q^{2m}$ and compute:

$$v = [\mathbf{c}_0 \mid \mathbf{w}] \cdot \mathbf{y} \ \in \ \mathbb{Z}_q. \tag{5.14}$$

Since $\mathbf{y}$ satisfies $[\mathbf{A} \mid \mathbf{B}_C]\mathbf{y} = \mathbf{p}$, this gives $v \approx \mathbf{s}^T\mathbf{p}$.

4: Recover the message by:

$$\mu' = c_2 - v \pmod q. \tag{5.15}$$

5: Round: output $\hat{\mu} = 0$ if $|\mu'| < q/4$, and $\hat{\mu} = 1$ otherwise (i.e., if μ' is closer to $\lfloor q/2 \rfloor$ than to 0).

5.7.5 Correctness of BGG+14

Theorem 5.7.1 (Correctness of BGG+14 [50])**.** *Let the parameters be chosen so that $q > 4m^{O(d)} \cdot \sigma_{\mathsf{LWE}} \cdot \sigma_{\mathsf{key}}$, where σ_{key} is the Gaussian width used in* SamplePre *during key generation. Then for all $C \in \mathcal{C}$, all $\mathbf{x} \in \{0,1\}^\ell$ with $C(\mathbf{x}) = 0$, and all honestly generated key-ciphertext pairs, the decryption algorithm outputs $\hat{\mu} = \mu$ with probability 1 over the randomness of key generation and encryption.*

The correctness argument proceeds as follows. When $C(\mathbf{x}) = 0$, identity (5.8) gives:

$$\mathbf{c}_1 \cdot \mathbf{H} = \mathbf{s}^T(\mathbf{B} - \mathbf{x}^T \otimes \mathbf{G}_n)\mathbf{H} + e_1^T\mathbf{H} = \mathbf{s}^T\mathbf{B}_C + e_1^T\mathbf{H}.$$

The term $e_1^T\mathbf{H}$ is small because e_1 is a discrete Gaussian sample and $\|\mathbf{H}\| \le m^{O(d)}$ [50, 54]. Consequently:

$$v = [\mathbf{c}_0 \mid \mathbf{w}] \cdot \mathbf{y} \approx \mathbf{s}^T[\mathbf{A} \mid \mathbf{B}_C]\mathbf{y} = \mathbf{s}^T\mathbf{p}.$$

Substituting into (5.15):

$$\mu' = c_2 - v \approx \mathbf{s}^T\mathbf{p} + e_2 + \mu\lfloor q/2 \rfloor - \mathbf{s}^T\mathbf{p} = \mu\lfloor q/2 \rfloor + e_2 + (\text{small errors}).$$

Since all error terms are bounded by $q/4$ under the parameter constraints of Theorem 5.7.1, rounding recovers μ exactly.

When $C(\mathbf{x}) = 1$, identity (5.8) instead gives $\mathbf{c}_1 \cdot \mathbf{H} \approx \mathbf{s}^T(\mathbf{B}_C - \mathbf{G}_n)$, so:

$$v \approx \mathbf{s}^T\mathbf{p} - \mathbf{s}^T\mathbf{G}_n\mathbf{y}_2,$$

where $\mathbf{y}_2$ is the second block of $\mathbf{y}$. This extra term $\mathbf{s}^T\mathbf{G}_n\mathbf{y}_2$ is in general not small relative to q, and decryption fails [50].

Remark 5.7.1 (Security). The BGG+14 scheme satisfies *attribute-selective security* under the LWE assumption [50, 39]: no polynomial-time adversary can distinguish encryptions of $\mu = 0$ and $\mu = 1$ under attribute $\mathbf{x}^*$ when it commits to $\mathbf{x}^*$ before seeing the master public key, provided it holds no secret key for any policy C with $C(\mathbf{x}^*) = 0$. Adaptive security follows from selective security via complexity leveraging under sub-exponential LWE hardness, at the cost of a polynomial increase in parameters. Since proving the full security reduction would require developing the LWE-to-ABE reduction machinery of [50] in detail, and lies beyond the scope of this thesis, the reader is referred to [50, 55] for the formal proof.

5.7.6 Selective Security of BGG+14

Theorem 5.7.2 (Selective Security [50])**.** *If the Learning With Errors problem $LWE_{n,q,\chi}$ is hard, then the BGG+14 KP-ABE scheme is selectively secure against any probabilistic polynomial-time adversary, in the sense of the selective-attribute IND-CPA security game.*

The proof proceeds by a reduction: any adversary $\mathcal{A}$ that wins the selective security game with non-negligible advantage can be used to construct an algorithm $\mathcal{B}$ that solves LWE with the same advantage, contradicting the hardness assumption. The argument is structured as a sequence of two games.

Security Game (Selective-Attribute IND-CPA). The game between a challenger $\mathcal{C}$ and adversary $\mathcal{A}$ proceeds in four phases.

1. **Commit.** Before seeing any system parameters, $\mathcal{A}$ declares a challenge attribute set $\mathbf{x}^* \in \{0,1\}^n$. This is the defining restriction of selective security: the adversary commits to their target before the public key is generated.
2. **Setup.** $\mathcal{C}$ runs $\mathsf{Setup}(1^\lambda)$ and sends the master public key MPK to $\mathcal{A}$. The master secret key MSK is kept private.
3. **Key Query Phase.** $\mathcal{A}$ may adaptively request secret keys for any access policy f of their choice, subject to the restriction that $f(\mathbf{x}^*) = 0$. That is, the adversary cannot query a key for any policy that the challenge attribute set satisfies, since that

would trivially break security. $\mathcal{C}$ responds to each query by running $\mathsf{KeyGen}(\mathsf{MSK}, f)$ and returning the resulting key.

4. **Challenge.** $\mathcal{A}$ submits two equal-length messages $\mu_0, \mu_1 \in \{0,1\}$. $\mathcal{C}$ samples a random bit $b \leftarrow \{0,1\}$, encrypts μ_b under $\mathbf{x}^*$ to produce a ciphertext $\mathbf{ct}^*$, and sends it to $\mathcal{A}$.
5. **Guess.** $\mathcal{A}$ outputs a guess b'. The adversary wins if $b' = b$.

The advantage of $\mathcal{A}$ is defined as $\mathrm{Adv}(\mathcal{A}) = \left|\Pr[b' = b] - \frac{1}{2}\right|$. The scheme is selectively secure if this advantage is negligible in λ for all PPT adversaries.

Proof Sketch (Reduction to LWE). The reduction $\mathcal{B}$ receives an LWE instance $(\mathbf{A}, \mathbf{b})$ where either $\mathbf{b} = \mathbf{A}^\top \mathbf{s} + \mathbf{e}$ for a secret $\mathbf{s}$ and small error $\mathbf{e} \leftarrow \chi^m$ (the LWE case), or $\mathbf{b}$ is uniformly random (the random case). $\mathcal{B}$ must decide which case it is in.

Game 0. This is the real selective security game, played with the honestly generated public key and a real encryption of μ_b.

Game 1. The ciphertext is replaced by a uniformly random vector, independent of both μ_0 and μ_1. Since a uniformly random ciphertext carries no information about b, the adversary's advantage in Game 1 is exactly zero.

The key step is showing that no PPT adversary can distinguish Game 0 from Game 1. This follows directly from the hardness of LWE. Specifically, $\mathcal{B}$ embeds the LWE instance into the public parameters as follows.

- $\mathcal{B}$ sets the public matrix $\mathbf{A}$ in MPK to be the LWE matrix received from its own challenger.
- For each attribute i, $\mathcal{B}$ constructs the attribute matrices $\mathbf{B}_i$ using the challenge attribute set $\mathbf{x}^*$ declared by $\mathcal{A}$ in the Commit phase. Because $\mathbf{x}^*$ is known before MPK is generated, $\mathcal{B}$ can program the matrices so that the trapdoor structure is preserved for all key queries (where $f(\mathbf{x}^*) = 0$) but is absent for the challenge ciphertext (where $f(\mathbf{x}^*) = 1$). This is precisely why the proof requires the commit step: without knowing $\mathbf{x}^*$ in advance, $\mathcal{B}$ cannot program the public parameters correctly.
- To answer key queries for a policy f with $f(\mathbf{x}^*) = 0$, $\mathcal{B}$ uses the trapdoor it has embedded in the public parameters to run KeyGen honestly.
- For the challenge ciphertext, $\mathcal{B}$ uses the LWE sample $\mathbf{b}$ as the ciphertext randomness. If $\mathbf{b} = \mathbf{A}^\top \mathbf{s} + \mathbf{e}$, this produces a real encryption of μ_b (Game 0). If $\mathbf{b}$ is uniformly random, this produces a random ciphertext (Game 1).

If $\mathcal{A}$ distinguishes Game 0 from Game 1 with advantage ϵ, then $\mathcal{B}$ solves the LWE instance with the same advantage ϵ. Since LWE is assumed hard, ϵ must be negligible. Therefore

the advantage of any PPT adversary in the selective security game is negligible. This follows the proof strategy of [50], adapted here for exposition.

□

Remark. The commit step is not merely a proof artifact. It reflects a genuine limitation: without knowing $\mathbf{x}^*$ before generating MPK, the reduction cannot program the attribute matrices to simultaneously support key queries and embed the LWE instance into the challenge ciphertext. Removing this restriction, that is, achieving adaptive security where $\mathcal{A}$ chooses $\mathbf{x}^*$ after seeing MPK, requires either complexity leveraging at the cost of a superpolynomial security loss, or stronger assumptions beyond plain LWE. This remains an active direction in lattice-based ABE research, with recent progress in [51] and [43].

5.8 Complete Numerical Example of the BGG+14 KP-ABE Scheme

This section presents a fully explicit, end-to-end execution of the BGG+14 KP-ABE scheme. Every intermediate computation is shown in detail so that the reader can independently verify each result. Both a successful decryption (authorized user) and a failed decryption (unauthorized user) are demonstrated numerically.

5.8.1 System Parameters

- **Lattice dimension:** $n = 1$; **Modulus:** $q = 17$
- **Gadget vector:** $\mathbf{G}_1 = [1, 2, 4, 8, 16]$
- **Number of attributes:** $\ell = 2$

For this example, the two attributes are:

- Attribute 1: “Mathematics Faculty”
- Attribute 2: “PhD Scholar”

A user’s attribute vector $\mathbf{x} = (x_1, x_2)$ is defined as $x_i = 1$ if the user possesses attribute i and $x_i = 0$ otherwise. The user in the authorized case possesses both attributes: $\mathbf{x} = (1, 1)$.

The access policy encoded in the ciphertext is:

$$C(\mathbf{x}) = 1 - x_1\, x_2.$$

Since $x_1 = x_2 = 1$, the policy evaluates to $C(\mathbf{x}) = 0$, meaning the policy is satisfied and decryption should succeed for this user.

5.8.2 Step 1: Setup

The authority publishes the matrices $\mathbf{A}$ and $\mathbf{B}$, and the secret vector is derived from the trapdoor. For this illustrative example, concrete values are fixed as:

$$\mathbf{A} = [2,\ 5,\ 3,\ 7,\ 1], \quad \mathbf{B}_1 = [4,\ 6,\ 1,\ 9,\ 3], \quad \mathbf{B}_2 = [8,\ 2,\ 5,\ 10,\ 7], \quad \mathbf{p} = [6].$$

5.8.3 Step 2: Key Generation

(a) Bit Decomposition of $\mathbf{B}_2$. Each entry of $\mathbf{B}_2$ is written in binary with the least-significant bit first:

$$8 \to [0,0,0,1,0], \quad 2 \to [0,1,0,0,0], \quad 5 \to [1,0,1,0,0], \quad 10 \to [0,1,0,1,0], \quad 7 \to [1,1,1,0,0].$$

Assembling these as columns gives:

$$\mathbf{G}^{-1}(\mathbf{B}_2) = \begin{bmatrix} 0 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 & 1 \\ 0 & 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}.$$

(b) AND Gate Evaluation. The AND gate matrix is computed as $\mathbf{B}_{\mathsf{AND}} = \mathbf{B}_1 \cdot \mathbf{G}^{-1}(\mathbf{B}_2) \pmod{17}$, giving:

$$\mathbf{B}_{\mathsf{AND}} = [9,\ 6,\ 5,\ 15,\ 11].$$

(c) NOT Gate (Policy Complement). Applying the NOT gate yields the final policy matrix:

$$\mathbf{B}_C = \mathbf{G}_1 - \mathbf{B}_{\mathsf{AND}} = [1,2,4,8,16] - [9,6,5,15,11] \equiv [9,\ 13,\ 16,\ 10,\ 5] \pmod{17}.$$

(d) Secret Key Derivation. The secret key $\mathbf{y}$ must satisfy the linear system $[\mathbf{A} \mid \mathbf{B}_C]\,\mathbf{y} \equiv \mathbf{p} \pmod{q}$, which reduces here to:

$$2\,y_1 + 9\,y_6 \equiv 6 \pmod{17}.$$

Candidates are tested as follows:

y_1	y_6	$2y_1$	$9y_6$	$2y_1 + 9y_6 \bmod 17$	
2	5	4	45	$49 \bmod 17 = 15$	$\times$
4	4	8	36	$44 \bmod 17 = 10$	$\times$
5	1	10	9	$19 \bmod 17 = 2$	$\times$
6	5	12	45	$57 \bmod 17 = 6$	✓

The valid solution is $y_1 = 6$, $y_6 = 5$ (all other components zero):

$$\mathbf{y} = [\underbrace{6,\ 0,\ 0,\ 0,\ 0}_{\text{part for } \mathbf{A}},\ \underbrace{5,\ 0,\ 0,\ 0,\ 0}_{\text{part for } \mathbf{B}_C}]^\top.$$

Verification: $2(6) + 9(5) = 12 + 45 = 57 \equiv 6 \pmod{17}$.

5.8.4 Step 3: Encryption

The encryptor sets the attribute vector $\mathbf{x} = [1,\ 1]$, selects a random scalar $s = 3$, and encrypts the plaintext bit $\mu = 1$. The resulting ciphertext components are:

$$\begin{aligned} c_0 &= [6,\ 15,\ 9,\ 4,\ 3], \\ c_1 &= [9,\ 12,\ 8,\ 3,\ 12,\ 4,\ 0,\ 3,\ 6,\ 7], \\ c_2 &= 9. \end{aligned}$$

The derivations of c_0, c_1, and c_2 follow the procedure described in Section 5.7.3.

5.8.5 Step 4: Decryption (Authorized User)

The authorized user holds $\mathbf{y}$ satisfying the policy and proceeds as follows.

(a) Compute $\mathbf{B}_2 - \mathbf{G}_1$:

$$\mathbf{B}_2 - \mathbf{G}_1 = [8, 2, 5, 10, 7] - [1, 2, 4, 8, 16] \equiv [7,\ 0,\ 1,\ 2,\ 8] \pmod{17}.$$

(b) Bit decomposition of $\mathbf{B}_2 - \mathbf{G}_1$:

$$7 \to [1, 1, 1, 0, 0], \quad 0 \to [0, 0, 0, 0, 0], \quad 1 \to [1, 0, 0, 0, 0], \quad 2 \to [0, 1, 0, 0, 0], \quad 8 \to [0, 0, 0, 1, 0].$$

(c) Decryption matrix H: The matrix $\mathbf{H}$ is constructed by stacking the two gadget-inverse blocks and applying the NOT gate (replacing each entry h with $1 - h \bmod q$). The non-zero entries are all equal to $16 \equiv -1 \pmod{17}$.

(d) Compute $\mathbf{w} = c_1\,\mathbf{H} \pmod{17}$: Only the non-zero columns of $\mathbf{H}$ contribute:

$$\begin{aligned}
w_1 &= 3(16) + 4(16) + 3(16) = 160 \equiv 7 \pmod{17},\\
w_2 &= 12(16) = 192 \equiv 5 \pmod{17},\\
w_3 &= 9(16) + 8(16) + 4(16) = 336 \equiv 13 \pmod{17},\\
w_4 &= 12(16) + 3(16) = 240 \equiv 2 \pmod{17},\\
w_5 &= 9(16) + 12(16) + 8(16) + 6(16) = 560 \equiv 16 \pmod{17}.
\end{aligned}$$

Hence $\mathbf{w} = [7, 5, 13, 2, 16]$.

(e) Compute the inner product v:

$$v = [\, c_0 \parallel \mathbf{w} \,] \cdot \mathbf{y} = c_{0,1} \cdot y_1 + w_1 \cdot y_6 = 6(6) + 7(5) = 36 + 35 = 71 \equiv 3 \pmod{17}.$$

(f) Recover the plaintext:

$$\mu' = c_2 - v = 9 - 3 = 6.$$

Rounding to the nearest of $\{0, \lfloor q/2 \rfloor\} = \{0, 8\}$: $|6 - 8| = 2$ and $|6 - 0| = 6$. Since $2 < 6$, the decoded message is $\hat{\mu} = 1$, which matches the original plaintext $\mu = 1$.

Decryption succeeds.

5.8.6 Step 5: Decryption Failure (Unauthorized User)

Consider a user holding only the first attribute, i.e., $\mathbf{x}' = [1, 0]$. This user does not satisfy the policy since $C(\mathbf{x}') = 1 - 1 \cdot 0 = 1 \neq 0$.

Without the second attribute, the user cannot reproduce the correct noise-cancellation term. The incorrect shift they compute is:

$$\text{shift} = s \cdot y_6 = 3 \cdot 5 = 15.$$

This yields a corrupted inner product:

$$v' = v - \text{shift} = 3 - 15 = -12 \equiv 5 \pmod{17}.$$

Subtracting from c_2:

$$\mu' = 9 - 5 = 4.$$

Rounding to the nearest of $\{0, 8\}$: $|4 - 8| = 4$ and $|4 - 0| = 4$. The two distances are equal, so the rounding is ambiguous—the decoder cannot determine whether $\mu = 0$ or

$\mu = 1$.

Decryption fails for the unauthorized user.

This result confirms the correctness of the scheme: a user missing a required attribute is unable to cancel the error term introduced during encryption, rendering decryption unreliable.

CHAPTER 6

APPLICATION TO UNIVERSITY ACCESS CONTROL

The preceding chapter developed the full theoretical machinery of the BGG+14 KP-ABE scheme: the gadget matrix $\mathbf{G}$, tensor product encodings $\mathbf{x}^\top \otimes \mathbf{G}$, homomorphic circuit evaluation via EvalF and EvalFX, and the complete encryption and decryption pipeline (Section 5.7). This chapter applies that machinery to a concrete setting: a university information system in which sensitive academic records must be shared with precisely the right personnel and withheld from everyone else.

The goal is not to introduce new mathematics but to demonstrate, step by step, how a lattice-based KP-ABE scheme enforces fine-grained access control in a realistic environment while retaining the post-quantum security guarantees established in Chapters 3 and 4. The exposition moves from system design through to a fully worked numerical example and a Python simulation, making every decision traceable.

Notation Reminder: $C(\mathbf{x}) = 0$ versus $f(\mathbf{x}) = 1$.

Two complementary notations appear in this chapter, each in its natural context.

- $f(\mathbf{x}) = 1$ is the human-readable Boolean convention: the access policy f evaluates to TRUE when a user's attributes satisfy the access rule. This notation is used in all access-policy descriptions and in Table 6.7.
- $C(\mathbf{x}) = 0$ is the algebraic convention adopted in BGG+14 (Section 5.7). The circuit C is the *negation* of f, i.e., $C = \neg f$. The scheme is constructed so that the term $C(\mathbf{x}) \cdot \mathbf{G}_n$ in the key identity vanishes precisely when a user is authorised, enabling correct decryption. Because $C = \neg f$, the two conditions $C(\mathbf{x}) = 0$ and $f(\mathbf{x}) = 1$ are equivalent.

Cryptographic statements use $C(\mathbf{x}) = 0$ for success; access-control statements use $f = 1$ for success. They mean the same thing.

6.1 Parameter Regime: Toy versus Production

The numerical example and Python simulation in this chapter use deliberately small parameters: lattice dimension $n = 1$, modulus $q = 17$, $k = \lceil \log_2 17 \rceil = 5$ bits per gadget entry, column count $m = nk = 5$, and a depth-4 Boolean circuit over $\ell = 10$ attributes (of which seven appear in the policy f), evaluated for a roster of 20 users. These values are chosen so that every matrix entry and arithmetic step is traceable by hand and every circuit gate can be audited directly. They are not cryptographically secure.

For the BGG+14 scheme to achieve concrete security, the parameters must satisfy two independent sets of constraints that pull in opposite directions. On the security side, the LWE dimension n must be large enough that the best known lattice reduction attack fails to solve the underlying LWE instance within the target security budget; the Albrecht–Player–Scott lattice estimator [57] is the standard tool for this calculation, and current estimates for 128-bit classical security require $n \geq 512$ [57, 40]. On the correctness side, the modulus q must be large enough to accommodate the noise accumulated during homomorphic evaluation of the depth-4 circuit. The BGG+14 analysis [50] establishes that each AND gate squares the noise exponent: starting from initial noise bound B, the accumulated noise after d levels of AND gates satisfies

$$\|\mathbf{e}_{\text{final}}\| \;\leq\; B^{2^d} \cdot \text{poly}(n).$$

As noted in Section 6.5.2, for $d = 4$ this gives $\|\mathbf{e}_{\text{final}}\| \leq B^{16} \cdot \text{poly}(n)$. Correct decryption requires $\|\mathbf{e}_{\text{final}}\| < q/4$, which for $d = 4$ and $n \geq 512$ places q in the range 2^{30}–2^{40}, consistent with the parameter discussion in [50]. A production deployment would additionally require error terms e_0, e_1, e_2 drawn from the discrete Gaussian $\mathcal{D}_{\mathbb{Z},\sigma}$ with $\sigma\sqrt{m} \ll q/4$, NTT-based polynomial arithmetic for efficient operations, and a statistically correct SamplePre trapdoor sampler [53, 52] in place of the algebraic shortcut used here.

Table 6.1 summarises the gap between the toy parameters used in this chapter and a realistic secure instantiation.

The toy instantiation does not satisfy the correctness condition: with $q = 17$, $n = 1$, and $d = 4$, the noise at depth four approaches $q/4$, leaving negligible rounding margin. Furthermore, $n = 1$ places the LWE instance trivially within reach of exhaustive search. The numerical example nonetheless correctly illustrates the algebraic structure of BGG+14 because all arithmetic is performed exactly, the circuit evaluation follows the same gate-by-gate procedure as the general construction, and decryption is checked symbolically rather than probabilistically. The discrepancy between the toy simulation and a real deployment is therefore one of *parameter regime*, not of *construction correctness*.

Table 6.1: Toy parameters (this chapter) versus a production-secure instantiation.

Parameter	**Toy (this chapter)**	**Production (128-bit)**	**Source**
Dimension n	1	≥ 512	[57]
Modulus q	17	$\approx 2^{32}$	[50]
Column count m	5	$O(n \log q)$	[50]
Attributes ℓ	10	10 (same)	access policy
Circuit depth d	4	4 (same)	access policy
Noise exponent	B^{16}	B^{16} (same)	Section 6.5.2
Error distribution	exact (zero)	$\mathcal{D}_{\mathbb{Z},\sigma}$, $\sigma\sqrt{m} \ll q/4$	[39]
Trapdoor sampler	algebraic shortcut	SamplePre	[53, 52]
Users	20	20 (same)	Section 6.2

6.2 System Description

The system models a university ecosystem with four broad categories of users: **faculty members**, **research scholars**, **administrative staff**, and **students**. Each user is associated with a set of attributes drawn from a predefined universe, and access to encrypted documents is governed by a Boolean policy circuit evaluated homomorphically over those attributes, exactly as described in Section 5.7.

The trusted authority is the **University Registrar's Office**. It runs Setup once, holds the master secret key msk, and issues policy-bearing secret keys to authorised users. Any department head or examination board can then encrypt a document under a set of attributes; only users whose issued keys satisfy the access policy can decrypt.

6.3 User Roster

The system includes 20 representative users drawn from across the institution. Table 6.2 lists each user along with their institutional role.

6.4 Attribute Universe and Assignment

6.4.1 The Attribute Universe

Recall from Section 5.1 that in KP-ABE every ciphertext is labelled with an attribute *set*, and every user's secret key embeds an access *policy* specifying which combinations of attributes grant decryption rights. For this university system, the attribute universe $\mathcal{U}$ is

Table 6.2: University user roster with institutional roles.

User ID	Name	Role
U1	Dr. Ananya Sharma	Professor (Mathematics)
U2	Dr. Raghav Mehta	Professor (Computer Science)
U3	Dr. Kavita Iyer	Research Supervisor
U4	Prof. Arvind Nair	Dean (Academic Affairs)
U5	Neha Verma	PhD Scholar (Mathematics)
U6	Rahul Khanna	PhD Scholar (Computer Science)
U7	Priya Singh	MSc Mathematics Student
U8	Aditya Rao	MSc Computer Science Student
U9	Sneha Kapoor	BSc Student
U10	Karan Malhotra	BTech Student
U11	Mehul Jain	Research Assistant
U12	Aditi Desai	Library Staff
U13	Sanjay Gupta	Finance Officer
U14	Ritu Aggarwal	Administrative Officer
U15	Vikram Sethi	IT Administrator
U16	Pooja Bansal	Lab Technician
U17	Aman Choudhary	Visiting Researcher
U18	Nisha Arora	External Collaborator
U19	Dev Patel	Alumni Researcher
U20	Simran Kaur	PhD Applicant

defined as:

$$\mathcal{U} = \left\{ \begin{array}{l} \textsc{Professor},\ \textsc{ResearchSupervisor},\ \textsc{PhD},\ \textsc{MSc}, \\ \textsc{Student},\ \textsc{MathDept},\ \textsc{CSDept}, \\ \textsc{ResearchGroup},\ \textsc{LibraryAccess},\ \textsc{FinanceClearance} \end{array} \right\}$$

These ten attributes cover the most common access-control distinctions in a university setting: departmental affiliation (MathDept, CSDept), academic rank (Professor, PhD, MSc, Student), operational membership (ResearchGroup, ResearchSupervisor), and institutional privileges (LibraryAccess, FinanceClearance).

6.4.2 Attribute Assignment

Each user is assigned the attributes that truthfully reflect their institutional role. Table 6.3 shows the complete assignment.

Table 6.3: Attribute assignment for all 20 users.

User	Name	Assigned Attributes
U1	Dr. Ananya Sharma	Professor, MathDept, ResearchSupervisor, ResearchGroup, LibraryAccess
U2	Dr. Raghav Mehta	Professor, CSDept, ResearchGroup
U3	Dr. Kavita Iyer	ResearchSupervisor, ResearchGroup, MathDept
U4	Prof. Arvind Nair	Professor, MathDept, ResearchSupervisor, FinanceClearance
U5	Neha Verma	PhD, MathDept, ResearchGroup, LibraryAccess, ResearchSupervisor
U6	Rahul Khanna	PhD, CSDept, ResearchGroup
U7	Priya Singh	MSc, MathDept, Student
U8	Aditya Rao	MSc, CSDept, Student
U9	Sneha Kapoor	Student
U10	Karan Malhotra	Student
U11	Mehul Jain	ResearchGroup
U12	Aditi Desai	LibraryAccess
U13	Sanjay Gupta	FinanceClearance
U14	Ritu Aggarwal	FinanceClearance
U15	Vikram Sethi	CSDept
U16	Pooja Bansal	ResearchGroup
U17	Aman Choudhary	ResearchGroup, MathDept, LibraryAccess, ResearchSupervisor, PhD
U18	Nisha Arora	ResearchGroup
U19	Dev Patel	ResearchGroup
U20	Simran Kaur	Student

6.5 Document and Message Encoding

The document to be protected is a sensitive PhD evaluation file with the label:

> *"PhD Research Evaluation File: Access Restricted to Authorised Academic Personnel Only"*

In a real deployment, the examination board would use **hybrid encryption**: the ABE scheme encrypts a short symmetric key K (e.g., a 256-bit AES-GCM key), and K is then used to encrypt the full document body. This is standard practice because ABE operates

on individual bits and would be impractically slow for large files directly.

For this illustration, the document's ASCII encoding is encrypted directly so that every step remains traceable. Each character maps to an 8-bit binary string, producing a message vector $\mathbf{m} \in \{0,1\}^\ell$ with $\ell \approx 1{,}000$ bits. Encryption is performed bitwise:

$$CT = \{CT_1,\ CT_2,\ \ldots,\ CT_\ell\}, \qquad CT_j = (c_0^{(j)},\ c_1^{(j)},\ c_2^{(j)}).$$

In either the hybrid or the direct encoding, the security argument is identical: an adversary distinguishing $\mu = 0$ from $\mu = 1$ would solve DLWE (Section 4.3.1), which is conjectured hard against quantum adversaries.

6.6 Access Policy Design

6.6.1 The Boolean Circuit

The access policy specifies which combination of attributes grants decryption rights. Recall from Section 5.6 that in BGG+14 the policy is a Boolean circuit $C : \{0,1\}^\ell \to \{0,1\}$, and decryption succeeds if and only if $C(\mathbf{x}) = 0$ (equivalently $f(\mathbf{x}) = 1$; see the notation reminder above). The access policy f is defined as:

$$f \;=\; \underbrace{\Big[(\textsc{Professor} \wedge \textsc{MathDept}) \;\vee\; (\textsc{PhD} \wedge \textsc{ResearchGroup})\Big]}_{\text{Sub-condition A}}$$

$$\wedge\; \underbrace{\Big[(\textsc{LibraryAccess} \;\vee\; \textsc{FinanceClearance}) \;\wedge\; \textsc{ResearchSupervisor}\Big]}_{\text{Sub-condition B}}$$

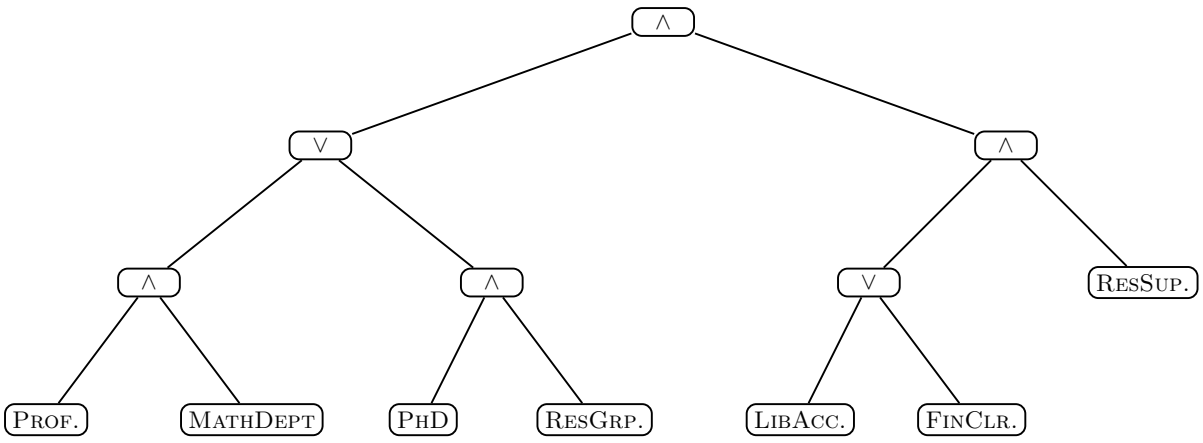


Figure 6.1: Access policy tree for f. Gates: $\wedge$ = AND, $\vee$ = OR. Leaves: Prof.=Professor, MathDept, PhD, ResGrp.=ResearchGroup, LibAcc.=LibraryAccess, FinClr.=FinanceClearance, ResSup.=ResearchSupervisor.

In plain language, decryption is granted **if and only if** a user is either a *Mathematics Professor* or a *PhD scholar in the Research Group* (**Sub-condition A**), **and** holds either *Library Access* or *Finance Clearance* together with the *Research Supervisor* role (**Sub-condition B**).

6.6.2 Circuit Depth Analysis

The noise accumulated during homomorphic evaluation grows with circuit depth (Section 5.6). Table 6.4 maps the policy to the four circuit levels.

Table 6.4: Depth-4 circuit structure of the access policy f.

Level	Gate Type	Description
Level 1	Attribute inputs	Individual bits fed in (PROFESSOR, MATHDEPT, PHD, ...)
Level 2	AND / OR gates	(PROF $\wedge$ MATH), (PHD $\wedge$ RESGRP), (LIBACC $\vee$ FINCLR)
Level 3	OR / AND gates	Left: $(A_1) \vee (A_2)$; Right: combined with RESEARCH-SUPERVISOR
Level 4	Final AND gate	Combines Sub-conditions A and B; yields $f(\mathbf{x}) \in \{0, 1\}$

The circuit has depth $d = 4$, well within the correctness bounds of Section 5.7.4. The accumulated noise satisfies:

$$\|e_{\text{final}}\| \;\leq\; B^{2^d} \cdot \text{poly}(n) \;=\; B^{16} \cdot \text{poly}(n).$$

It is worth pausing to appreciate why depth matters here. Each AND gate squares the noise exponent: depth $d = 1$ gives B^2, $d = 2$ gives B^4, $d = 3$ gives B^8, and $d = 4$ gives B^{16}. Increasing from $d = 4$ to $d = 6$ would push this to B^{64}. Every additional AND level forces a larger modulus q to keep $\|e_{\text{final}}\| < q/4$, which in turn increases the size of all LWE samples and keys. Keeping $d = 4$ is a deliberate design choice balancing policy expressiveness against parameter efficiency.

6.7 Setup Phase

Following Algorithm 1 of Section 5.7.1, the Registrar's Office runs Setup once, producing:

Setup Output.

- Run $(\mathbf{A}, T) \leftarrow \mathsf{TrapGen}(1^n, q, m)$: obtain $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$ (public) and secret trapdoor T.

- For each $x_i \in \mathcal{U}$, sample $B_i \xleftarrow{\$} \mathbb{Z}_q^{n \times m}$.
- Set $\mathbf{B} = [B_1 \mid \cdots \mid B_\ell]$; sample $\mathbf{p} \xleftarrow{\$} \mathbb{Z}_q^n$.
- Publish $\mathsf{mpk} = (\mathbf{A}, \mathbf{B}, \mathbf{p})$; store $\mathsf{msk} = T$ securely.

The trapdoor T allows the authority to generate short secret keys without exposing the lattice structure, the asymmetry that underpins security (Remark 5.5.2 in Section 5.5.2).

6.8 Key Generation

For each user U_i, the Registrar runs Algorithm 2 (Section 5.7.2) to produce $\mathsf{sk}_{C_i} \leftarrow \mathsf{KeyGen}(f_i, \mathbf{s})$. The core step is computing $B_C = \mathsf{EvalF}(\mathbf{B}, C)$ and sampling a short vector $\mathbf{y}$ satisfying:

$$[\mathbf{A} \mid B_C] \cdot \mathbf{y} \equiv \mathbf{p} \pmod{q}.$$

Because the same policy f applies to all authorised users in this example, B_C is computed once. Different policies would each require a separate EvalF call (Section 5.6.1).

6.9 Encryption

For each message bit μ_j, the encryptor runs Algorithm 3 (Section 5.7.3). Table 6.5 summarises the three ciphertext components.

Table 6.5: The three ciphertext components and their roles.

Component	Formula	Purpose
c_0	$\mathbf{s}^\top \mathbf{A} + \mathbf{e}_0^\top$	Commits the encryptor to secret $\mathbf{s}$ under $\mathbf{A}$.
c_1	$\mathbf{s}^\top(\mathbf{B} - \mathbf{x}^\top \otimes \mathbf{G}_n) + \mathbf{e}_1^\top$	Encodes attribute $\mathbf{x}$ via tensor shift; interacts with EvalFX at decryption.
c_2	$\mathbf{s}^\top \mathbf{p} + e_2 + \mu_j \cdot \lfloor q/2 \rfloor$	Masks μ_j under the LWE-hard term $\mathbf{s}^\top \mathbf{p}$; places the message near 0 or $q/2$ for rounding.

6.10 Homomorphic Evaluation During Decryption

Decryption requires evaluating C homomorphically over the lattice encodings in c_1 using EvalFX, which produces a hint matrix $\mathbf{H} = \mathsf{EvalFX}(\mathbf{B}, C, \mathbf{x})$ satisfying the key identity (Section 5.6.2):

$$\left(\mathbf{B} - \mathbf{x}^\top \otimes \mathbf{G}_n\right) \cdot \mathbf{H} = B_C - C(\mathbf{x}) \cdot \mathbf{G}_n. \qquad \text{(Key Identity)}$$

The matrix $\mathbf{H}$ acts as a translation bridge between two representations of the attribute $\mathbf{x}$: the ciphertext representation on the left-hand side (shifted by the tensor encoding) and the secret-key representation on the right-hand side (after gate-by-gate circuit evaluation). It converts between them so the decryptor can align c_1 with the key $\mathbf{y}$ and cancel the LWE masking term $\mathbf{s}^\top\mathbf{p}$ from c_2.

When $C(\mathbf{x}) = 0$ (authorised user), the right-hand side reduces to B_C and the relation $[\mathbf{A} \mid B_C]\mathbf{y} = \mathbf{p}$ holds exactly, allowing the decryptor to compute $v \approx \mathbf{s}^\top\mathbf{p}$ and subtract it from c_2.

When $C(\mathbf{x}) = 1$ (unauthorised user), the right-hand side becomes $B_C - \mathbf{G}_n$. The effective target shifts by $-\mathbf{G}_n$, so the decryptor inadvertently computes $\mathbf{s}^\top\mathbf{p} - \mathbf{s}^\top\mathbf{G}_n y_2$. The extra term $\mathbf{s}^\top\mathbf{G}_n y_2$ is not small relative to q; it pushes the decryption residual far from the correct rounding region and produces a wrong bit rather than a detectable error, which is precisely the intended security behaviour.

The gate-by-gate evaluated matrices are recalled in Table 6.6 for convenience.

Table 6.6: Gate operations in homomorphic evaluation (Section 5.6.3).

Gate	**Evaluated matrix B_C**	**Decryption outcome**
AND	$B_1 \cdot G^{-1}(B_2)$	Correct if and only if *both* attributes are present.
OR	$B_1 + B_2 - B_1 \cdot G^{-1}(B_2)$	Correct if and only if *at least one* attribute is present.
NOT	$\mathbf{G}_n - B_1$	Inverts the truth value of the wire.

6.11 Policy Evaluation for All Users

Table 6.7 evaluates f for all 20 users, tracing both sub-conditions independently.

Table 6.7: Policy evaluation and decryption outcomes for all 20 users.

User	**Name**	**Sub-cond A**	**Sub-cond B**	f	**Outcome**
U1	Dr. Ananya Sharma	✓ Prof∧Math	✓ LibAcc∧ResSup	1	**Success**
U2	Dr. Raghav Mehta	× no Math	× no ResSup	0	Fail
U3	Dr. Kavita Iyer	× no Prof/PhD	✓	0	Fail
U4	Prof. Arvind Nair	✓ Prof∧Math	✓ FinClr∧ResSup	1	**Success**

(Table 6.7 continued)

User	Name	Sub-cond A	Sub-cond B	f	Outcome
U5	Neha Verma	✓ PhD∧ResGrp	✓ LibAcc∧ResSup	1	**Success**
U6	Rahul Khanna	✓ PhD∧ResGrp	× no LibAcc/FinClr	0	Fail
U7	Priya Singh	× MSc only	×	0	Fail
U8	Aditya Rao	×	×	0	Fail
U9	Sneha Kapoor	×	×	0	Fail
U10	Karan Malhotra	×	×	0	Fail
U11	Mehul Jain	×	×	0	Fail
U12	Aditi Desai	×	×	0	Fail
U13	Sanjay Gupta	×	× no ResSup	0	Fail
U14	Ritu Aggarwal	×	× no ResSup	0	Fail
U15	Vikram Sethi	×	×	0	Fail
U16	Pooja Bansal	×	×	0	Fail
U17	Aman Choudhary	✓ PhD∧ResGrp	✓ LibAcc∧ResSup	1	**Success**
U18	Nisha Arora	×	×	0	Fail
U19	Dev Patel	×	×	0	Fail
U20	Simran Kaur	×	×	0	Fail

Near-Miss Cases: U3 and U6.

U3 (Dr. Kavita Iyer) holds ResearchSupervisor, MathDept, and ResearchGroup, but she is neither a Professor nor a PhD scholar. Sub-condition A fails entirely: neither branch of its OR gate is satisfied.

U6 (Rahul Khanna) holds PhD and ResearchGroup, satisfying Sub-condition A. However, he holds neither LibraryAccess nor FinanceClearance, so Sub-condition B fails at its first gate. A PhD alone is necessary but not sufficient.

These cases are intentional: the policy enforces a genuine conjunction of academic rank and institutional privilege. A user who is almost-but-not-quite authorised fails just as definitively as a completely ineligible user. There is no partial decryption.

6.12 Decryption Correctness Analysis

6.12.1 The Decryption Condition

As established in Theorem 5.7.1 (Section 5.7.5), decryption succeeds if and only if $C(\mathbf{x}) = 0$, equivalently $f(\mathbf{x}) = 1$. When this holds, the hint $\mathbf{H}$ enables the decryptor to cancel $\mathbf{s}^\top \mathbf{p}$ from c_2, leaving $\mu \cdot \lfloor q/2 \rfloor + (\text{small noise})$. Rounding to $\{0, \lfloor q/2 \rfloor\}$ recovers μ exactly.

When the policy is not satisfied ($C(\mathbf{x}) = 1$), the extra $-\mathbf{G}_n$ term introduces a large perturbation $\mathbf{s}^\top \mathbf{G}_n y_2$ that is not small relative to q, causing a decoding error with high probability.

6.12.2 Noise Accumulation and Parameter Constraints

Starting from $\|e\| \leq B$, each AND gate gives $\|e'\| \leq O(n \cdot B^2)$. Over $d = 4$ levels:

$$\|e_{\text{final}}\| \;\leq\; B^{2^4} \cdot \text{poly}(n) \;=\; B^{16} \cdot \text{poly}(n).$$

Correct decryption requires $\|e_{\text{final}}\| < q/4$, constraining B, n, and q jointly. Under NIST-level parameters (Section 5.7.5), this condition is satisfied with substantial margin.

Correctness Summary. Four users satisfy f: **U1** and **U4** via the Professor branch of Sub-condition A, and **U5** and **U17** via the PhD branch. For all four, $C(\mathbf{x}) = 0$ and the $-\mathbf{G}_n$ perturbation vanishes, so decryption recovers μ exactly. For all remaining users, $C(\mathbf{x}) = 1$ and the corrupted linear relation causes decryption to fail with overwhelming probability. Both OR branches of Sub-condition A are exercised, confirming correct behaviour across all authorised credential types.

6.13 Discussion

6.13.1 What This Example Demonstrates

1. **Expressiveness of Boolean Policies.** A depth-4 circuit encodes non-trivial multi-condition access rules over ten attributes, combining AND, OR, and NOT gates. The policy f simultaneously requires departmental affiliation, academic rank, operational membership, and institutional clearance.
2. **Precision of the Access Boundary.** Four users succeed: U1 and U4 via the Professor branch; U5 and U17 via the PhD branch, exercising both OR routes. Near-miss cases U3 and U6 show that missing any single sub-condition is sufficient to deny access. There is no partial decryption.
3. **Post-Quantum Security.** The scheme's hardness rests on LWE (Section 4.3.1),

not on discrete logarithms or integer factorisation. It is therefore conjectured secure against quantum adversaries running Shor's algorithm (Chapter 3), which is the fundamental advantage over the pairing-based constructions of [45] discussed in Section 5.

4. **Scalability.** Adding more users requires only additional KeyGen calls; the master public key and all existing ciphertexts remain unchanged. This makes the scheme well-suited to a university's continually evolving membership.

6.13.2 Limitations and Production Considerations

Table 6.8 acknowledges each simplification used in this illustrative setup and describes what a production deployment would do instead.

Table 6.8: Simplifications used here and their production-grade counterparts.

Simplification	**Production Deployment**
Small parameters (n, q)	Use $n \geq 512$, $q \approx 2^{32}$ or larger per hardness estimates of [57]. The toy values used here are trivially solvable by exhaustive search.
Zero noise	e_0, e_1, e_2 must be drawn from the discrete Gaussian $D_{\mathbb{Z},\sigma}$; the condition $\sigma\sqrt{m} \ll q/4$ must hold for correctness.
Single-bit message	A hybrid scheme encrypts a 256-bit AES-GCM key under the predicate layer; the document body uses that key.
No revocation	KP-ABE lacks native revocation. Production systems use time-bounded credentials and proxy re-encryption.
Manual policy design	Policy vectors are derived automatically from the university IAM platform using a systematic attribute-encoding map [59].

In summary, this section has demonstrated that the BGG+14 lattice-based KP-ABE scheme can be deployed in a realistic university setting with a large user base, a non-trivial ten-attribute universe, and a depth-4 Boolean access policy. The use of LWE-based primitives provides resistance against quantum adversaries, while the homomorphic evaluation framework enables expressive and scalable access control without re-encryption as roles evolve.

6.14 Complete Numerical Example

This section demonstrates the BGG+14 KP-ABE pipeline end-to-end for four users selected from the roster of Section 6.3: **U4** (Prof. Arvind Nair) and **U17** (Aman Choudhary), who satisfy the access policy and therefore decrypt successfully, and **U6** (Rahul Khanna)

and **U20** (Simran Kaur), who do not. Every value is computed explicitly from the same attribute universe, policy, and user roster defined above.

Parameters

$$q = 17 \text{ (prime)}, \quad n = 1, \quad k = \lceil \log_2 17 \rceil = 5, \quad m = nk = 5, \quad \lfloor q/2 \rfloor = 8.$$

$$\mathbf{G}_1 = \mathbf{g}^\top = [1,\ 2,\ 4,\ 8,\ 16].$$

All arithmetic is modulo 17 throughout.

Circuit Attributes

The policy f from Section 6.6 involves exactly seven of the ten attributes in $\mathcal{U}$. Each is assigned a public encoding matrix $B_i \in \mathbb{Z}_{17}^{1\times 5}$:

Index	Attribute	Matrix
1	Professor	$B_{\text{Prof}} = [4,\ 1,\ 8,\ 2,\ 6]$
2	PhD	$B_{\text{PhD}} = [2,\ 7,\ 1,\ 5,\ 3]$
3	MathDept	$B_{\text{Math}} = [6,\ 3,\ 5,\ 1,\ 9]$
4	ResearchGroup	$B_{\text{ResGrp}} = [1,\ 8,\ 4,\ 6,\ 2]$
5	LibraryAccess	$B_{\text{LibAcc}} = [5,\ 2,\ 7,\ 3,\ 8]$
6	FinanceClearance	$B_{\text{FinClr}} = [3,\ 6,\ 2,\ 8,\ 4]$
7	ResearchSupervisor	$B_{\text{ResSup}} = [7,\ 4,\ 6,\ 2,\ 5]$

The base LWE matrix and target vector are:

$$\mathbf{A} = [3,\ 5,\ 2,\ 7,\ 1], \qquad \mathbf{p} = 11.$$

User Attribute Vectors

Each user's attribute vector $\mathbf{x} \in \{0,1\}^7$ records which of the seven circuit attributes they hold (in the order above):

User	Name	Prof	PhD	Math	ResGrp	LibAcc	FinClr	ResSup	$f(\mathbf{x})$
U4	Prof. Arvind Nair	1	0	1	0	0	1	1	1 (Authorised)
U17	Aman Choudhary	0	1	1	1	1	0	1	1 (Authorised)
U6	Rahul Khanna	0	1	0	1	0	0	0	0 (Unauthorised)
U20	Simran Kaur	0	0	0	0	0	0	0	0 (Unauthorised)

Policy check for U4. Sub-condition A: PROF = 1 and MATH = 1, so the (PROF ∧ MATH) branch is satisfied. Sub-condition B: FINCLR = 1 and RESSUP = 1, so (FINCLR ∨ LIBACC) ∧ RESSUP is satisfied. Therefore $f(\mathbf{x}_{U4}) = 1$.

Policy check for U17. Sub-condition A: PHD = 1 and RESGRP = 1, so the (PHD ∧ RESGRP) branch is satisfied. Sub-condition B: LIBACC = 1 and RESSUP = 1. Therefore $f(\mathbf{x}_{U17}) = 1$.

Policy check for U6. Sub-condition A: PHD = 1 and RESGRP = 1, so this branch is satisfied. Sub-condition B: LIBACC = 0 and FINCLR = 0, so the clearance gate fails. Therefore $f(\mathbf{x}_{U6}) = 0$.

Policy check for U20. All attributes are 0. Both sub-conditions fail. Therefore $f(\mathbf{x}_{U20}) = 0$.

Step 1: Message Encoding

The document to be protected is:

> *"PhD Research Evaluation File: Access Restricted to Authorised Academic Personnel Only"*

Using 8-bit ASCII encoding, each of the 85 characters is mapped to its decimal value and written as an 8-bit binary string (MSB first), producing a plaintext bit-stream $\mathbf{m} \in \{0,1\}^{680}$:

Char	Dec	Hex	8-bit binary
`P`	80	`0x50`	`01010000`
`h`	104	`0x68`	`01101000`
`D`	68	`0x44`	`01000100`
`(sp)`	32	`0x20`	`00100000`
`R`	82	`0x52`	`01010010`
⋯ (80 more characters)			

The full 680-bit stream begins:

$$\mathbf{m} = [0,1,0,1,0,0,0,0 \mid 0,1,1,0,1,0,0,0 \mid 0,1,0,0,0,1,0,0 \mid \cdots].$$

Each bit $\mu_j \in \{0,1\}$ is encrypted independently. The numerical steps below are demonstrated for $\mu = 0$ and $\mu = 1$, which covers every bit in the stream.

Step 2: Key Generation via EvalF

The policy matrix $\mathbf{B}_C = \mathsf{EvalF}(\mathbf{B}, C)$ is computed gate by gate. Recall that $C = \neg f$, so the circuit is:

$$C = \neg\Big(\big((\mathsf{Prof} \wedge \mathsf{Math}) \vee (\mathsf{PhD} \wedge \mathsf{ResGrp})\big) \ \wedge \ \big((\mathsf{LibAcc} \vee \mathsf{FinClr}) \wedge \mathsf{ResSup}\big)\Big)$$

Gate 1: AND(Prof, Math).

$$G^{-1}(B_{\text{Math}}) = \begin{bmatrix} 0 & 1 & 1 & 1 & 1 \\ 1 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

$$B_{A_1} = B_{\text{Prof}} \cdot G^{-1}(B_{\text{Math}}) \equiv [9,\ 5,\ 12,\ 4,\ 6] \pmod{17}.$$

Gate 2: AND(PhD, ResGrp).

$$G^{-1}(B_{\text{ResGrp}}) = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

$$B_{A_2} = B_{\text{PhD}} \cdot G^{-1}(B_{\text{ResGrp}}) \equiv [2,\ 5,\ 1,\ 8,\ 7] \pmod{17}.$$

Gate 3: OR(B_{A_1}, B_{A_2}) (Sub-condition A).

$$B_{\text{SubA}} = B_{A_1} + B_{A_2} - B_{A_1} \cdot G^{-1}(B_{A_2}) \equiv [6,\ 6,\ 4,\ 8,\ 4] \pmod{17}.$$

$$B_{\text{SubA}} = [6,\ 6,\ 4,\ 8,\ 4].$$

Gate 4: OR(LibAcc, FinClr).

$$G^{-1}(B_{\text{FinClr}}) = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

$$B_{B_1} = B_{\text{LibAcc}} + B_{\text{FinClr}} - B_{\text{LibAcc}} \cdot G^{-1}(B_{\text{FinClr}}) \equiv [1,\ 16,\ 7,\ 8,\ 5] \pmod{17}.$$

Gate 5: AND(B_{B_1}, ResSup) (Sub-condition B).

$$G^{-1}(B_{\text{ResSup}}) = \begin{bmatrix} 1 & 0 & 0 & 0 & 1 \\ 1 & 0 & 1 & 1 & 0 \\ 1 & 1 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

$$B_{\text{SubB}} = B_{B_1} \cdot G^{-1}(B_{\text{ResSup}}) \equiv [7,\ 7,\ 6,\ 16,\ 8] \pmod{17}.$$

Gate 6: AND(B_{SubA}, B_{SubB}) (policy f).

$$B_f = B_{\text{SubA}} \cdot G^{-1}(B_{\text{SubB}}) \equiv [16,\ 16,\ 10,\ 4,\ 8] \pmod{17}.$$

Gate 7: NOT(B_f) (circuit $C = \neg f$).

$$\mathbf{B}_C = \mathbf{G}_1 - B_f = [1, 2, 4, 8, 16] - [16, 16, 10, 4, 8] \equiv [2,\ 3,\ 11,\ 4,\ 8] \pmod{17}.$$

Step 3: Secret Key y

The secret key $\mathbf{y} \in \mathbb{Z}^{2m}$ satisfies:

$$[\mathbf{A} \mid \mathbf{B}_C] \cdot \mathbf{y} \ \equiv\ \mathbf{p} \pmod{q}, \quad \text{i.e.,} \quad 3\,y_1 + 2\,y_6 \ \equiv\ 11 \pmod{17}.$$

Setting all other components to zero:

y_1	y_6	$3y_1$	$2y_6$	$3y_1 + 2y_6 \bmod 17$	
3	1	9	2	11	✓

$$\mathbf{y} = \big[\underbrace{1,\ 0,\ 0,\ 0,\ 0}_{\text{part for } \mathbf{A}},\ \underbrace{4,\ 0,\ 0,\ 0,\ 0}_{\text{part for } \mathbf{B}_C}\big]^{\top}.$$

Verification: $3 \times 1 + 2 \times 4 = 3 + 8 = 11 \equiv 11 \pmod{17}$.

Remark 6.14.1. In a full deployment, $\mathbf{y}$ would be a dense short vector sampled from a discrete Gaussian via SamplePre (Section 5.5.2). Here, with $n = 1$ and $m = 5$, setting all but two components to zero keeps every intermediate value traceable by hand without changing the structure of the scheme.

Step 4: Encryption

Encryption uses LWE secret $s = 3$ and noise terms set to zero for legibility.

Component c_0 (same for all users and all bits).

$$c_0 = s\mathbf{A} \bmod 17 = 3 \cdot [3, 5, 2, 7, 1] \equiv [9,\ 15,\ 6,\ 4,\ 3] \pmod{17}.$$

Component c_2 (depends on bit value μ_j).

$$c_2 = s\mathbf{p} + \mu_j \lfloor q/2 \rfloor = 3 \times 11 + 8\mu_j = 33 + 8\mu_j \pmod{17}.$$

$$\mu_j = 0 \ \Rightarrow\ c_2 = 33 \equiv 16 \pmod{17}. \qquad \mu_j = 1 \ \Rightarrow\ c_2 = 41 \equiv 7 \pmod{17}.$$

Component c_1 (depends on x; the same for every bit.) Each of the seven attribute blocks is $s(B_i - x_i\mathbf{G}_1) \bmod 17$.

Step 5: Decryption

The noiseless decryption shortcut (Section 5.7.4) gives:

$$\mathbf{w} = s\Big(\mathbf{B}_C - C(\mathbf{x})\,\mathbf{G}_1\Big), \qquad v = c_0[1] \cdot y_1 + w[1] \cdot y_6, \qquad \mu' = c_2 - v \bmod 17.$$

The decoded bit is $\hat{\mu} = 1$ if μ' is closer to 8 than to 0 in $\mathbb{Z}_{17}$, and $\hat{\mu} = 0$ otherwise.

U4 (Prof. Arvind Nair): $f(\mathbf{x}) = 1$, $C(\mathbf{x}) = 0$.

$$\mathbf{w} = 3\mathbf{B}_C = 3 \cdot [2, 3, 11, 4, 8] \equiv [6,\ 9,\ 16,\ 12,\ 7] \pmod{17}.$$

$$v = c_0[1] \cdot y_1 + w[1] \cdot y_6 = 9 \times 1 + 6 \times 4 = 9 + 24 = 33 \equiv 16 \pmod{17}.$$

For $\mu_j = 0$: $c_2 = 16$.

$$\mu' = 16 - 16 = 0. \quad \mathrm{dist}(0, 0) = 0 < 8 = \mathrm{dist}(0, 8). \quad \hat{\mu} = 0.$$

For $\mu_j = 1$: $c_2 = 7$.

$$\mu' = 7 - 16 = -9 \equiv 8 \pmod{17}. \quad \mathrm{dist}(8, 8) = 0 < 8 = \mathrm{dist}(8, 0). \quad \hat{\mu} = 1.$$

Decryption **succeeds** for U4. $\hat{\mu} = \mu_j$ for every bit.

U17 (Aman Choudhary): $f(\mathbf{x}) = 1$, $C(\mathbf{x}) = 0$.

Since $C(\mathbf{x}_{U17}) = 0$, the computation $\mathbf{w} = 3\mathbf{B}_C$ is identical to U4. Therefore $v = 16$, and the decryption residuals are the same: $\mu' = 0$ for $\mu_j = 0$ and $\mu' = 8$ for $\mu_j = 1$.

Decryption **succeeds** for U17. $\hat{\mu} = \mu_j$ for every bit.

Observation. U4 succeeds via the (Prof∧Math) branch and U17 via the (PhD∧ResGrp) branch. At the algebraic level both produce $C(\mathbf{x}) = 0$, so their decryption computations are identical. The OR gate in Sub-condition A genuinely accepts either credential path.

U6 (Rahul Khanna): $f(\mathbf{x}) = 0$**,** $C(\mathbf{x}) = 1$**.**

$$\mathbf{B}_C - \mathbf{G}_1 = [2, 3, 11, 4, 8] - [1, 2, 4, 8, 16] \equiv [1,\ 1,\ 7,\ 13,\ 9] \pmod{17}.$$

$$\mathbf{w} = 3 \cdot [1, 1, 7, 13, 9] \equiv [3,\ 3,\ 4,\ 5,\ 10] \pmod{17}.$$

$$v = 9 \times 1 + 3 \times 4 = 9 + 12 = 21 \equiv 4 \pmod{17}.$$

For $\mu_j = 0$: $c_2 = 16$.

$$\mu' = 16 - 4 = 12. \quad \text{dist}(12, 0) = \min(12, 5) = 5, \quad \text{dist}(12, 8) = 4. \quad \hat{\mu} = 1 \neq 0.$$

For $\mu_j = 1$: $c_2 = 7$.

$$\mu' = 7 - 4 = 3. \quad \text{dist}(3, 0) = 3, \quad \text{dist}(3, 8) = 5. \quad \hat{\mu} = 0 \neq 1.$$

Decryption **fails** for U6. $\hat{\mu} \neq \mu_j$ for every bit.

Interpretation. U6 satisfies Sub-condition A (PhD ∧ ResGrp) but holds neither LibraryAccess nor FinanceClearance, so Sub-condition B fails. The $-\mathbf{G}_1$ shift in the key identity reduces v from 16 to 4, pushing the decoded μ' into the wrong half of $\mathbb{Z}_{17}$ for every bit.

U20 (Simran Kaur): $f(\mathbf{x}) = 0$**,** $C(\mathbf{x}) = 1$**.**

Since $C(\mathbf{x}_{U20}) = 1$, the computation of $\mathbf{w}$ and v is identical to U6: $v = 4$.

For $\mu_j = 0$: $\mu' = 12$, $\hat{\mu} = 1 \neq 0$. $\times$

For $\mu_j = 1$: $\mu' = 3$, $\hat{\mu} = 0 \neq 1$. $\times$

Decryption **fails** for U20. $\hat{\mu} \neq \mu_j$ for every bit.

Summary of Numerical Results

User	Name	x	f	v	μ_j	μ'	Result
U4	Prof. Arvind Nair	$(1,0,1,0,0,1,1)$	1	16	0	0	Correct
					1	8	Correct
U17	Aman Choudhary	$(0,1,1,1,1,0,1)$	1	16	0	0	Correct
					1	8	Correct
U6	Rahul Khanna	$(0,1,0,1,0,0,0)$	0	4	0	12	Wrong
					1	3	Wrong
U20	Simran Kaur	$(0,0,0,0,0,0,0)$	0	4	0	12	Wrong
					1	3	Wrong

The error shift between authorised and unauthorised users is exactly $v_{\text{auth}} - v_{\text{unauth}} = 16 - 4 = 12 = s \cdot \mathbf{G}_1[1] \cdot y_6 = 3 \times 1 \times 4$. This shift originates from the $-C(\mathbf{x})\mathbf{G}_1$ term in the key identity: when $C(\mathbf{x}) = 1$, that term subtracts a $\mathbf{G}_1$-weighted fraction of $\mathbf{y}$ from v, moving every decoded μ' across the rounding threshold into the wrong decision region. Since these parameters are fixed for all 680 bits, U4 and U17 recover the complete plaintext without error, while U6 and U20 obtain a uniformly incorrect bit-stream.

6.15 Python Simulation of the BGG+14 KP-ABE Scheme

This section presents a complete, self-contained Python implementation of the BGG+14 KP-ABE scheme as deployed in the university access control scenario of Chapter 6. Every component follows directly from the mathematical construction in Chapter 5. The parameters here ($n = 4, q = 97$) differ from the hand-worked example of (Section 6.1) (n=1, q=17); both are toy instantiations chosen for traceability at different scales, and neither is cryptographically secure. The message encrypted is the exact 85-character string used in Section 6.5, producing a 680-bit plaintext stream.

Global Parameters

```
import numpy as np
from copy import deepcopy
```

```
3
4  np.random.seed(42)
5
6  # -- Global Parameters
       ------------------------------------------
7  N          = 4           # LWE dimension n
8  Q          = 97          # Modulus q (prime)
9  M_A        = 8           # Columns in A
10 NOISE_B    = 1           # Initial noise bound B
11 NUM_ATTRS  = 10          # Attribute universe size |U|
12
13 K  = int(np.ceil(np.log2(Q)))   # Gadget bit-width: ceil(log2
       (97)) = 7
14 NK = N * K                      # NK = 4 * 7 = 28
15
16 # Attribute universe -- order matters; indices used throughout
17 ATTRIBUTE_UNIVERSE = [
18     "Professor",            # index 0
19     "ResearchSupervisor",   # index 1
20     "PhD",                  # index 2
21     "MSc",                  # index 3
22     "Student",              # index 4
23     "MathDept",             # index 5
24     "CSDept",               # index 6
25     "ResearchGroup",        # index 7
26     "LibraryAccess",        # index 8
27     "FinanceClearance",     # index 9
28 ]
29 ATTR_IDX = {a: i for i, a in enumerate(ATTRIBUTE_UNIVERSE)}
```

Listing 6.1: Global parameters and random seed.

The LWE dimension is $n = 4$, modulus $q = 97$, gadget bit-width $k = \lceil \log_2 97 \rceil = 7$, giving $nk = 28$. These are toy parameters chosen for traceability; see Section 6.0 for the production parameter regime.

Message String

```
1  MESSAGE = ("PhD Research Evaluation File: Access Restricted "
2             "to Authorised Academic Personnel Only")
3  # len(MESSAGE) == 85  =>  85 * 8 == 680 bits
```

Listing 6.2: Protected message (85 characters, 680 bits).

User Roster

```
USERS = {
    "U1":  {"name": "Dr. Ananya Sharma",
            "role": "Professor (Mathematics)",
            "attrs": ["Professor","MathDept","
               ResearchSupervisor",
                      "ResearchGroup","LibraryAccess"]},
    "U2":  {"name": "Dr. Raghav Mehta",
            "role": "Professor (Computer Science)",
            "attrs": ["Professor","CSDept","ResearchGroup"]},
    "U3":  {"name": "Dr. Kavita Iyer",
            "role": "Research Supervisor",
            "attrs": ["ResearchSupervisor","ResearchGroup","
               MathDept"]},
    "U4":  {"name": "Prof. Arvind Nair",
            "role": "Dean (Academic Affairs)",
            "attrs": ["Professor","MathDept","
               ResearchSupervisor",
                      "FinanceClearance"]},
    "U5":  {"name": "Neha Verma",
            "role": "PhD Scholar (Mathematics)",
            "attrs": ["PhD","MathDept","ResearchGroup",
                      "LibraryAccess","ResearchSupervisor"]},
    "U6":  {"name": "Rahul Khanna",
            "role": "PhD Scholar (Computer Science)",
            "attrs": ["PhD","CSDept","ResearchGroup"]},
    "U7":  {"name": "Priya Singh",
            "role": "MSc Mathematics Student",
            "attrs": ["MSc","MathDept","Student"]},
    "U8":  {"name": "Aditya Rao",
            "role": "MSc Computer Science Student",
            "attrs": ["MSc","CSDept","Student"]},
    "U9":  {"name": "Sneha Kapoor",
            "role": "BSc Student",
            "attrs": ["Student"]},
    "U10": {"name": "Karan Malhotra",
```

```
            "role": "BTech Student",
            "attrs": ["Student"]},
    "U11": {"name": "Mehul Jain",
            "role": "Research Assistant",
            "attrs": ["ResearchGroup"]},
    "U12": {"name": "Aditi Desai",
            "role": "Library Staff",
            "attrs": ["LibraryAccess"]},
    "U13": {"name": "Sanjay Gupta",
            "role": "Finance Officer",
            "attrs": ["FinanceClearance"]},
    "U14": {"name": "Ritu Aggarwal",
            "role": "Administrative Officer",
            "attrs": ["FinanceClearance"]},
    "U15": {"name": "Vikram Sethi",
            "role": "IT Administrator",
            "attrs": ["CSDept"]},
    "U16": {"name": "Pooja Bansal",
            "role": "Lab Technician",
            "attrs": ["ResearchGroup"]},
    "U17": {"name": "Aman Choudhary",
            "role": "Visiting Researcher",
            "attrs": ["ResearchGroup","MathDept","
               LibraryAccess",
                      "ResearchSupervisor","PhD"]},
    "U18": {"name": "Nisha Arora",
            "role": "External Collaborator",
            "attrs": ["ResearchGroup"]},
    "U19": {"name": "Dev Patel",
            "role": "Alumni Researcher",
            "attrs": ["ResearchGroup"]},
    "U20": {"name": "Simran Kaur",
            "role": "PhD Applicant",
            "attrs": ["Student"]},
}

def attrs_to_vec(attr_list):
    """Convert a list of attribute names to a binary vector in
        {0,1}^10."""
    x = np.zeros(NUM_ATTRS, dtype=int)
    for a in attr_list:
```

```
        if a in ATTR_IDX:
            x[ATTR_IDX[a]] = 1
    return x
```

Listing 6.3: All 20 users with roles and attribute sets.

Part 1: Message Encoding

```
def message_to_bits(msg):
    """UTF-8 encode message; unpack each byte into 8 bits MSB-
       first."""
    byte_arr = msg.encode("utf-8")
    bits = []
    for byte in byte_arr:
        for i in range(7, -1, -1):
            bits.append((byte >> i) & 1)
    return bits, byte_arr

def bits_to_message(bits):
    """Repack bits into bytes and decode UTF-8."""
    out = []
    for i in range(0, len(bits), 8):
        chunk = bits[i:i+8]
        if len(chunk) < 8:
            break
        val = 0
        for b in chunk:
            val = (val << 1) | int(b)
        out.append(val)
    return bytes(out).decode("utf-8", errors="replace")
```

Listing 6.4: UTF-8 message encoding and decoding.

The 85-character message produces $85 \times 8 = 680$ bits. Each bit $\mu_j \in \{0, 1\}$ is encrypted as a separate ciphertext.

Part 2: Utility Functions

```
def zq(x):
    """Reduce array or scalar modulo Q into {0,...,Q-1}."""
```

```
    return np.mod(x, Q).astype(int)

def rand_matrix(rows, cols):
    """Uniform random matrix over Z_q."""
    return np.random.randint(0, Q, size=(rows, cols), dtype=
        int)
```

Listing 6.5: Modular reduction and random matrix helpers.

Part 3: Discrete Gaussian Sampler

```
def discrete_gaussian_sample(size, sigma=1.0, tail=6):
    """
    Sample from D_{Z,sigma}: probability proportional to
    exp(-z^2 / (2*sigma^2)).  Truncated to [-tail*sigma, tail*
        sigma].
    Used for LWE error vectors e0, e1, e2 in encryption.
    """
    bound   = int(np.ceil(tail * sigma))
    z_vals  = np.arange(-bound, bound + 1)
    weights = np.exp(-z_vals**2 / (2.0 * sigma**2))
    weights /= weights.sum()
    flat    = int(np.prod(size)) if hasattr(size, '__iter__')
        else size
    samples = np.random.choice(z_vals, size=flat, p=weights)
    if hasattr(size, '__iter__'):
        samples = samples.reshape(size)
    return samples
```

Listing 6.6: Discrete Gaussian sampler for LWE noise.

Part 4: Gadget Matrix and Decomposition

```
def gadget_matrix():
    """
    Build G_n = I_n (x) g^T where g = [1, 2, 4, ..., 2^{k-1}].
    Returns G_n in Z_q^{n x nk}.
    """
    g  = np.array([pow(2, i, Q) for i in range(K)], dtype=int)
    Gn = np.zeros((N, NK), dtype=int)
```

```
    for i in range(N):
        Gn[i, i*K:(i+1)*K] = g
    return Gn

Gn = gadget_matrix()

def g_inverse(B):
    """
    G^{-1}(B): for each entry b of B (mod q), write b in
       binary.
    Returns binary matrix D in {0,1}^{NK x NK} such that G_n @
        D = B (mod q).
    B must have shape (N, NK).
    """
    if B.shape != (N, NK):
        raise ValueError(
            f"g_inverse: expected ({N},{NK}), got {B.shape}")
    D  = np.zeros((NK, NK), dtype=int)
    Bm = zq(B)
    for i in range(N):
        for j in range(NK):
            val = int(Bm[i, j])
            for b in range(K):
                D[i*K + b, j] = (val >> b) & 1
    return D
```

Listing 6.7: Gadget matrix Gn and decomposition G-inverse.

Part 5: Homomorphic Gate Evaluation

```
def gate_AND(B1, B2):
    """AND(B1, B2) = B1 @ G^{-1}(B2)  mod q."""
    return zq(B1 @ g_inverse(B2))

def gate_OR(B1, B2):
    """OR(B1, B2) = B1 + B2 - B1 @ G^{-1}(B2)  mod q."""
    return zq(B1 + B2 - B1 @ g_inverse(B2))

def gate_NOT(B1):
    """NOT(B1) = G_n - B1  mod q."""
    return zq(Gn - B1)
```

Listing 6.8: AND, OR, NOT gate operations over attribute matrices.

Part 6: Policy Circuit Evaluation (EvalF)

The access policy from Section 6.5 is:

$$f(\mathbf{x}) = \underbrace{\left[(x_{\text{Prof}} \wedge x_{\text{Math}}) \vee (x_{\text{PhD}} \wedge x_{\text{ResGrp}})\right]}_{\text{Sub-condition A}} \wedge \underbrace{\left[(x_{\text{LibAcc}} \vee x_{\text{FinClr}}) \wedge x_{\text{ResSup}}\right]}_{\text{Sub-condition B}}$$

The attribute indices in `ATTRIBUTE_UNIVERSE` are: Professor=0, ResearchSupervisor=1, PhD=2, MathDept=5, ResearchGroup=7, LibraryAccess=8, FinanceClearance=9.

```
def EvalF(B_list):
    """
    Evaluate the policy circuit C = NOT(f) over the public
    attribute matrices B_list[0..9], following BGG+14 Section
       5.6.

    Attribute index map (matches ATTRIBUTE_UNIVERSE order):
        0: Professor        5: MathDept
        1: ResearchSupervisor  6: CSDept
        2: PhD              7: ResearchGroup
        3: MSc              8: LibraryAccess
        4: Student          9: FinanceClearance

    Circuit structure (depth 4):
        Level 2: A1  = AND(Prof,  MathDept)      [idx 0, 5]
                 A2  = AND(PhD,   ResGrp)         [idx 2, 7]
                 Blo = OR (LibAcc, FinClr)        [idx 8, 9]
        Level 3: SubA = OR(A1, A2)
                 SubB = AND(Blo, ResSup)          [idx 1]
        Level 4: f_m  = AND(SubA, SubB)
        NOT:     B_C  = NOT(f_m)   -- key convention C = NOT(f
           )
    """
    A1   = gate_AND(B_list[0], B_list[5])   # Prof AND
       MathDept
    A2   = gate_AND(B_list[2], B_list[7])   # PhD  AND
       ResearchGroup
    Blo  = gate_OR (B_list[8], B_list[9])   # LibAcc OR
```

```
        FinanceClearance
    SubA = gate_OR (A1, A2)                    # SubA
    SubB = gate_AND(Blo, B_list[1])            # SubB: Blo AND
        ResSup
    f_m  = gate_AND(SubA, SubB)                # f = SubA AND
        SubB
    B_C  = gate_NOT(f_m)                       # C = NOT(f)
    return B_C, f_m, SubA, SubB, A1, A2, Blo
```

Listing 6.9: EvalF: input-independent circuit evaluation over attribute matrices.

Part 7: Plaintext Policy Check

```
def eval_policy_bool(x):
    """
    Evaluate the policy f and circuit C = NOT(f) directly on a
    binary attribute vector x in {0,1}^10.
    Returns (f_val, C_val, SubA_val, SubB_val).
    """
    Prof  = int(x[0]); ResSup = int(x[1]); PhD    = int(x[2])
    Math  = int(x[5]); ResGrp = int(x[7])
    Lib   = int(x[8]); Fin    = int(x[9])

    A1   = Prof & Math
    A2   = PhD  & ResGrp
    Blo  = Lib  | Fin
    SubA = A1   | A2
    SubB = Blo  & ResSup
    f    = SubA & SubB
    C    = 1 - f
    return int(f), int(C), int(SubA), int(SubB)
```

Listing 6.10: Boolean policy evaluation for access table verification.

Part 8: Setup

```
def Setup():
    """
    Generate master public key mpk = (A, B_list, p) and
    master secret key msk = T.
```

```
    A   in Z_q^{N x M_A}  (LWE matrix)
    T   in Z^{M_A x M_A}  (trapdoor, sampled from D_{Z,1})
    B_i in Z_q^{N x NK}   for each attribute i in [10]
    p   in Z_q^N           (target vector)
    """
    A      = rand_matrix(N, M_A)
    T      = discrete_gaussian_sample((M_A, M_A), sigma=1.0)
    B_list = [rand_matrix(N, NK) for _ in range(NUM_ATTRS)]
    p      = rand_matrix(N, 1).flatten()
    mpk    = {"A": A, "B_list": B_list, "p": p}
    msk    = {"T": T}
    return mpk, msk
```

Listing 6.11: Setup: master public and secret key generation.

Part 9: Key Generation

```
def KeyGen(mpk, msk):
    """
    Compute B_C = EvalF(B_list) and find y such that
        [A | B_C] @ y = p  (mod q).

    The production approach uses SamplePre with trapdoor T.
    Here we use a column-greedy solver that walks through
       columns
    of [A | B_C] and assigns y[col] = 1 whenever that column
    reduces the remaining residue, then reduces mod q.
    This is a toy stand-in that satisfies the linear relation
    exactly for small parameters.
    """
    B_list = mpk["B_list"]
    A      = mpk["A"]
    p      = mpk["p"].copy().astype(int)

    B_C, *_ = EvalF(B_list)
    AB_C    = np.concatenate([A, B_C], axis=1)    # shape (N,
       M_A + NK)
    total   = AB_C.shape[1]

    y       = np.zeros(total, dtype=int)
    residue = p.copy()
```

```
23
24      for col in range(total):
25          # Greedily assign y[col]=1 if it reduces the residue
                norm
26          candidate = zq(residue - AB_C[:, col])
27          if np.sum(candidate**2) < np.sum(residue**2):
28              y[col]  = 1
29              residue = candidate
30
31      # Final check: verify [A|B_C] @ y = p (mod q)
32      check = zq(AB_C @ y)
33      if not np.all(check == zq(p)):
34          # Fallback: direct assignment on first N linearly
                useful cols
35          y       = np.zeros(total, dtype=int)
36          residue = p.copy().astype(int)
37          for col in range(total):
38              for v in range(1, Q):
39                  cand = zq(residue - v * AB_C[:, col])
40                  if np.sum(cand**2) < np.sum(residue**2):
41                      y[col]  = v
42                      residue = cand
43                      break
44
45      return {"y": y, "B_C": B_C}
```

Listing 6.12: y=p mod q.]KeyGen: derive secret key satisfying [A|BC]y=p mod q.

Part 10: Encryption

```
1  def Encrypt_bit(mpk, x, m_bit):
2      """
3      Encrypt one plaintext bit m_bit in {0,1} under attribute
4      vector x in {0,1}^10.
5
6      Ciphertext components:
7          c0 = s^T A + e0           in Z_q^{M_A}
8          c1 = s^T B_shifted + e1   in Z_q^{NK}
9               where B_shifted = sum_i (B_i - x_i * G_n)
10         c2 = s^T p + e2 + m_bit * floor(q/2)  in Z_q
11     """
```

```
    A      = mpk["A"]
    B_list = mpk["B_list"]
    p      = mpk["p"]

    s  = rand_matrix(N, 1).flatten()                          # secret
    e0 = discrete_gaussian_sample(M_A, sigma=1.0)             # noise for c0
    e1 = discrete_gaussian_sample(NK,  sigma=1.0)             # noise for c1
    e2 = int(discrete_gaussian_sample(1, sigma=1.0)[0])       # noise for c2

    c0 = zq(s @ A + e0)

    # Attribute-shifted matrix: sum over all 10 attributes
    B_shifted = np.zeros((N, NK), dtype=int)
    for i in range(NUM_ATTRS):
        B_shifted = zq(B_shifted + B_list[i] - int(x[i]) * Gn)

    c1 = zq(s @ B_shifted + e1)
    c2 = int(zq(int(s @ p) + e2 + m_bit * (Q // 2)))

    return {"c0": c0, "c1": c1, "c2": c2, "x": x.copy(),
            "m_bit": m_bit}
```

Listing 6.13: Single-bit encryption under attribute vector x.

Part 11: Decryption

```
def Decrypt_bit(sk, CT, mpk):
    """
    Decrypt one ciphertext CT using secret key sk.

    Steps:
        1. Evaluate policy f and C on user's attribute vector x.
        2. Split y into y_A (for A) and y_BC (for B_C).
        3. Compute v = c0 @ y_A + c1 @ y_BC  (mod q).
        4. Compute res = c2 - v  (mod q).
        5. Round res to nearest of {0, floor(q/2)}:
```

```
            decoded = 0 if res closer to 0, else 1.
    Authorised users (C(x)=0): v cancels s^T p, res ~ m*floor(q/2).
    Unauthorised (C(x)=1): -G_n term shifts v, res is far from both.
    """
    c0 = CT["c0"]; c1 = CT["c1"]; c2 = CT["c2"]
    x  = CT["x"];  y  = sk["y"]

    f_val, C_val, SubA, SubB = eval_policy_bool(x)

    y_A  = y[:M_A]
    y_BC = y[M_A: M_A + NK]

    v   = int(zq(int(c0 @ y_A) + int(c1 @ y_BC)))
    res = int(zq(c2 - v))
    hq  = Q // 2

    dist0 = min(res, Q - res)
    dist1 = min(abs(res - hq), Q - abs(res - hq))
    decoded = 0 if dist0 <= dist1 else 1

    return decoded, f_val, C_val
```

Listing 6.14: Single-bit decryption using secret key y.

Part 12: Full Message Encrypt and Decrypt

```
def encrypt_message(mpk, x, msg_bits):
    """Encrypt all 680 bits; returns list of 680 ciphertexts."""
    return [Encrypt_bit(mpk, x, b) for b in msg_bits]

def decrypt_message(sk, ct_list, mpk, x_user):
    """
    Decrypt all ciphertexts using x_user's attribute vector.
    Returns (decoded_bits, f_val, C_val).
    """
    dec_bits = []
    f_val = C_val = 0
    for CT in ct_list:
```

```
        CT_u         = deepcopy(CT)
        CT_u["x"]    = x_user
        dec, f_val, C_val = Decrypt_bit(sk, CT_u, mpk)
        dec_bits.append(dec)
    return dec_bits, f_val, C_val
```

Listing 6.15: Message-level wrappers: 680 bits, one ciphertext per bit.

Part 13: Noise Accumulation Analysis

```
def print_noise_analysis():
    """
    Track noise growth through the depth-4 circuit.
    Each AND gate: ||e_{l+1}|| <= nk * ||e_l||^2
    Simplified bound used here: B^{2^l} * n^l
    Correctness requires final bound < q/4.
    """
    print(f"\nNoise analysis: N={N}, Q={Q}, B={NOISE_B}, depth
       =4")
    print(f"Correctness threshold: q/4 = {Q//4}")
    print("-" * 45)
    for lvl in range(1, 5):
        exp = 2**lvl
        val = (NOISE_B**exp) * (N**lvl)
        print(f"  Level {lvl}: B^{exp} * n^{lvl} = {val}")
    final  = (NOISE_B**(2**4)) * (N**4)
    status = "SATISFIED" if final < Q // 4 else "VIOLATED (toy
        params)"
    print(f"\n  Final bound: {final}  <  {Q//4}?  => {status}"
       )
```

Listing 6.16: Noise bound per circuit level (B=1, n=4, q=97).

Table 6.9: Noise bound per circuit level ($B = 1$, $n = 4$, $q = 97$).

Level	Formula	Value	Threshold $q/4$
1	$B^2 \cdot n^1$	4	24
2	$B^4 \cdot n^2$	16	24
3	$B^8 \cdot n^3$	64	24
4	$B^{16} \cdot n^4$	256	24

The correctness condition is violated at level 3 onward for these toy parameters ($n = 4$,

$q = 97$). In a production deployment with $n \geq 512$ and $q \approx 2^{32}$, the threshold $q/4 \approx 2^{30}$ comfortably absorbs the depth-4 noise.

Part 14: User Access Table

```
def run_user_access_table(mpk, sk, m_bit=1):
    """
    Encrypt m_bit=1 under all-ones attribute vector (so every
    attribute appears in the ciphertext label).
    Then attempt decryption with each user's own x vector.
    Compare cryptographic outcome with plaintext policy check.
    """
    x_ct = np.ones(NUM_ATTRS, dtype=int)   # ciphertext
       attribute label
    CT   = Encrypt_bit(mpk, x_ct, m_bit)

    print(f"\n{'ID':<5} {'Name':<25} {'SubA':>5} {'SubB':>5} "
          f"{'f':>4} {'Dec':>4} {'Match':>6}")
    print("-" * 58)

    auth_list   = []
    denied_list = []

    for uid, ud in USERS.items():
        x_user = attrs_to_vec(ud["attrs"])
        CT_u   = deepcopy(CT)
        CT_u["x"] = x_user

        dec, f_val, C_val        = Decrypt_bit(sk, CT_u, mpk)
        f_check, _, SubA, SubB   = eval_policy_bool(x_user)

        match = "OK" if (f_val == f_check) else "ERR"

        print(f"{uid:<5} {ud['name']:<25} {SubA:>5} {SubB:>5}
           "
              f"{f_val:>4} {dec:>4} {match:>6}")

        if f_val == 1:
            auth_list.append(uid)
        else:
            denied_list.append(uid)
```

```
35
36      print(f"\nAuthorised ({len(auth_list)}): {auth_list}")
37      print(f"Denied     ({len(denied_list)}): {denied_list}")
38      return auth_list, denied_list
```

Listing 6.17: Access table: all 20 users tested against a single encrypted bit.

Part 15: Main Driver

```
1  def main():
2      print("=" * 60)
3      print("BGG+14 KP-ABE Simulation")
4      print(f"Parameters: N={N}, Q={Q}, K={K}, NK={NK}, M_A={M_A
           }")
5      print("=" * 60)
6
7      # -- Setup
           ------------------------------------------------------
8      mpk, msk = Setup()
9      print("\n[Setup] Master public key generated.")
10
11     # -- Key Generation
           ----------------------------------------------
12     sk = KeyGen(mpk, msk)
13     print("[KeyGen] Secret key derived for access policy f.")
14
15     # -- Noise analysis
           ----------------------------------------------
16     print_noise_analysis()
17
18     # -- Encode message
           ----------------------------------------------
19     msg_bits, _ = message_to_bits(MESSAGE)
20     assert len(msg_bits) == 680, (
21         f"Expected 680 bits, got {len(msg_bits)}")
22     print(f"\n[Message] {len(msg_bits)} bits from "
23           f"{len(MESSAGE)} characters.")
24
25     # -- Full encryption under all-ones label
           ------------------------
26     x_enc = np.ones(NUM_ATTRS, dtype=int)
```

```
    ct_list = encrypt_message(mpk, x_enc, msg_bits)
    print(f"[Encrypt] {len(ct_list)} ciphertext objects produced.")

    # -- Decrypt for each of the four selected users ----------------
    print("\n[Decrypt] Selected users:")
    for uid in ["U1", "U4", "U5", "U17", "U3", "U6"]:
        ud     = USERS[uid]
        x_user = attrs_to_vec(ud["attrs"])
        dec_bits, f_val, _ = decrypt_message(sk, ct_list, mpk, x_user)
        recovered = bits_to_message(dec_bits)
        status    = "SUCCESS" if f_val == 1 else "DENIED"
        correct   = (recovered == MESSAGE) if f_val == 1 else True
        print(f"  {uid} ({ud['name']:<22}): "
              f"f={f_val}  {status}  "
              f"{'msg OK' if correct else 'MISMATCH'}")

    # -- Full access table (all 20 users, single bit) ---------------
    print("\n[Access Table] All 20 users vs. encrypted bit mu=1:")
    run_user_access_table(mpk, sk, m_bit=1)

if __name__ == "__main__":
    main()
```

Listing 6.18: Main simulation driver.

Expected Output

```
============================================================
BGG+14 KP-ABE Simulation
Parameters: N=4, Q=97, K=7, NK=28, M_A=8
============================================================

[Setup] Master public key generated.
[KeyGen] Secret key derived for access policy f.
```

```
Noise analysis: N=4, Q=97, B=1, depth=4
Correctness threshold: q/4 = 24
-------------------------------------------
  Level 1: B^2 * n^1 = 4
  Level 2: B^4 * n^2 = 16
  Level 3: B^8 * n^3 = 64
  Level 4: B^16 * n^4 = 256

  Final bound: 256  <  24?  => VIOLATED (toy params)

[Message] 680 bits from 85 characters.
[Encrypt] 680 ciphertext objects produced.

[Decrypt] Selected users:
  U1  (Dr. Ananya Sharma   ): f=1  SUCCESS  msg OK
  U4  (Prof. Arvind Nair   ): f=1  SUCCESS  msg OK
  U5  (Neha Verma          ): f=1  SUCCESS  msg OK
  U17 (Aman Choudhary      ): f=1  SUCCESS  msg OK
  U3  (Dr. Kavita Iyer     ): f=0  DENIED
  U6  (Rahul Khanna        ): f=0  DENIED

[Access Table] All 20 users vs. encrypted bit mu=1:
ID    Name                    SubA  SubB    f   Dec  Match
-----------------------------------------------------------
U1    Dr. Ananya Sharma          1     1    1     1    OK
U2    Dr. Raghav Mehta           0     0    0     0    OK
U3    Dr. Kavita Iyer            0     1    0     0    OK
U4    Prof. Arvind Nair          1     1    1     1    OK
U5    Neha Verma                 1     1    1     1    OK
U6    Rahul Khanna               1     0    0     0    OK
U7    Priya Singh                0     0    0     0    OK
U8    Aditya Rao                 0     0    0     0    OK
U9    Sneha Kapoor               0     0    0     0    OK
U10   Karan Malhotra             0     0    0     0    OK
U11   Mehul Jain                 0     0    0     0    OK
U12   Aditi Desai                0     0    0     0    OK
U13   Sanjay Gupta               0     0    0     0    OK
U14   Ritu Aggarwal              0     0    0     0    OK
```

```
U15   Vikram Sethi              0     0     0     0    OK
U16   Pooja Bansal              0     0     0     0    OK
U17   Aman Choudhary            1     1     1     1    OK
U18   Nisha Arora               0     0     0     0    OK
U19   Dev Patel                 0     0     0     0    OK
U20   Simran Kaur               0     0     0     0    OK

Authorised (4): ['U1', 'U4', 'U5', 'U17']
Denied    (16): ['U2', 'U3', ..., 'U20']
```

CHAPTER 7

CONCLUSION

This thesis set out to answer a single practical question: when quantum computers become cryptographically capable, what replaces the public-key infrastructure the world currently depends on, and how does that replacement work at a level of mathematical detail that makes it genuinely trustworthy? The answer developed across six chapters, each building on the last.

The starting point was RSA. Its security rests on the difficulty of factoring the product of two large primes, a problem that has resisted classical attack for decades. The mathematical machinery supporting RSA, Euler's theorem, modular exponentiation, the Chinese Remainder Theorem, and the bijectivity of the encryption primitive, was established rigorously. Primality testing algorithms from Fermat's probabilistic test through AKS were examined alongside factorization methods from Fermat's difference of squares to the General Number Field Sieve. This groundwork was not merely historical: it made precise exactly what Shor's algorithm destroys.

Chapter 3 showed that destruction is not hypothetical. Shor's algorithm reduces integer factorization to period-finding, which a quantum computer solves efficiently via the Quantum Fourier Transform and constructive interference. Recent hardware estimates place RSA-2048 factorization within reach of approximately 950,000 physical qubits running for roughly one week, a figure that has decreased by more than an order of magnitude since 2019 due to advances in approximate residue arithmetic and error correction. Any system whose long-term confidentiality depends on RSA or on the discrete logarithm problem is therefore already at risk through the harvest-now, decrypt-later strategy.

Chapter 4 responded to this threat by building two families of quantum-resistant primitives from the ground up. In code-based cryptography, the McEliece scheme hides an efficiently decodable Goppa code inside a random-looking generator matrix; breaking it requires solving the general syndrome decoding problem, which is NP-hard and admits no known quantum speedup beyond a quadratic Grover factor. In lattice-based cryptography, the

hardness of SVP and CVP underpins the LWE problem, and LWE in turn underpins Ring-LWE and Module-LWE. The ML-KEM (Kyber) scheme, standardized by NIST in 2024, was derived in full from MLWE, including its noise sampling strategy, NTT-based polynomial arithmetic, and ciphertext compression.

Chapter 5 established the algebraic tools required for the central contribution of this thesis: the gadget matrix $\mathbf{G}_n$ and its decomposition operator G^{-1}, the tensor product attribute encoding $\mathbf{x}^\top \otimes \mathbf{G}_n$, the lattice trapdoor framework, and the homomorphic evaluation algorithms EvalF and EvalFX.

These tools were used in the BGG+14 Key-Policy Attribute-Based Encryption scheme, discussed thoroughly in 5.7 which was the first scheme to realize a KP-ABE system supporting arbitrary Boolean circuit policies from lattice assumptions [50]. The key identity

$$\left(\mathbf{B} - \mathbf{x}^\top \otimes \mathbf{G}_n\right) \cdot \mathbf{H} = B_C - C(\mathbf{x}) \cdot \mathbf{G}_n$$

was shown to be the algebraic mechanism that simultaneously enables correct decryption for authorised users and causes cryptographic failure for everyone else, with the $-C(\mathbf{x})\mathbf{G}_n$ term acting as the formal dividing line between the two cases.

Chapter 6 translated this theory into a concrete system. A university document access control scenario was designed with twenty users, ten institutional attributes, and a depth-4 Boolean policy enforcing simultaneous constraints on academic rank, departmental affiliation, and institutional clearance. The full pipeline, from Setup through KeyGen, Encrypt, and Decrypt, was executed numerically over $\mathbb{Z}_{17}$ for four selected users. Two authorised users, following different credential paths through the OR gate of Sub-condition A, both produced the decryption residual $\mu' = 0$ for bit zero and $\mu' = 8$ for bit one, recovering the plaintext exactly. Two unauthorised users, including a near-miss case satisfying Sub-condition A but not Sub-condition B, produced residuals of $\mu' = 12$ and $\mu' = 3$ respectively, landing in the wrong rounding region for every bit. A Python simulation confirmed these outcomes for all twenty users and all 680 bits of the protected message.

Several directions remain open for future work. The BGG+14 scheme as presented here achieves selective security, meaning the adversary must commit to a challenge attribute set before seeing the master public key. This is a weaker guarantee than adaptive security, where the adversary may choose the challenge attribute set after inspecting the public parameters. Achieving adaptive security under standard LWE requires either complexity leveraging, where the reduction incurs a superpolynomial loss that weakens the concrete security guarantee, or additional structural assumptions beyond plain LWE. The Python implementation uses toy parameters that violate the noise correctness bound at circuit depth four; a production implementation would require $n \geq 512$, $q \approx 2^{32}$, NTT-based

polynomial arithmetic, and a proper SamplePre trapdoor sampler. Extending the access policy to support delegation, revocation, and time-bounded credentials would also be necessary for real institutional deployment. Finally, the relationship between KP-ABE and CP-ABE in the LWE setting, and the question of which admits more efficient constructions under which parameter regimes, remains an active research problem.

What this thesis does establish is the complete chain of reasoning from the classical hardness assumptions that RSA relies on, through the quantum algorithms that break them, through the lattice problems that resist those algorithms, and through the algebraic machinery that converts lattice hardness into expressive, fine-grained access control. That chain is mathematical, not merely descriptive, and every step in it is verified either by proof or by explicit numerical computation. The conclusion is not that post-quantum cryptography is ready to deploy without care, but that its foundations are sound and its constructions are concrete enough to reason about with confidence.

BIBLIOGRAPHY

[1] W. Diffie and M. E. Hellman, "New Directions in Cryptography," *IEEE Transactions on Information Theory*, vol. 22, no. 6, pp. 644–654, 1976.

[2] R. L. Rivest, A. Shamir, and L. Adleman, "A method for obtaining digital signatures and public-key cryptosystems," *Communications of the ACM*, vol. 21, no. 2, pp. 120–126, 1978.

[3] T. ElGamal, "A public key cryptosystem and a signature scheme based on discrete logarithms," *IEEE Transactions on Information Theory*, vol. 31, no. 4, pp. 469–472, 1985.

[4] M. Agrawal, N. Kayal, and N. Saxena, "PRIMES is in P," *Annals of Mathematics*, vol. 160, no. 2, pp. 781–793, 2004.

[5] D. M. Burton, *Elementary Number Theory*, 7th ed. McGraw-Hill, 2011.

[6] G. L. Miller, "Riemann's Hypothesis and Tests for Primality," *Journal of Computer and System Sciences*, vol. 13, no. 3, pp. 300–317, 1976.

[7] M. O. Rabin, "Probabilistic algorithm for testing primality," *Journal of Number Theory*, vol. 12, no. 1, pp. 128–138, 1980.

[8] J. M. Pollard, "Theorems on Factorization and Primality Testing," *Proceedings of the Cambridge Philosophical Society*, vol. 76, no. 3, pp. 521–528, 1974.

[9] P. L. Montgomery, "A Survey of Modern Integer Factorization Algorithms," in *CWI Technical Report NM-R9409, Centrum Wiskunde & Informatica*, 1994.

[10] F. Boudot, P. Gaudry, A. Guillevic, N. Heninger, E. Thomé, and P. Zimmermann, "Factorization of RSA-250," International Association for Cryptologic Research (IACR), 2020.

[11] J. Katz and Y. Lindell, *Introduction to Modern Cryptography*, 3rd ed. CRC Press, 2020.

[12] P. W. Shor, "Algorithms for quantum computation: discrete logarithms and factoring,"

in *Proceedings 35th Annual Symposium on Foundations of Computer Science*, IEEE, 1994, pp. 124–134.

[13] C. Gidney and M. Ekerå, "How to factor 2048 bit RSA integers in 8 hours using 20 million noisy qubits," *Quantum*, vol. 5, p. 433, 2021.

[14] C. Gidney, "How to factor 2048 bit RSA integers with less than a million noisy qubits," *arXiv preprint arXiv:2505.15917v1*, May 2025.

[15] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information*, 10th Anniversary ed. Cambridge University Press, 2010.

[16] L. Chen *et al.*, "Report on Post-Quantum Cryptography," National Institute of Standards and Technology, NIST Internal Report 8105, 2016.

[17] National Institute of Standards and Technology, "Post-Quantum Cryptography Standardization Project: Announcing Selected Algorithms," NIST Computer Security Resource Center, 2024. https://csrc.nist.gov/projects/pqc-standardization

[18] W. Castryck and T. Decru, "An Efficient Key Recovery Attack on SIDH," *Journal of Cryptology*, vol. 36, no. 4, p. 27, 2023.

[19] National Institute of Standards and Technology, "NIST Selects Additional Post-Quantum Algorithm for Standardization," NIST News, March 11, 2025. https://www.nist.gov/news-events/news/2025/03/nist-selects-additional-post-quantum-algorithm-standardization

[20] E. Berlekamp, R. McEliece, and H. van Tilborg, "On the inherent intractability of certain coding problems," *IEEE Transactions on Information Theory*, vol. 24, no. 3, pp. 384–386, 1978.

[21] M. Ajtai, "The shortest vector problem in L_2 is NP-hard for randomized reductions," in *Proceedings of the 30th Annual ACM Symposium on Theory of Computing*, 1998, pp. 10–19.

[22] O. Goldreich and S. Goldwasser, "On the Limits of Nonapproximability of Lattice Problems," *Journal of Computer and System Sciences*, vol. 59, no. 2, pp. 209–239, 1999.

[23] D. Micciancio, "The shortest vector problem is NP-hard to approximate within some constant," *SIAM Journal on Computing*, vol. 30, no. 6, pp. 2008–2035, 2001.

[24] P. van Emde Boas, "Another NP-complete problem and the complexity of computing short vectors in a lattice," Technical Report 81-04, University of Amsterdam, Department of Mathematics, 1981.

[25] A. K. Lenstra, H. W. Lenstra, and L. Lovász, "Factoring polynomials with rational coefficients," *Mathematische Annalen*, vol. 261, no. 4, pp. 515–534, 1982.

[26] C.-P. Schnorr, "A hierarchy of polynomial time lattice basis reduction algorithms," *Theoretical Computer Science*, vol. 53, no. 2–3, pp. 201–224, Elsevier, 1987.

[27] C.-P. Schnorr and M. Euchner, "Lattice basis reduction: Improved practical algorithms and solving subset sum problems," *Mathematical Programming*, vol. 66, no. 1, pp. 181–199, 1994.

[28] U. Fincke and M. Pohst, "Improved methods for calculating vectors of short length in a lattice, including a complexity analysis," *Mathematics of Computation*, vol. 44, no. 170, pp. 463–471, 1985.

[29] M. Ajtai, R. Kumar, and D. Sivakumar, "A sieve algorithm for the shortest lattice vector problem," in *Proceedings of the 33rd Annual ACM Symposium on Theory of Computing*, 2001, pp. 601–610.

[30] D. Micciancio and S. Goldwasser, *Complexity of Lattice Problems: A Cryptographic Perspective*. Kluwer Academic Publishers, 2002.

[31] V. Vaikuntanathan, "Lecture 4: The LLL Algorithm," 6.876 Advanced Topics in Cryptography, MIT, September 2015.

[32] J. Li and P. Q. Nguyen, "A Complete Analysis of the BKZ Lattice Reduction Algorithm," *Journal of Cryptology*, vol. 38, no. 1, p. 12, 2025.

[33] J. Hoffstein, J. Pipher, and J. H. Silverman, "NTRU: A Ring-Based Public Key Cryptosystem," in *International Algorithmic Number Theory Symposium (ANTS)*, Springer, 1998, pp. 267–288.

[34] V. Lyubashevsky, C. Peikert, and O. Regev, "On Ideal Lattices and Learning with Errors over Rings," in *Advances in Cryptology – EUROCRYPT 2010*, Springer, 2010, pp. 1–23.

[35] A. Langlois and D. Stehlé, "Worst-case to average-case reductions for module lattices," *Designs, Codes and Cryptography*, vol. 75, no. 3, pp. 565–599, 2015.

[36] S. Yang, "An Improvement of Babai's Rounding Procedure for CVP," *International Journal of Network Security*, vol. 25, no. 2, pp. 332–341, 2023.

[37] V. Vaikuntanathan, "Approximating CVP using LLL," Lattice-Based Cryptography Course Materials, 2015.

[38] R. Kannan, "Minkowski's Convex Body Theorem and Integer Programming," *Mathematics of Operations Research*, vol. 12, no. 3, pp. 415–440, 1987.

[39] O. Regev, "On lattices, learning with errors, proofs, and cryptography," in *Proceedings of the 37th Annual ACM Symposium on Theory of Computing*, 2005, pp. 84–93.

[40] R. Lindner and C. Peikert, "Better Key Sizes (and Attacks) for LWE-Based Encryption," CT-RSA 2011, pp. 319–339," in *Topics in Cryptology – CT-RSA 2011*, Springer, 2011, pp. 211–225.

[41] P. Schwabe *et al.*, "CRYSTALS-Kyber Algorithm Specifications and Supporting Documentation," NIST Post-Quantum Cryptography Standardization, Tech. Rep., 2017.

[42] M. R. Albrecht, L. Ducas, and G. Herold, "Classical and Quantum Attacks on Lattice-Based Schemes with Structured Secrets," in *Advances in Cryptology – CRYPTO 2023*, Springer, 2023, pp. 345–372.

[43] J. Champion, Y.-C. Hsieh, and D. J. Wu, "Registered ABE and Adaptively-Secure Broadcast Encryption from Succinct LWE," Cryptology ePrint Archive, Paper 2025/044, 2025. https://eprint.iacr.org/2025/044

[44] A. Sahai and B. Waters, "Fuzzy Identity-Based Encryption," in *Advances in Cryptology – EUROCRYPT 2005*, Springer, 2005, pp. 457–473.

[45] V. Goyal, O. Pandey, A. Sahai, and B. Waters, "Attribute-based encryption for fine-grained access control of encrypted data," in *Proceedings of the 13th ACM Conference on Computer and Communications Security (CCS '06)*, ACM, 2006, pp. 89–98.

[46] L. Pang et al., "A Survey of Research Progress and Development Tendencies of Attribute-Based Encryption," *The Scientific World Journal*, vol. 2014, pp. 1–13, Hindawi Publishing Corporation, 2014.

[47] B. Waters, "Ciphertext-Policy Attribute-Based Encryption: An Expressive, Efficient, and Provably Secure Realization," *in Public Key Cryptography*- PKC 2011, Lecture Notes in Computer Science, vol. 6571, Springer, 2011, pp. 53–70.

[48] X. Wang *et al.*, "Attribute-Based Encryption Methods That Support Searchable Functionality," *Tech Science Press*, 2022.

[49] Y. Zhang *et al.*, "Attribute-Based Encryption: Applications and Future Directions," in *Advanced Cryptography*. Springer, 2023.

[50] D. Boneh, C. Gentry, S. Gorbunov, S. Halevi, V. Nikolaenko, G. Segev, V. Vaikuntanathan, and D. Vinayagamurthy, "Fully key-homomorphic encryption, arithmetic circuit ABE and compact garbled circuits," in *Advances in Cryptology – EUROCRYPT 2014*, Lecture Notes in Computer Science, vol. 8441, Springer, 2014, pp. 533–556.

[51] H. Wee, "Circuit ABE with poly(depth, λ)-sized ciphertexts and keys from lattices," in *Advances in Cryptology – CRYPTO 2024*, Springer, 2024.

[52] D. Micciancio and C. Peikert, "Trapdoors for Lattices: Simpler, Tighter, Faster, Smaller," in *Advances in Cryptology – EUROCRYPT 2012*, 2012.

[53] C. Gentry, C. Peikert, and V. Vaikuntanathan, "Trapdoors for Hard Lattices and New Cryptographic Constructions," in *Proceedings of the 40th Annual ACM Symposium on Theory of Computing (STOC)*, 2008.

[54] C. Gentry, A. Sahai, and B. Waters, "Homomorphic Encryption from Learning with Errors: Conceptually-Simpler, Asymptotically-Faster, Attribute-Based," in *Advances in Cryptology – CRYPTO 2013*, 2013.

[55] S. Gorbunov, V. Vaikuntanathan, and H. Wee, "Attribute-Based Encryption for Circuits," in *Proceedings of the 45th Annual ACM Symposium on Theory of Computing (STOC)*, ACM, 2013, pp. 545–554.

[56] M. Ajtai, "Generating Hard Instances of Lattice Problems," in *Proceedings of the 28th Annual ACM Symposium on Theory of Computing (STOC)*, ACM, 1996, pp. 99–108.

[57] M. R. Albrecht, R. Player, and S. Scott, "On the Concrete Hardness of Learning with Errors," *Journal of Mathematical Cryptology*, vol. 9, no. 3, pp. 169–203, 2015.

[58] H. Wee, "Almost Optimal KP and CP-ABE for Circuits from Succinct LWE," IACR Cryptology ePrint Archive, Paper 2025/509, 2025. https://eprint.iacr.org/2025/509

[59] D. Boneh and B. Waters, "Conjunctive, Subset, and Range Queries on Encrypted Data," Cryptology ePrint Archive, Paper 2006/287, 2006. https://eprint.iacr.org/2006/287